



UNIVERSIDAD
DE MÁLAGA

Escuela Técnica Superior
de Ingeniería Informática

Máster Universitario en Ciberseguridad

Curso 2025-2026

Trabajo Fin de Máster

**EVALUACIÓN DE SEGURIDAD EN ECOSISTEMAS
WIDEVINE DRM: DISEÑO DE UNA
ARQUITECTURA DE DEFENSA EN PROFUNDIDAD
PARA EL *STREAMING* EN DIRECTO**

Autor: Jesús Ramírez Gálvez

Tutor: Antonio Muñoz Gallego

Licencia de la memoria

© 2026 Jesús Ramírez Gálvez.

Salvo que se indique lo contrario, el texto y las figuras de elaboración propia de esta memoria se distribuyen bajo una licencia

Creative Commons Atribución 4.0 Internacional (CC BY 4.0).

Se permite compartir y adaptar la obra, incluso con fines comerciales, siempre que se reconozca adecuadamente la autoría, se incluya un enlace a la licencia y se indiquen los cambios realizados.

El código fuente del proyecto se distribuye por separado bajo la licencia GNU GPL v3. Los logotipos, marcas, contenidos, capturas y demás materiales pertenecientes a terceros conservan sus derechos y condiciones de uso originales.

Resumen

La distribución de contenidos audiovisuales, especialmente en directo, se apoya cada vez más en plataformas *Over-The-Top* (OTT) y sistemas de gestión de derechos digitales. Widevine DRM ocupa una posición destacada por su integración en Android y navegadores basados en Chromium. Sin embargo, la protección criptográfica de las claves no cubre por sí sola toda la cadena de confianza. La autenticación, la autorización, la CDN y las sesiones también pueden exponer vías de redistribución no autorizada, como el *Key Leak*, el *CDN leeching* y el *restreaming*.

Este Trabajo Fin de Máster diseña, implementa y audita un entorno experimental de *streaming* para estudiar el papel de Widevine en una arquitectura de defensa en profundidad. La Fase 0 establece una línea base protegida con Widevine y Common Encryption (CENC) y examina la exposición del manifiesto, el contenido y las licencias. La Fase 1 incorpora autenticación, autorización por activo, tokens de reproducción efímeros, validación contextual y control de concurrencia. La Fase 2 añade estado compartido en Redis, eventos, puntuación de riesgo y mecanismos de bloqueo y revocación.

La metodología incremental e iterativa combina el diseño, el despliegue con Docker y experimentos controlados. La implementación integra Varnish, Node.js, Nginx, Redis y Shaka Player, junto con laboratorios de auditoría, filtración de claves y uso externo de la CDN. La evaluación reúne pruebas automatizadas, reproducciones en navegador, mediciones temporales y evidencias PCAP, JSON y capturas. La memoria documenta así el flujo DRM y las capas adicionales aplicadas a su alrededor como base para estudiar la madurez de la protección en plataformas OTT.

Palabras clave:

- *Streaming* OTT.
- Widevine DRM.

- Defensa en profundidad.
- Control de acceso.
- *CDN leeching*.
- Detección de anomalías.

Abstract

The distribution of audiovisual content, particularly live broadcasts, increasingly relies on Over-The-Top (OTT) platforms and digital rights management systems. Widevine DRM holds a prominent position due to its integration with Android and Chromium-based browsers. However, the cryptographic protection of content keys does not cover the entire chain of trust on its own. Authentication, authorization, CDN delivery, and session control can also expose avenues for unauthorized redistribution, including Key Leak, CDN leeching, and restreaming.

This Master's Thesis designs, implements, and audits an experimental streaming environment to study the role of Widevine in a defense-in-depth architecture. Phase 0 establishes a baseline protected with Widevine and Common Encryption (CENC) and examines the exposure of the manifest, content, and licenses. Phase 1 introduces authentication, per-asset authorization, short-lived playback tokens, contextual validation, and concurrency control. Phase 2 adds shared state in Redis, events, risk scoring, and blocking and revocation mechanisms.

The incremental and iterative methodology combines design, Docker deployment, and controlled experiments. The implementation integrates Varnish, Node.js, Nginx, Redis, and Shaka Player with laboratories for auditing, key leakage, and external CDN usage. The evaluation comprises automated tests, browser playback, timing measurements, and PCAP, JSON, and screenshot evidence. The thesis thus documents the DRM flow and its additional protective layers as a basis for studying security maturity in OTT platforms.

Keywords:

- OTT streaming.
- Widevine DRM.
- Defense in depth.
- Access control.
- CDN leeching.

- Anomaly detection.

Agradecimientos

El primer agradecimiento se lo debo a mi familia, por su apoyo incondicional y por estar siempre a mi lado en los momentos difíciles. A mis padres, por enseñarme el valor del esfuerzo y la perseverancia, y a mi hermano, por su constante ánimo y compañía.

Seguidamente, el máster ha sido más ameno gracias a los compañeros, el famoso *Club de Debate*, con quienes he compartido trabajos, risas y bastantes anécdotas que han marcado esta etapa en Málaga. El acceso al máster estuvo acompañado de numerosas dificultades burocráticas, pero he podido superarlas y alcanzar el objetivo.

También agradezco la ayuda y compañía de mi amiga Elena, que ha estado presente en los momentos de estrés y ha sabido darme buenos consejos para afrontar los retos del máster, además de ser una gran compañía para vivir en esta ciudad nueva para mí.

Por último, agradezco a mi empresa, Tarlogic, que me ha permitido compaginar el trabajo con el máster, brindándome flexibilidad y apoyo en mi desarrollo profesional, además de incluir el máster en mi plan de formación.

Índice de contenidos

Resumen	III
Abstract	V
Agradecimientos	VIII
Glosario de términos y acrónimos	XXIII
1. Introducción	1
1.1. Contexto y motivación	1
1.1.1. Crecimiento del <i>streaming</i> OTT	1
1.1.2. Importancia del DRM	2
1.1.3. Widevine como estándar dominante	2
1.1.4. Limitaciones de asumir que tener DRM implica seguridad .	3
1.2. Problema de investigación	3
1.2.1. Qué protege realmente Widevine	3
1.2.2. Qué no protege Widevine	4
1.2.3. Amenazas persistentes en ecosistemas OTT	4
1.3. Hipótesis de trabajo	5
1.4. Objetivos	6
1.4.1. Objetivo general	6

1.4.2.	Objetivos específicos	6
1.5.	Contribuciones	7
1.6.	Metodología	7
1.6.1.	Metodología incremental	7
1.6.2.	Laboratorio experimental	8
1.6.3.	Auditoría técnica	8
1.6.4.	Diseño iterativo de defensas	8
1.7.	Estructura de la memoria	9
2.	Fundamentos teóricos y estado del arte	10
2.1.	<i>Streaming</i> OTT	10
2.1.1.	Arquitectura OTT	10
2.1.2.	<i>Live streaming</i>	11
2.1.3.	CDN	12
2.1.4.	HLS y MPEG-DASH	12
2.2.	Sistemas DRM	12
2.2.1.	Concepto de DRM	13
2.2.2.	PlayReady	13
2.2.3.	FairPlay	13
2.2.4.	Widevine	14
2.3.	Arquitectura Widevine	14
2.3.1.	EME	14
2.3.2.	CDM	15
2.3.3.	<i>License server</i>	15
2.3.4.	<i>Packaging</i>	15
2.3.5.	CENC	16
2.3.6.	Flujo de intercambio de claves	16

2.4.	Amenazas en ecosistemas DRM	17
2.4.1.	<i>Token theft</i>	17
2.4.2.	<i>Replay</i>	18
2.4.3.	<i>Credential sharing</i>	18
2.4.4.	<i>CDN leeching</i>	18
2.4.5.	<i>Session hijacking</i>	19
2.4.6.	<i>Proxying</i>	19
2.4.7.	<i>Abuse automation</i>	19
2.5.	Estado del arte	19
2.5.1.	<i>Exploring Widevine for Fun and Profit</i>	20
2.5.2.	<i>Your DRM Can Watch You Too</i>	20
2.5.3.	Otros trabajos sobre DRM y protección OTT	20
2.5.4.	Comparativa de enfoques y limitaciones	21
2.6.	Limitaciones actuales de Widevine	21
3.	Diseño del entorno experimental	23
3.1.	Objetivos del laboratorio	24
3.2.	Arquitectura general	24
3.2.1.	Cliente	25
3.2.2.	CDN	26
3.2.3.	<i>Origin server</i>	27
3.2.4.	<i>Packager</i>	27
3.2.5.	<i>License server</i>	27
3.2.6.	Proxy de seguridad	28
3.3.	Tecnologías utilizadas	29
3.3.1.	Docker	29
3.3.2.	Python	29

3.3.3.	Widevine	29
3.3.4.	HTTP	30
3.3.5.	Linux	30
3.3.6.	Wireshark	30
3.3.7.	<i>tcpdump</i>	31
3.3.8.	Credenciales de acceso y reproducción	31
3.4.	Diseño de la infraestructura OTT	32
3.4.1.	Segmentación	32
3.4.2.	Flujo de vídeo	32
3.4.3.	<i>Packaging</i>	33
3.4.4.	Distribución	33
3.5.	Integración de Widevine	33
3.5.1.	Configuración	33
3.5.2.	Flujo de licencias	34
3.6.	Configuración del entorno de captura y monitorización	34
3.6.1.	PCAP	34
3.6.2.	<i>Logging</i>	35
3.6.3.	Métricas	35
4.	Auditoría de seguridad del ecosistema Widevine	37
4.1.	Metodología de auditoría	38
4.1.1.	Caja negra	38
4.1.2.	Caja gris	39
4.1.3.	Análisis de tráfico	39
4.1.4.	Reproducción de ataques	40
4.2.	Análisis del flujo de licencias	41
4.2.1.	Validaciones existentes	41

4.2.2.	Dependencias de confianza	42
4.3.	Evaluación de vulnerabilidades	42
4.3.1.	CDN <i>leeching</i>	43
4.3.2.	Reutilización de sesiones	43
4.3.3.	<i>Credential sharing</i>	44
4.3.4.	Abuso de concurrencia	44
4.3.5.	Ausencia de validaciones contextuales	45
4.4.	Escenarios de ataque reproducidos	45
4.4.1.	Procedimiento	45
4.4.2.	Resultados	46
4.4.3.	Evidencias	47
4.5.	Limitaciones del modelo DRM tradicional	51
5.	Arquitectura de defensa en profundidad propuesta	53
5.1.	Objetivos de la arquitectura defensiva	54
5.2.	Modelo <i>Defense in Depth</i> aplicado a OTT	54
5.3.	Diseño del acceso protegido a manifiestos y licencias	56
5.3.1.	Interceptación	57
5.3.2.	Validación	57
5.3.3.	Política de acceso	58
5.3.4.	CDN leeching y clave conocida	59
5.4.	Sistema de autenticación y autorización	59
5.4.1.	Credenciales de acceso y reproducción	59
5.4.2.	Expiración y renovación	60
5.4.3.	Validación contextual	60
5.5.	Controles de concurrencia y ciclo de vida	61
5.5.1.	Límite de sesiones	61

5.5.2.	Correlación de IP y dispositivo	61
5.5.3.	Expiración y finalización	61
5.6.	Detección de anomalías	62
5.6.1.	Patrones sospechosos	62
5.6.2.	Límites y contadores temporales	62
5.6.3.	Correlación básica	62
5.7.	Bloqueo y mitigación	63
5.7.1.	Bans de cuenta y dispositivo	63
5.7.2.	Revocación y recuperación	64
5.7.3.	Invalidación de sesión	64
5.8.	Arquitectura final integrada	64
6.	Implementación	65
6.1.	Despliegue con Docker	65
6.1.1.	Contenedores	65
6.1.2.	Redes	66
6.1.3.	Servicios y puertos	67
6.2.	Desarrollo del plano de control y del proxy protegido	68
6.2.1.	Arquitectura software	68
6.2.2.	<i>Endpoints</i>	68
6.2.3.	<i>Middleware</i> y perímetro	70
6.3.	Implementación de validaciones	70
6.3.1.	Credenciales de acceso y reproducción	70
6.3.2.	Validación de IP, dispositivo, instancia, activo y sesión	71
6.3.3.	Control de sesiones, concurrencia y expiración	72
6.4.	Sistema de <i>logging</i> y monitorización	72
6.5.	Clientes y laboratorios de auditoría	73

6.6. Integración completa del entorno	77
7. Evaluación y resultados	78
7.1. Escenarios de evaluación	79
7.2. Métricas utilizadas	81
7.2.1. Tiempo de respuesta	82
7.2.2. Número de accesos bloqueados	83
7.2.3. Detección de abuso	83
7.2.4. Impacto en rendimiento	84
7.3. Evaluación del sistema base	85
7.4. Evaluación del sistema propuesto	85
7.5. Evaluación de Key Leak y CDN leeching	87
7.6. Comparativa de resultados	90
7.7. Discusión técnica	92
7.7.1. Mejoras observadas	92
7.7.2. Limitaciones persistentes	92
7.7.3. Ataques que seguirían siendo posibles	93
7.7.4. Mitigaciones posibles	94
8. Conclusiones	95
8.1. Conclusiones generales	95
8.2. Cumplimiento de objetivos	97
8.3. Limitaciones del trabajo	98
8.4. Líneas futuras	99
8.4.1. Evolución hacia un modelo de madurez	100
A. Organización, conservación y reproducción de las evidencias	102
A.1. Estructura general del directorio	103

A.1.1. Convención de nombres	103
A.1.2. Tipos de artefacto	104
A.2. Evidencias de Fase 0	106
A.3. Evidencias de Fase 1	107
A.4. Evidencias de Fase 2	107
A.5. Resultados comparativos	108
A.6. Scripts de adquisición y tratamiento	109
A.6.1. Orden recomendado de ejecución	110
A.7. Integridad	111
A.8. Método de consulta y trazabilidad	111
A.9. Alcance y limitaciones de la evidencia	112
Bibliografía	113

Índice de figuras

2.1. Flujo general de distribución OTT con DRM	11
2.2. Flujo de reproducción y obtención de claves DRM en un entorno OTT	17
4.1. Reproducción del activo Widevine desde el cliente oficial con un usuario autorizado.	49
4.2. Reproducción Widevine desde el laboratorio externo mediante el endpoint de licencia sin autenticación.	50
4.3. Reproducción del activo CENC local con una clave ClearKey conocida y sin servidor de licencias.	51
5.1. Capas de la arquitectura defensiva y responsabilidades de Fase 1 y Fase 2.	55
5.2. Secuencia de autenticación, sesión, manifiesto y licencia en las fases endurecidas.	58
5.3. Flujo de eventos, riesgo y respuesta activa en Fase 2.	63
6.1. Cliente oficial y laboratorio de Fase 1 utilizando manifiesto y licencia protegidos.	74
6.2. Cliente de Fase 2 con reproducción activa y panel de observabilidad.	75
6.3. Laboratorio de Fase 2 tras alcanzar 100 puntos por rechazos de concurrencia y activar el ban automático de cuenta.	76
7.1. Evaluación de Fase 2 tras ocho fallos de autenticación y activación del ban de dispositivo.	84

7.2. Pruebas negativas de Fase 1 con reproducción legítima activa y binding de token, activo y sesión.	86
7.3. Reproducción externa en Fase 2 con sesión única, token válido y clave conocida; se han servido segmentos sin licencia y el riesgo ha aumentado en 20 puntos.	88
7.4. Controles de Fase 2 frente a origen web externo, concurrencia, copia entre instancias, cruce de activo y ausencia de token.	89

Índice de tablas

3.1. Componentes del entorno experimental	25
4.1. Resumen de hallazgos de seguridad en Fase 0	43
4.2. Resultados de los escenarios reproducidos en Fase 0	47
5.1. Trazabilidad entre hallazgos y controles defensivos	56
6.1. Contenedores de las fases endurecidas	66
6.2. Interfaces publicadas en el anfitrión	67
6.3. Endpoints principales de Fase 1 y Fase 2	69
6.4. Parámetros de credenciales y sesiones	71
6.5. Reglas implementadas en Fase 2	73
7.1. Escenarios de la batería comparativa	80
7.2. Tiempos de respuesta observados, 20 muestras por operación	82
7.3. Resultados específicos de Key Leak y CDN leeching	89
7.4. Comparación final de las tres fases	91
8.1. Propuesta inicial de niveles de madurez para la protección OTT .	100
A.1. Tipos de artefacto conservados	105
A.2. Escenarios documentados en Fase 0	106
A.3. Ficheros del directorio de resultados	108

A.4. Herramientas de reproducción y tratamiento de evidencias	110
---	-----

Índice de códigos

A.1. Estructura simplificada del directorio de evidencias	103
A.2. Secuencia para regenerar los resultados derivados	110

Glosario de términos y acrónimos

Término	Definición adaptada al TFM
OTT – <i>Over-The-Top</i>	Distribución de contenido audiovisual por Internet mediante un servicio independiente del operador de acceso.
<i>Streaming</i>	Reproducción de contenido audiovisual mientras se recibe, sin esperar a descargarlo completamente.
VOD – <i>Video on Demand</i>	Vídeo bajo demanda: contenido previamente disponible que el usuario puede reproducir cuando lo solicita.
CDN – <i>Content Delivery Network</i>	Red de distribución que acerca el contenido a los usuarios y reduce la carga del servidor de origen.
DRM – <i>Digital Rights Management</i>	Conjunto de tecnologías para proteger contenido digital y controlar sus condiciones de uso.
Widevine	Sistema DRM de Google utilizado para proteger la distribución y reproducción de contenido audiovisual.
EME – <i>Encrypted Media Extensions</i>	API del navegador que permite a una aplicación interactuar con sistemas de protección de contenido.
CDM – <i>Content Decryption Module</i>	Componente del cliente que procesa licencias, gestiona claves y participa en el descifrado del contenido.
TEE – <i>Trusted Execution Environment</i>	Entorno de ejecución aislado, normalmente respaldado por hardware, que protege código y datos sensibles frente al sistema operativo y otras aplicaciones del dispositivo.
Widevine L1	Nivel de seguridad en el que las operaciones criptográficas y el procesamiento del contenido protegido se realizan dentro de un TEE. Ofrece el mayor nivel de protección de Widevine y suele exigirse para contenidos de alta resolución.
Widevine L3	Nivel de seguridad basado en software, sin que el procesamiento del contenido protegido permanezca íntegramente dentro de un TEE. Presenta una superficie de exposición mayor y suele asociarse a restricciones de calidad.

Término	Definición adaptada al TFM
CENC – <i>Common Encryption</i>	Estándar de cifrado común que permite utilizar medios cifrados con diferentes sistemas DRM, según su compatibilidad.
ClearKey	Sistema de claves de EME que permite proporcionar claves de contenido sin la protección propia de un DRM comercial.
KID – <i>Key Identifier</i>	Identificador de una clave de contenido; no contiene por sí mismo la clave de descifrado.
PSSH – <i>Protection System Specific Header</i>	Caja de metadatos que contiene información de inicialización asociada a un sistema de protección.
MPEG-DASH	Estándar de distribución adaptativa de contenido audiovisual segmentado sobre HTTP.
MPD – <i>Media Presentation Description</i>	Manifiesto de MPEG-DASH que describe las representaciones y la localización de los segmentos.
HLS – <i>HTTP Live Streaming</i>	Protocolo de distribución audiovisual sobre HTTP basado en listas de reproducción y segmentos.
Segmento	Fragmento de contenido audiovisual que el reproductor solicita como parte de una presentación.
<i>Packaging</i> – Empaquetado	Preparación del contenido codificado para su distribución, incluyendo segmentación, manifiestos y, cuando corresponda, cifrado.
Servidor de origen – <i>Origin server</i>	Servicio que almacena o publica los objetos multimedia que recupera la capa de distribución.
Servidor de licencias	Servicio que genera o entrega licencias con claves y condiciones de uso para un sistema DRM.
<i>Challenge</i> DRM	Mensaje generado por el CDM y enviado al servicio de licencias como parte del intercambio DRM.
Plano de control – <i>Control plane</i>	Componentes que gestionan identidad, permisos, sesiones y decisiones de acceso.
Plano de medios	Componentes y rutas encargados de entregar manifiestos, inicializaciones y segmentos audiovisuales.
<i>Entitlement</i>	Permiso de una cuenta para acceder a un contenido o servicio determinado.
Token de acceso – <i>Access token</i>	Credencial utilizada para autorizar operaciones de una identidad autenticada.
Token de reproducción – <i>Playback token</i>	Credencial temporal que autoriza operaciones asociadas a una sesión de reproducción.

Término	Definición adaptada al TFM
<i>Bearer token</i>	Credencial cuyo uso depende de su posesión, sin exigir por sí misma una prueba criptográfica adicional del emisor de la petición.
<i>Binding – Vinculación contextual</i>	Asociación de una credencial con atributos como cuenta, dispositivo, activo, instancia y sesión.
<i>Heartbeat</i>	Mensaje periódico que mantiene o actualiza el estado de una sesión activa.
<i>TTL – Time To Live</i>	Tiempo durante el cual un dato o registro conserva su validez antes de caducar.
<i>Concurrencia</i>	Existencia de varias sesiones activas simultáneamente para una misma cuenta.
<i>Revocación</i>	Retirada de una autorización antes de su expiración prevista.
<i>Rate limiting</i>	Limitación del número de solicitudes admitidas durante una ventana temporal.
<i>CORS – Cross-Origin Resource Sharing</i>	Mecanismo del navegador que regula el acceso de una página a respuestas procedentes de otro origen web.
<i>Key leak</i>	Filtración de una clave de contenido fuera del entorno en el que debería permanecer protegida.
<i>CDN leeching</i>	Uso no autorizado de la infraestructura de distribución de un proveedor para obtener o servir contenido.
<i>Restreaming</i>	Redistribución de un flujo audiovisual obtenido previamente de otra fuente.
<i>Replay</i>	Reutilización de mensajes o credenciales válidos para repetir una operación fuera del contexto previsto.
<i>Credential sharing</i>	Uso compartido de las credenciales de una cuenta entre varias personas.
<i>Session hijacking</i>	Apropiación de una sesión ajena mediante el compromiso de sus credenciales o mecanismos de identificación.
<i>Watermarking – Marca de agua</i>	Incorporación de información identificativa al contenido audiovisual para facilitar su atribución o seguimiento.
<i>Analog hole – Agujero analógico</i>	Posibilidad de volver a registrar imágenes y sonido una vez representados físicamente para su consumo.
<i>PCAP / PCAPNG</i>	Formatos de archivo utilizados para conservar capturas de tráfico de red.

1

Introducción

1.1. Contexto y motivación

La distribución de contenidos audiovisuales a través de servicios OTT (*Over-The-Top*) se ha consolidado como uno de los principales modelos de consumo digital. El crecimiento sostenido de las suscripciones a servicios de vídeo en línea, el desplazamiento progresivo del visionado lineal hacia modelos bajo demanda y la expansión del consumo multipantalla evidencian que el *streaming* ya no constituye un canal complementario, sino una infraestructura central para la difusión de contenidos de alto valor [1, 2, 3]. En este escenario, la protección del contenido deja de ser una cuestión puramente técnica para convertirse también en un requisito de negocio, de cumplimiento contractual y de sostenibilidad operativa.

1.1.1. Crecimiento del *streaming* OTT

La transformación del mercado audiovisual en la última década se explica por la convergencia entre conectividad ubicua, consumo bajo demanda y distribución sobre infraestructuras IP de propósito general. Informes sectoriales recientes muestran no solo un aumento del número de suscripciones, sino también un cambio estructural en los hábitos de consumo y en la forma en que las audiencias acceden a contenidos premium [1, 2]. A ello se suma el incremento del tráfico

móvil y el peso creciente de 5G en la distribución de datos, factores que amplían el alcance del *streaming* a más contextos de movilidad y a una mayor diversidad de dispositivos [3].

Desde el punto de vista técnico, este crecimiento se apoya en arquitecturas basadas en entrega segmentada, reproducción adaptativa y uso intensivo de redes de distribución de contenidos (CDN). Esta modularidad facilita la escalabilidad del servicio, pero introduce también múltiples puntos de control y de fallo: la aplicación del lado del cliente, reproductor, CDN, sistema de empaquetado y compresión del contenido, servidor de origen, mecanismos de autenticación y servidores de licencias. Por ello, la seguridad efectiva del ecosistema depende de la coordinación de todos estos elementos, y no exclusivamente del mecanismo DRM empleado.

1.1.2. Importancia del DRM

En un servicio OTT, el DRM (*Digital Rights Management*) aporta un mecanismo estandarizado para cifrar el contenido, entregar licencias de uso y habilitar la descodificación únicamente en entornos de reproducción autorizados [4, 5]. En el ámbito web, EME (*Encrypted Media Extensions*) proporciona la interfaz de interacción con los sistemas de protección, mientras que el CDM (*Content Decryption Module*) materializa, en el lado del cliente, la lógica de descifrado y de aplicación de las restricciones asociadas a la licencia [4, 6]. Por tanto, el DRM sigue siendo una pieza imprescindible para la distribución de contenido de pago, aunque su función debe entenderse como una capa concreta dentro de un sistema mucho más amplio.

1.1.3. Widevine como estándar dominante

Dentro del ecosistema actual, Widevine se ha consolidado como la solución DRM de referencia en un amplio conjunto de despliegues OTT. La documentación oficial de Google lo sitúa como sistema utilizado por grandes plataformas internacionales y soportado en un conjunto muy amplio de navegadores, dispositivos y sistemas operativos [5]. Esta presencia se refuerza por su integración en Android y por el soporte del CDM de Google Widevine en navegadores de escritorio como Chrome o Firefox [7, 8]. La literatura académica reciente también lo identifica como una de las soluciones más extendidas o desplegadas dentro del ecosistema EME, lo que convierte a Widevine en un objeto de estudio especialmente relevante [9, 6].

1.1.4. Limitaciones de asumir que tener DRM implica seguridad

La hipótesis implícita de que “tener DRM” equivale a “tener el servicio protegido” resulta técnicamente insuficiente. La especificación EME deja fuera de su alcance funciones como la autenticación y la autorización, que recaen en la lógica de la aplicación [4]. Del mismo modo, el análisis formal reciente de Widevine subraya que las cuestiones de confianza entre proveedor de contenido, servidor de licencias y CDN quedan fuera de su modelo base [6]. A ello se añaden trabajos que han mostrado la viabilidad de analizar internamente Widevine, interceptar mensajes o comprometer escenarios *software-only* como Widevine L3, así como problemas de privacidad asociados a determinadas implementaciones del navegador [9, 10]. En otras palabras, el DRM es necesario, pero no basta por sí solo para cerrar todos los vectores de abuso relevantes en una plataforma OTT real.

1.2. Problema de investigación

El problema que aborda este TFM surge de una brecha entre dos planos distintos de seguridad. Por un lado, Widevine protege el contenido cifrado y regula la obtención y el uso de claves de descifrado. Por otro, la explotación abusiva de un servicio OTT real suele producirse en capas diferentes: acceso a la CDN, emisión o reutilización de tokens, compartición de credenciales, uso simultáneo no autorizado o *bypass* de controles perimetrales. En consecuencia, la pregunta que guía este trabajo puede formularse del siguiente modo: *¿hasta qué punto el despliegue aislado de Widevine DRM aporta seguridad suficiente frente al abuso de distribución de contenido en servicios OTT reales?*

1.2.1. Qué protege realmente Widevine

Desde una perspectiva funcional, Widevine protege el contenido en tránsito y en reproducción mediante cifrado, licencias y ejecución controlada dentro del CDM. La literatura formal sobre Widevine a través de EME identifica como propiedades centrales la confidencialidad de la clave de descifrado, la integridad y autenticidad de las licencias y de los derechos de uso, así como la cadencia de refresco de determinadas licencias para evitar reutilizaciones incompatibles con el protocolo [6]. En términos prácticos, esto significa que Widevine está diseñado para dificultar que un atacante obtenga la clave de contenido fuera del entorno previsto y para evitar alteraciones arbitrarias de los derechos asociados a dicha clave.

1.2.2. Qué no protege Widevine

Que Widevine controle la entrega de licencias no significa que resuelva por sí solo el problema completo de autorización. La propia especificación EME señala expresamente que la autenticación y la autorización quedan del lado de la aplicación, no del estándar DRM [4]. Además, la documentación oficial de Widevine describe un flujo en el que las peticiones del cliente deben pasar por un *proxy* operado por el proveedor, encargado de validar la solicitud y añadir reglas de negocio antes de que intervenga el servicio de licencias [5]. Esto evidencia que la legitimidad del acceso, la vigencia de la suscripción, la coherencia del contexto de uso o la política de sesión pertenecen a capas complementarias al DRM.

1.2.3. Amenazas persistentes en ecosistemas OTT

Las amenazas más relevantes en un entorno OTT moderno no siempre adoptan la forma de una ruptura directa de la criptografía. Con frecuencia aparecen como abusos que explotan credenciales válidas, configuraciones permisivas o debilidades en la correlación entre identidad, sesión, dispositivo y entrega del contenido [11, 12, 13, 14, 15].

CDN *leeching*

El *CDN leeching* consiste en el uso ilegítimo de la infraestructura de distribución del servicio para servir contenido a terceros no autorizados. En estos casos, el atacante no necesita romper el DRM: basta con simular mediante software un CDM, reutilizar mecanismos válidos de acceso a la CDN, como enlaces, tokens o rutas expuestas indebidamente, trasladando además el coste de ancho de banda al operador legítimo [16, 11, 12].

Reutilización de tokens

La reutilización de tokens es un problema de sesión y de control de acceso, no de descifrado del contenido. Si un token se emite con una validez excesiva, sin vinculación contextual suficiente o sin mecanismos que dificulten su *replay*, un tercero puede reutilizarlo para solicitar manifiestos, segmentos o incluso licencias dentro de una ventana temporal indebida [13].

Credential sharing

El *credential sharing* consiste en el acceso de múltiples personas a un servicio mediante una misma cuenta o identidad digital. En el contexto OTT, este fenómeno erosiona el modelo de suscripción y altera las métricas reales de uso, aun cuando las reproducciones sigan utilizando licencias válidas emitidas por el DRM [15]. Por ello, constituye un problema de control de acceso y de política de servicio, no de cifrado del contenido.

Accesos concurrentes

El abuso de concurrencia aparece cuando una misma identidad mantiene más sesiones activas de las previstas por la política de servicio, ya sea en múltiples dispositivos, ubicaciones o contextos de red. La experiencia industrial aborda este problema mediante servicios específicos de monitorización de sesiones activas y evaluación de políticas en tiempo real, lo que confirma que el control de concurrencia pertenece a una capa de gobierno del acceso que complementa, pero no sustituye, al DRM [14].

***Bypass* de controles perimetrales**

Un servicio OTT puede ser vulnerable aunque sus flujos estén protegidos con DRM si sus controles perimetrales son eludibles. La documentación de grandes proveedores de CDN advierte expresamente del riesgo de acceso directo al origen o de *bypass* de restricciones cuando las firmas no se validan de extremo a extremo o cuando existen rutas alternativas no protegidas [11, 12]. Este tipo de fallo revela que la seguridad del servicio depende también de la coherencia entre CDN, origen, tokenización y políticas de acceso.

1.3. Hipótesis de trabajo

La hipótesis central de este TFM es que el uso aislado de Widevine DRM mejora de forma significativa la protección criptográfica del contenido, pero no proporciona por sí solo seguridad suficiente frente al abuso de distribución en entornos OTT reales. En particular, se plantea que, sin mecanismos adicionales de autenticación contextual, autorización dinámica, control de concurrencia y validación de acceso a la CDN, siguen existiendo vectores viables de explotación que permiten consumo no autorizado, redistribución y uso abusivo de la

infraestructura.

1.4. Objetivos

1.4.1. Objetivo general

El objetivo principal de este TFM es diseñar, implementar y auditar un entorno experimental de *streaming* de vídeo en directo que integre el sistema de protección Widevine DRM, con el fin de evaluar sus vulnerabilidades inherentes y proponer una arquitectura de defensa en profundidad mediante el desarrollo de capas adicionales de autenticación, autorización dinámica y control de concurrencia.

1.4.2. Objetivos específicos

1. Desplegar una infraestructura completa de distribución OTT (*Over-The-Top*) mediante contenedores, simulando la cadena completa de vídeo, incluyendo el empaquetado con cifrado Common Encryption (CENC), un servidor de origen y una red de distribución de contenidos (CDN).
2. Realizar una auditoría de seguridad del ecosistema DRM Widevine, analizando el flujo de intercambio de licencias entre el cliente, el *Content Decryption Module* (CDM) y el servidor de licencias, e identificando vectores de ataque críticos.
3. Desarrollar un *proxy* para gestionar las licencias del contenido con lógica de autorización dinámica, interceptando las peticiones de licencia y validando en tiempo real la identidad del usuario mediante tokens JWT, su ubicación geográfica y el estado de su suscripción antes de permitir la entrega efectiva de la licencia.
4. Implementar mecanismos adicionales de seguridad que limiten el abuso, como controles de concurrencia y detección de anomalías capaces de bloquear automáticamente sesiones o direcciones IP al detectar patrones compatibles con uso compartido de cuentas, reutilización indebida de sesiones o intentos de acceso masivo.

1.5. Contribuciones

Las principales contribuciones de este TFM se concretan en cuatro resultados complementarios:

1. El diseño y despliegue de un laboratorio OTT/DRM reproducible, basado en contenedores y compuesto por los elementos necesarios para representar de extremo a extremo la distribución y reproducción de contenido protegido.
2. La caracterización experimental de los límites de una configuración base protegida mediante Widevine, que ha permitido diferenciar la protección criptográfica proporcionada por el DRM de otros riesgos asociados a la autenticación, las sesiones, la concurrencia y el acceso a la CDN.
3. El diseño y la implementación incremental de una arquitectura de defensa en profundidad estructurada en dos fases, mediante la incorporación progresiva de autenticación, autorización contextual, control de concurrencia y detección de comportamientos anómalos.
4. Una evaluación comparativa de las distintas fases del sistema, acompañada de pruebas automatizadas y de un conjunto organizado de artefactos y evidencias reproducibles, entre los que se incluyen capturas de tráfico, resultados en formato JSON, capturas visuales y scripts de apoyo.

1.6. Metodología

La metodología del trabajo se ha desarrollado como un proceso incremental e iterativo, con una orientación experimental. Esta elección ha respondido a la necesidad de construir primero un entorno base funcional, observar su comportamiento, identificar debilidades verificables y, a partir de ellas, diseñar e implementar contramedidas evaluables.

1.6.1. Metodología incremental

En una primera etapa se ha construido un entorno OTT mínimo, pero representativo, que ha permitido distribuir contenido cifrado y gestionar licencias Widevine de manera reproducible. Sobre esa base se han añadido progresivamente instrumentación, mecanismos de captura, lógica de autorización y controles

defensivos. Cada iteración ha producido un resultado verificable: primero el funcionamiento del sistema base, después la identificación de amenazas reproducibles y, finalmente, la evaluación comparativa del sistema reforzado.

1.6.2. Laboratorio experimental

El núcleo práctico del TFM ha sido un laboratorio experimental desplegado sobre tecnologías reproducibles y fácilmente instrumentables. Este laboratorio ha incluido los elementos necesarios para representar la cadena OTT completa: empaquetado del contenido, distribución segmentada, servidor de origen, punto de entrega mediante CDN, cliente de reproducción y lógica de licencias. Su finalidad no ha sido replicar exhaustivamente una plataforma comercial, sino disponer de un entorno suficientemente realista para observar interacciones, dependencias y puntos de abuso.

1.6.3. Auditoría técnica

Sobre el entorno base se ha llevado a cabo una auditoría técnica con enfoque mixto de caja negra y caja gris. En la dimensión de caja negra se ha analizado el comportamiento observable del sistema desde la perspectiva de un usuario o atacante con acceso limitado. En la dimensión de caja gris se ha aprovechado el conocimiento del laboratorio para inspeccionar el flujo de licencias, la gestión de tokens, la relación entre cliente y CDN y las validaciones aplicadas en cada punto del proceso.

1.6.4. Diseño iterativo de defensas

A partir de los resultados de la auditoría se ha definido una arquitectura de defensa en profundidad orientada a endurecer el control de acceso sin alterar el papel central del DRM. Esta arquitectura se ha implementado de forma iterativa mediante medidas como la validación contextual de tokens, el control de sesiones activas, restricciones adicionales de acceso y mecanismos básicos de detección de anomalías. Cada defensa se ha evaluado tanto por su capacidad de mitigación como por su impacto operativo, ya que en entornos OTT la seguridad no puede desligarse ni del rendimiento ni de la experiencia de usuario.

1.7. Estructura de la memoria

La memoria se organiza en ocho capítulos. Tras esta introducción, el Capítulo 2 desarrolla los fundamentos teóricos y el estado del arte, con especial atención a la arquitectura OTT, los sistemas DRM y las limitaciones actuales de Widevine. El Capítulo 3 presenta el diseño del entorno experimental y describe los componentes que permiten reproducir una cadena OTT funcional sobre la que realizar pruebas y capturas.

El Capítulo 4 recoge la auditoría de seguridad del ecosistema Widevine y estructura los hallazgos por escenarios de amenaza. El Capítulo 5 propone una arquitectura de defensa en profundidad orientada a complementar Widevine con validaciones contextuales, control de concurrencia y mecanismos básicos de mitigación. El Capítulo 6 describe la implementación concreta de dicha arquitectura. El Capítulo 7 evalúa comparativamente el sistema base y el sistema reforzado mediante métricas objetivas. Finalmente, el Capítulo 8 resume las conclusiones principales del trabajo y valora el alcance real de la contribución realizada.

2

Fundamentos teóricos y estado del arte

2.1. *Streaming* OTT

El modelo *Over-The-Top* (OTT) designa la distribución de contenido audiovisual sobre redes IP abiertas, desacoplando el servicio de vídeo del operador de acceso y apoyándose en mecanismos de entrega adaptativa sobre HTTP. Desde un punto de vista técnico, su consolidación se explica por la convergencia entre segmentación del flujo, manifiestos de reproducción, escalado horizontal mediante CDN y soporte multiplataforma en navegadores, móviles, *smart TV* y *set-top boxes*. Esta combinación ha convertido a OTT en el paradigma dominante para la distribución de vídeo por Internet, tanto en directo como bajo demanda, y ha desplazado el foco de seguridad desde el simple transporte del contenido hacia la protección extremo a extremo del ecosistema de reproducción y autorización [17, 18, 19].

2.1.1. Arquitectura OTT

Una arquitectura OTT típica separa claramente varias funciones. En origen, una cadena de ingestión y codificación genera múltiples representaciones del mismo contenido a distintos *bitrates* y resoluciones. A continuación, un *packager* segmenta esas representaciones, genera los manifiestos y, si procede, aplica cifra-

do y metadatos de protección. El servidor *origin* publica dichos objetos, mientras que la CDN replica segmentos y manifiestos cerca del usuario final para reducir latencia y absorber carga. En el extremo cliente, un reproductor selecciona la calidad adecuada en función del estado de red, solicita licencias DRM y coordina el descifrado con el CDM correspondiente. Esta separación funcional aporta escalabilidad, pero también fragmenta los puntos de control de seguridad, ya que autenticación, autorización, expedición de licencias y entrega de segmentos pueden recaer en componentes distintos [20, 21, 18, 4].

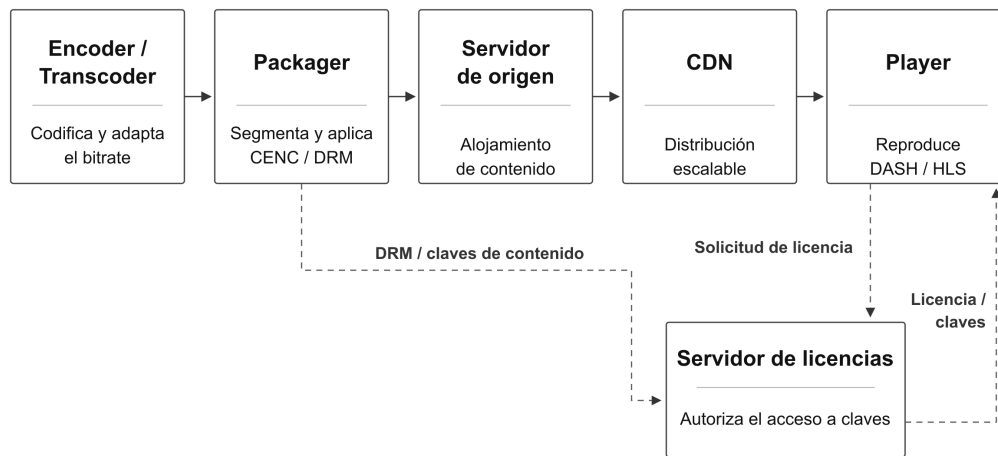


Figura 2.1: Flujo general de distribución OTT con DRM

2.1.2. *Live streaming*

En *live streaming*, el contenido no está completamente disponible en el instante de la solicitud, sino que los segmentos se producen y publican conforme avanza el evento. Este rasgo introduce un compromiso constante entre latencia, tamaño de segmento, estabilidad del *buffer* y capacidad de recuperación ante pérdidas o variaciones de red. HLS define *live playlists* capaces de añadir y retirar segmentos de la ventana activa, mientras que MPEG-DASH contempla perfiles y directrices de interoperabilidad específicas para directo. En consecuencia, los despliegues OTT en vivo son especialmente sensibles a la sincronización temporal, a la propagación de cachés y a la validez de tokens y sesiones, ya que cualquier desalineación afecta tanto a la experiencia de usuario como al cumplimiento de las políticas de acceso [19, 17, 18].

2.1.3. CDN

La CDN es el subsistema encargado de distribuir manifiestos y segmentos desde ubicaciones cercanas al consumidor, reduciendo el recorrido de red y descargando al *origin*. En OTT moderno, la CDN no solo mejora rendimiento y escalabilidad, sino que se convierte en una pieza crítica del perímetro de acceso, pues suele servir directamente los objetos multimedia que el reproductor necesita. DASH-IF destaca, entre otras ventajas, el soporte de múltiples CDN y cachés a partir del mismo manifiesto, mejorando flexibilidad y tolerancia a fallos. Sin embargo, desde la perspectiva de seguridad, el hecho de que un segmento viaje cifrado no implica que su entrega sea legítima: si URLs firmadas, tokens o manifiestos se comparten fuera de contexto, la CDN puede seguir sirviendo tráfico válido desde el punto de vista criptográfico pero abusivo desde el punto de vista del negocio [18, 22, 4].

2.1.4. HLS y MPEG-DASH

HLS y MPEG-DASH resuelven el mismo problema general —la entrega adaptativa de contenido segmentado sobre HTTP—, pero lo hacen con modelos de señalización distintos. HLS organiza la presentación mediante *Master Playlists* y *Media Playlists*, e identifica los segmentos y los parámetros de cifrado a través de etiquetas como **EXT-X-KEY**. MPEG-DASH, por su parte, define una *Media Presentation Description* (MPD) como documento maestro que describe representaciones, alineamiento temporal y localización de segmentos. DASH se apoya de forma natural en CENC y en el ecosistema EME para entornos web y *multi-DRM*, mientras que HLS constituye la base obligada en el ecosistema Apple y se integra de manera nativa con FairPlay Streaming. En la práctica, ambos protocolos conviven en arquitecturas OTT reales, y su elección responde menos a una superioridad técnica absoluta que a requisitos de compatibilidad, DRM y parque de dispositivos [19, 17, 23, 18].

2.2. Sistemas DRM

Los sistemas DRM (*Digital Rights Management*) combinan cifrado del contenido, gestión de licencias y aplicación de reglas de negocio para limitar quién puede reproducir un activo digital, durante cuánto tiempo y bajo qué condiciones. En el ámbito OTT, su función principal no es impedir toda copia posible, sino elevar el coste de acceso no autorizado y condicionar el descifrado a la obtención de una licencia válida por parte de un módulo confiable del cliente. Esta precisión

es importante, porque ni EME ni el propio DRM sustituyen a los mecanismos de autenticación, autorización contextual, control de concurrencia o monitorización del abuso; de hecho, la especificación EME deja expresamente esas funciones en manos de la aplicación [4, 24].

2.2.1. Concepto de DRM

En términos funcionales, un DRM introduce tres decisiones técnicas. La primera consiste en cifrar el contenido o partes del mismo con claves que no viajan en claro junto al flujo. La segunda es establecer un protocolo de licenciamiento por el que el cliente solicita y recibe, bajo ciertas condiciones, el material necesario para descifrar. La tercera es hacer cumplir en el extremo cliente restricciones de uso, calidad, persistencia o salida segura. Esta arquitectura protege razonablemente bien el camino criptográfico entre contenido cifrado y reproducción autorizada, pero deja fuera del núcleo DRM otras superficies de ataque relevantes, como el robo de tokens, el abuso de credenciales, la compartición de sesiones o la redistribución desde clientes comprometidos [4, 24, 9].

2.2.2. PlayReady

PlayReady es la solución DRM promovida por Microsoft para la entrega de contenido premium en múltiples dispositivos y entornos HTML5. La documentación oficial lo presenta como una *suite* orientada a facilitar modelos de negocio flexibles y acceso multiplataforma, incluyendo soporte para estándares y componentes de protección reforzada. Desde la literatura académica reciente, PlayReady aparece, junto con FairPlay y Widevine, como uno de los tres grandes DRM comerciales desplegados de forma masiva en el ecosistema móvil y OTT. Su relevancia en este capítulo es comparativa: permite contextualizar que Widevine no es un caso aislado, sino parte de un conjunto de soluciones que comparten la misma lógica general de cifrado, licencias y robustez del cliente [25, 24].

2.2.3. FairPlay

FairPlay Streaming es la tecnología DRM de Apple para la entrega segura de medios sobre HLS en plataformas Apple. Apple la documenta como un mecanismo que permite cifrar contenido, intercambiar claves de forma segura y proteger la reproducción en iOS, tvOS, macOS y otros entornos del fabricante. La pieza de servidor clave es el *Key Server Module* (KSM), para el que Apple distribuye

un SDK con guías de implementación, claves de desarrollo y ejemplos de manejo de peticiones. Su principal fortaleza es la fuerte integración vertical con el ecosistema Apple, lo que simplifica la interoperabilidad y el control del camino de reproducción; su principal limitación, desde una perspectiva multiproveedor, es que ese mismo acoplamiento reduce su utilidad fuera del stack HLS + Apple [23, 24].

2.2.4. Widevine

Widevine es la solución DRM de Google para servicios OTT y CAS. Su documentación oficial destaca una aproximación basada en estándares, con capacidad para licenciar, distribuir de forma segura y proteger la reproducción de contenido sobre un gran número de dispositivos y plataformas. Además, remite explícitamente a CENC y EME como especificaciones abiertas sobre las que se construye parte de su ecosistema, y subraya la importancia de la raíz hardware de confianza mediante *keyboxes* y procesos de descifrado y renderizado seguros cuando el dispositivo lo permite. Esta amplitud de despliegue, junto con su presencia en navegadores, Android y electrónica de consumo, explica que Widevine sea el foco principal del presente TFM [5, 9].

2.3. Arquitectura Widevine

La arquitectura Widevine puede entenderse como la interacción coordinada entre interfaz web o aplicación, CDM, servidor de licencias, *packager*, metadatos CENC y la infraestructura de distribución. Aunque muchos de sus detalles internos son propietarios, la combinación de documentación oficial, estándares abiertos y literatura académica permite reconstruir suficientemente bien su funcionamiento para fines de auditoría [4, 5, 9].

2.3.1. EME

Encrypted Media Extensions (EME) es la interfaz estandarizada por el W3C para controlar desde el navegador la reproducción de contenido cifrado. La especificación es especialmente relevante porque aclara dos cuestiones esenciales. En primer lugar, EME no define un sistema DRM, sino una API común para descubrir, seleccionar e interactuar con sistemas de protección de contenido. En segundo lugar, la propia especificación indica que funciones como autenticación y autorización quedan a cargo de los autores de la aplicación o de la página. Por

tanto, EME habilita el canal técnico entre reproductor y CDM, pero no resuelve por sí sola los controles de acceso de negocio. Adicionalmente, EME exige contexto seguro y recomienda transporte seguro para todo el tráfico que involucre mensajes del CDM [4].

2.3.2. CDM

El CDM (*Content Decryption Module*) es el componente que procesa las peticiones de licencia, gestiona claves y participa en el descifrado del contenido. En Widevine, la robustez real del CDM depende del nivel de seguridad disponible en el dispositivo. Patat et al. describen que Widevine L1 sitúa criptografía y procesamiento de vídeo dentro del TEE, mientras que L3 desplaza ambas funciones fuera de dicho entorno, quedando implementadas totalmente en software. Este punto es capital para esta investigación: cuando el CDM es *software-only*, el cliente deja de ser una base de confianza fuerte y pasa a ser un entorno potencialmente instrumentable desde software de terceros. La literatura ha mostrado que ese escenario permite estudiar y recuperar material criptográfico crítico del flujo de descifrado, de modo que L3 no debe considerarse equivalente a un perímetro de seguridad robusto para contenido de alto valor [9, 24, 5].

2.3.3. *License server*

El servidor de licencias es el componente que responde a los mensajes generados por el CDM y decide, en última instancia, si se entrega o no el material de licencia necesario para descifrar. En entornos EME, la aplicación media explícitamente entre el navegador y el servidor, por lo que el flujo completo incluye una capa adicional de lógica del lado del servicio. La documentación de Shaka Packager muestra que el aprovisionamiento de Widevine se articula en torno a parámetros como *key server URL*, *content ID* y *policy*, evidenciando que el *license server* no actúa como mero contenedor de claves, sino como punto de aplicación de reglas de negocio. Esta observación es importante porque muchas debilidades prácticas de los sistemas OTT no residen en el cifrado, sino en cómo se autoriza el acceso a la licencia [4, 21, 5].

2.3.4. *Packaging*

El *packaging* prepara el flujo ya codificado para su distribución OTT. Shaka Packager, como referencia ampliamente utilizada, documenta que puede empaquetar contenido para DASH y HLS, segmentar el flujo, generar manifiestos y

aplicar cifrado sin realizar *transcoding*, que debe haberse completado previamente. También soporta obtención de claves desde servidores Widevine y PlayReady, así como generación de contenido *multi-DRM*. Por ello, el *packager* es un punto de convergencia entre requisitos audiovisuales y de protección: asigna claves o identificadores de clave a pistas concretas, inserta metadatos necesarios para el cliente y condiciona la forma en que manifiestos, segmentos y licencias se relacionarán durante la reproducción [20, 21].

2.3.5. CENC

Common Encryption (CENC) es el estándar que permite describir y aplicar un esquema común de cifrado sobre contenedores ISO BMFF, facilitando que el mismo contenido protegido pueda ser consumido por diferentes DRM. El registro de formatos de EME para CENC define el uso de cajas PSSH y un *Common SystemID*, mientras que DASH-IF destaca la ventaja operativa de cifrar una vez y reproducir en distintos entornos mediante múltiples tecnologías DRM. En otras palabras, CENC separa el problema del cifrado del problema del sistema concreto que entrega la licencia, y esa separación es una de las razones por las que Widevine, PlayReady y FairPlay pueden coexistir en flujos OTT complejos con una misma base de medios segmentados [26, 18, 5].

2.3.6. Flujo de intercambio de claves

De forma simplificada, el flujo completo comienza cuando el reproductor detecta que el contenido está protegido y solicita acceso al sistema de claves apropiado mediante EME. Después, crea un objeto `MediaKeys` y una sesión de claves; el CDM genera un mensaje con los datos de inicialización relevantes, normalmente asociados a metadatos CENC y cajas PSSH. La aplicación envía dicho mensaje al servidor de licencias, que valida el contexto y responde con la licencia. Una vez procesada la respuesta por el CDM, el reproductor puede solicitar a la CDN los segmentos cifrados y reproducirlos a través del camino permitido por el nivel de seguridad disponible. Este flujo muestra que el acceso efectivo al contenido depende tanto de la validez criptográfica de la licencia como de las decisiones de control de acceso tomadas fuera del núcleo DRM [4, 26, 9, 5].

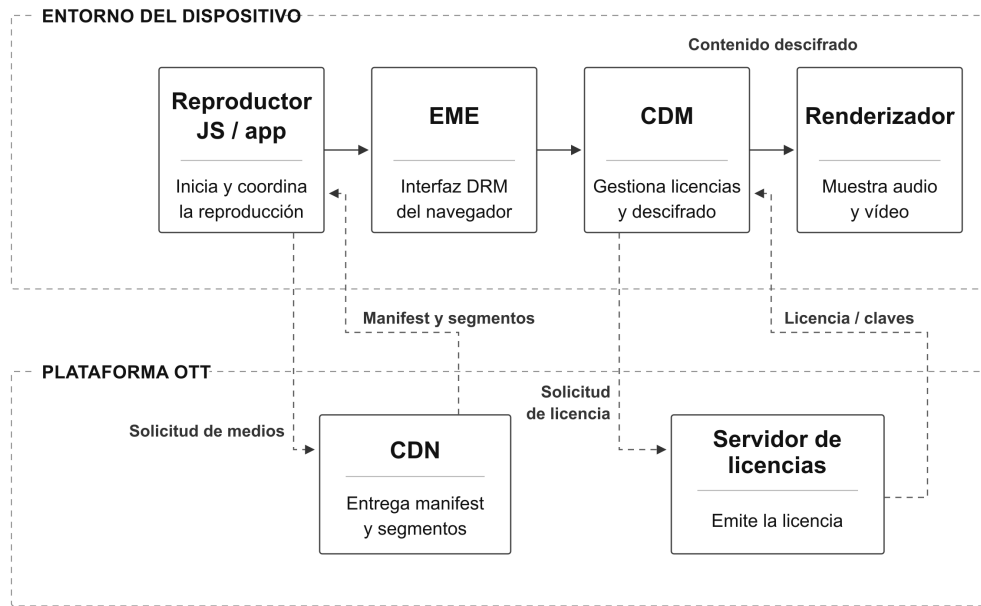


Figura 2.2: Flujo de reproducción y obtención de claves DRM en un entorno OTT

2.4. Amenazas en ecosistemas DRM

En ecosistemas OTT reales, buena parte de las amenazas más relevantes no consisten en romper el algoritmo de cifrado, sino en abusar del contexto de autorización que rodea al DRM. Precisamente por eso, un sistema puede mantener intacto Widevine y, aun así, sufrir robo de tokens, compartición de cuentas, secuestro de sesiones o consumo abusivo en CDN [4, 22].

2.4.1. *Token theft*

Muchos servicios OTT usan tokens, URLs firmadas o credenciales temporales para autorizar el acceso a manifiestos, segmentos o licencias. Si esos artefactos se filtran desde el cliente, desde un proxy intermedio o desde un entorno de depuración, el atacante no necesita romper el DRM para reutilizarlos. El problema se agrava cuando el token no está estrictamente ligado al dispositivo, a la IP, a la ventana temporal o al estado de la sesión. Rodrigues et al. analizan precisamente el impacto del uso compartido y abuso de tokens sobre la infraestructura CDN, mostrando que el control de acceso debe apoyarse en límites y patrones observables, no solo en la mera existencia del token [22, 4].

2.4.2. *Replay*

La reutilización de peticiones o credenciales válidas fuera de su contexto original es una amenaza natural en flujos de licencia y distribución. En OTT, el *replay* suele materializarse no como una rotura del cifrado, sino como la repetición de tokens válidos, URLs firmadas o solicitudes emitidas dentro de una ventana temporal aceptada. Por ello, la defensa eficaz exige acotar la vida útil de los artefactos de autorización y vincularlos a contexto, dispositivo o sesión. Cuando esto no se hace, el sistema sigue viendo mensajes formalmente válidos, aunque su uso ya no responda al escenario autorizado [22, 4].

2.4.3. *Credential sharing*

En el contexto OTT, el abuso de credenciales incluye tanto la compartición voluntaria de cuentas como el uso automatizado de credenciales robadas. OWASP define *credential stuffing* como la inyección automatizada de pares usuario-contraseña sustraídos en formularios de acceso, una técnica especialmente problemática cuando las credenciales se reutilizan entre servicios. Aunque este fenómeno no rompe el DRM en sentido estricto, sí invalida el supuesto comercial de un usuario legítimo por sesión o por hogar. Desde la perspectiva del proveedor, el problema real no es si el contenido está cifrado, sino si la identidad que activa la licencia coincide con el sujeto autorizado a consumirla [27, 22].

2.4.4. *CDN leeching*

El *CDN leeching* describe situaciones en las que terceros consumen masivamente contenido servido por la CDN aprovechando artefactos de autorización reutilizables o insuficientemente ligados al contexto. El activo que se explota no es la clave de descifrado, sino la capacidad de la infraestructura para seguir entregando segmentos aparentemente legítimos. Rodrigues et al. estudian este problema desde el uso compartido de tokens y proponen contramedidas basadas en límites aceptables de peticiones, degradación de servicio y análisis estadístico de accesos. Es una amenaza especialmente relevante para este TFM, porque conecta directamente el plano de distribución con el plano económico del servicio [22].

2.4.5. *Session hijacking*

OWASP describe el *session hijacking* como el compromiso del token de sesión mediante robo o predicción, con el objetivo de obtener acceso no autorizado. Aplicado a OTT, este riesgo implica que un atacante puede heredar el contexto de una sesión ya autenticada y utilizarla para solicitar manifiestos, segmentos o incluso licencias sin poseer necesariamente las credenciales originales. Desde el punto de vista del servicio, esta amenaza demuestra que la robustez del DRM es insuficiente si el contexto de sesión que lo invoca puede ser suplantado o reutilizado [28].

2.4.6. *Proxying*

El *proxying* introduce un intermediario que reenvía peticiones entre el cliente legítimo y consumidores no autorizados. La importancia de este vector reside en que el servicio puede observar tráfico que sigue pareciendo originado por un cliente válido, mientras que en realidad actúa como *relay* para terceros. En otras palabras, el DRM sigue funcionando desde el punto de vista del cliente original, pero el perímetro lógico del servicio queda desbordado. Este patrón se relaciona con escenarios de interposición y de secuestro de contexto, más cercanos al abuso operacional que a la criptanálisis del DRM [28, 4].

2.4.7. *Abuse automation*

La automatización amplifica todos los vectores anteriores. OWASP enfatiza que el *credential stuffing* se basa precisamente en herramientas automáticas capaces de probar credenciales a gran escala, mientras que Rodrigues et al. abordan el abuso de acceso a CDN desde patrones masivos y estadísticos. En OTT, la automatización permite probar credenciales, explotar tokens compartidos, multiplicar accesos concurrentes y simular poblaciones enteras de clientes. Por ello, la detección de abuso no puede limitarse a verificar si la licencia es válida, sino que debe incorporar ritmo de peticiones, correlación entre IPs, repetición de patrones y comportamiento concurrente [27, 22].

2.5. Estado del arte

La investigación reciente sobre DRM en sistemas OTT ha dejado de centrarse exclusivamente en el supuesto teórico de que el cifrado basta para frenar la re-

distribución. En su lugar, los trabajos más relevantes analizan tres dimensiones complementarias: la robustez real del cliente, las implicaciones de privacidad de los identificadores y el abuso operacional de la infraestructura de distribución [9, 10, 24, 22].

2.5.1. *Exploring Widevine for Fun and Profit*

El trabajo de Patat et al. supone, probablemente, la referencia más importante para cualquier estudio técnico contemporáneo sobre Widevine. Los autores presentan una ingeniería inversa detallada de sus componentes en Android, reconstruyen la *key ladder*, diseccionan mensajes intercambiados en el ecosistema DRM y desarrollan la herramienta WideXtractor para inspeccionar la actividad interna de Widevine durante la reproducción. Su hallazgo más relevante para este TFM es que la comprensión profunda del sistema permite comprometer la variante Android *software-only* L3 hasta recuperar su raíz de confianza. En términos de investigación, este resultado invalida la idea de que Widevine ofrece una garantía homogénea en todos los dispositivos: la seguridad efectiva depende críticamente de si el contenido se reproduce bajo L1 o bajo L3 [9].

2.5.2. *Your DRM Can Watch You Too*

En un plano distinto, aunque complementario, Patat, Sabt y Fouque estudian las implicaciones de privacidad de las implementaciones de Widevine EME en navegadores. Su trabajo demuestra que varios navegadores exponen el *Widevine Client ID* con poco o ningún consentimiento explícito y que las divergencias respecto a las guías de privacidad de EME pueden facilitar seguimiento de usuarios. Este enfoque amplía el estado del arte de forma decisiva: no solo importa si el DRM protege el contenido, sino también qué información del dispositivo y del usuario filtra en el proceso. Para el presente TFM, esta línea es valiosa porque muestra que el DRM puede introducir una tensión adicional entre robustez, control y privacidad [10, 4].

2.5.3. *Otros trabajos sobre DRM y protección OTT*

Rafi, Shepherd y Markantonakis realizan una comparación sistemática entre Widevine, FairPlay y PlayReady como soluciones DRM móviles de referencia. Su aportación principal consiste en consolidar propiedades comunes y señalar debilidades de diseño compartidas, incluyendo exposición a ataques por canales laterales microarquitectónicos y ausencia de preparación *post-quantum*. En

paralelo, Rodrigues et al. abordan un problema menos tratado en la literatura clásica DRM: el abuso de acceso a CDN mediante compartición de tokens y patrones anómalos de consumo. Finalmente, propuestas como SecureRights exploran la combinación de blockchain, *watermarking*, *hashing* perceptual e IPFS para reforzar la protección y la atribución de derechos en contenidos digitales. En conjunto, estas líneas revelan que la protección OTT actual desborda ampliamente el perímetro tradicional del licenciamiento DRM [24, 22, 29].

2.5.4. Comparativa de enfoques y limitaciones

Comparados entre sí, estos trabajos se complementan, pero también muestran lagunas. La ingeniería inversa de Widevine aporta profundidad sobre el cliente y la obtención de claves, pero no diseña controles perimetrales de producción. El estudio de privacidad en EME descubre fugas de identificadores y riesgos de seguimiento, pero no evalúa control de concurrencia ni abuso de CDN. La comparativa global de DRM móviles ofrece una visión valiosa de fortalezas y debilidades de diseño, aunque permanece en un plano más panorámico que experimental. Por su parte, los trabajos centrados en tokens y CDN capturan muy bien el abuso operacional, pero abstraen los detalles internos del CDM y del flujo de licencias. Precisamente ahí se sitúa el hueco que justifica este TFM: analizar Widevine en un laboratorio OTT reproducible y, sobre esa base, proponer defensas en profundidad que unan licencias, contexto, concurrencia y monitorización [9, 10, 24, 22, 29].

2.6. Limitaciones actuales de Widevine

La primera limitación de Widevine es conceptual: protege principalmente el camino de distribución cifrada y reproducción autorizada, pero no gobierna por sí mismo todo el proceso de acceso al servicio. El propio W3C especifica que EME deja la autenticación y autorización a la aplicación, lo que significa que un despliegue con Widevine puede seguir siendo débil si los tokens, las sesiones o la lógica de acceso están mal diseñados. Dicho de otro modo, tener DRM no equivale a tener control de acceso seguro [4, 5].

La segunda limitación es la heterogeneidad del parque de dispositivos. Widevine no ofrece el mismo nivel de protección en todos los entornos: mientras L1 aprovecha TEE y una raíz hardware de confianza, L3 ejecuta el CDM fuera de ese entorno y reduce sensiblemente la robustez del extremo cliente. La literatura académica ha demostrado que esta diferencia no es meramente teórica, sino explotable en la práctica para estudiar, instrumentar y comprometer variantes

software-only. Por tanto, un servicio OTT que permita la reproducción en L3 debe asumir que el DRM se encuentra mucho más cerca de una barrera de coste que de una frontera de seguridad fuerte [9, 24, 5].

La tercera limitación es operativa. Aun cuando el flujo audiovisual permanezca cifrado, persisten amenazas como compartición de credenciales, secuestro de sesión, reutilización de tokens y *CDN leeching*. Estas amenazas explotan el ecosistema que rodea al DRM y muestran que la seguridad real de OTT depende tanto del control contextual como de la criptografía. De ahí que la literatura reciente sobre abuso en CDN y automatización resulte especialmente pertinente para un TFM orientado a defensa en profundidad [22, 28, 27].

La cuarta limitación es la tensión entre robustez y privacidad. EME y los sistemas DRM admiten el uso de identificadores distintivos, y los trabajos recientes sobre Widevine en navegadores han evidenciado que dichos identificadores pueden derivar en capacidades de seguimiento superiores a las esperadas por el usuario. Esta característica no invalida el DRM como mecanismo de protección, pero sí obliga a tratarlo como un componente con implicaciones de seguridad y privacidad simultáneamente [4, 10].

En consecuencia, el estado del arte converge en una idea central: Widevine es una pieza importante del ecosistema OTT, pero no una solución autosuficiente frente al abuso real de distribución. Su eficacia depende del nivel de seguridad del dispositivo, de la calidad de la implementación, de la arquitectura de licencias y del conjunto de controles externos que envuelven al flujo de reproducción. Esta conclusión justifica la orientación del presente TFM hacia la auditoría experimental del ecosistema Widevine y el diseño posterior de mecanismos adicionales de autenticación, autorización contextual, control de concurrencia y detección de anomalías [4, 9, 24, 22].

3

Diseño del entorno experimental

Este capítulo describe el laboratorio diseñado para estudiar la seguridad de un sistema OTT que usa protección mediante DRM. El objetivo no es reproducir una plataforma comercial completa ni proporcionar un sistema de producción, sino construir un entorno controlado, reproducible e instrumentable en el que sea posible comparar un estado base deliberadamente permisivo con iteraciones progresivamente endurecidas. De este modo, las conclusiones de los capítulos posteriores pueden apoyarse en flujos observables de autenticación, entrega de licencias, distribución de contenido y gestión de sesiones.

El laboratorio se articula en tres fases. La Fase 0 representa el punto de partida: incorpora reproducción Widevine y contenido CENC local, pero mantiene una ruta de licencia sin autenticación para hacer reproducible el problema de investigación. La Fase 1 introduce un plano de control que vincula identidad, activo, dispositivo y sesión de reproducción. Finalmente, la Fase 2 añade persistencia de estado, telemetría y mecanismos de respuesta ante comportamiento anómalo. La separación permite evaluar no solo si un contenido puede reproducirse, sino bajo qué contexto se permite su reproducción y qué evidencias deja cada solicitud.

Aunque el título general del trabajo se refiere al *streaming* en directo, el banco de pruebas emplea activos de vídeo bajo demanda de corta duración. Esta elección reduce la complejidad asociada a la ingesta, codificación en tiempo real y gestión de un evento en directo, y permite aislar los controles de acceso que constituyen el objeto principal del estudio. Los mecanismos evaluados (licencias,

tokens, concurrencia, CDN y monitorización) son aplicables igualmente a flujos en directo, si bien su validación sobre producción de segmentos en tiempo real queda fuera del alcance de esta primera implementación.

3.1. Objetivos del laboratorio

El diseño del laboratorio persigue cinco objetivos. En primer lugar, debe simular un sistema OTT práctico: cliente, manifiesto DASH, segmentos de vídeo, servidor de origen, CDN, empaquetado CENC y flujo de licencia. En segundo lugar, debe permitir separar el plano criptográfico del plano de autorización. Es decir, el sistema debe poder demostrar que una licencia Widevine válida no implica, por sí sola, que el acceso al servicio sea legítimo.

En tercer lugar, el entorno debe hacer repetibles escenarios de abuso sin utilizar contenido comercial ni credenciales reales. Para ello se emplea un activo Widevine público de pruebas proporcionado por el ecosistema Shaka y un segundo activo CENC generado localmente con claves de laboratorio. El primero permite observar un flujo EME/Widevine funcional; el segundo permite mostrar, de forma controlada, que la disponibilidad conjunta de un MPD y de una clave de contenido conocida evita la necesidad de solicitar una licencia. Esta segunda demostración no pretende extraer claves de Widevine ni modificar un CDM, sino aislar la consecuencia de una hipotética filtración de claves sobre un activo CENC propio.

En cuarto lugar, el laboratorio debe permitir introducir controles de manera incremental. Las mejoras de las fases posteriores no sustituyen al DRM, sino que lo rodean de validaciones: autenticación, autorización por activo, token de sesión de corta duración, control de concurrencia, correlación de peticiones y respuesta ante anomalías. Por último, la solución debe ser portable y repetible. El despliegue mediante contenedores reduce las dependencias del equipo anfitrión y permite reiniciar cada fase en un estado conocido [30].

3.2. Arquitectura general

La arquitectura se organiza alrededor de dos planos complementarios. El plano de medios contiene los manifiestos MPD, los segmentos CENC y los componentes encargados de servirlos. El plano de control gobierna quién puede obtener configuración de reproducción, iniciar una sesión, solicitar una licencia o mantener el acceso. Esta separación es relevante porque permite evidenciar el problema central

del TFM: un contenido puede estar correctamente cifrado y, aun así, distribuirse de manera abusiva si el plano de control es demasiado permisivo.

La Tabla 3.1 resume los componentes principales y su papel. En Fase 0, algunos servicios se publican de forma deliberadamente amplia para facilitar la auditoría. En Fase 1 y Fase 2, el *control-plane* pasa a ser el único intermediario público hacia el origen y el servidor de licencias. Así se puede comparar la exposición inicial con una arquitectura en la que las llamadas sensibles se condicionan a un contexto de sesión.

Tabla 3.1: Componentes del entorno experimental

Componente	Responsabilidad	Fases
Cliente OTT con Shaka Player	Autenticación, selección de activo, carga del MPD y coordinación con EME/CDM.	0, 1 y 2
Varnish	Punto de entrada HTTP; en Fase 0 actúa como CDN básica y en fases posteriores como frontal del plano de control.	0, 1 y 2
<i>Origin</i> Nginx	Publicación de manifiestos, segmentos e inicializaciones locales.	0, 1 y 2
Shaka Packager	Segmentación y empaquetado CENC de los activos de laboratorio.	0
Servidor de licencias / <i>proxy</i>	Reenvío de <i>challenges</i> Widevine al servicio de pruebas y aplicación de política local.	0, 1 y 2
<i>Control-plane</i>	Gestión de identidad, sesiones, tokens, autorización y acceso protegido a contenido/licencias.	1 y 2
Redis	Persistencia de sesiones, eventos, puntuación de riesgo y bloqueos temporales.	2

3.2.1. Cliente

El cliente oficial se implementa como una aplicación web estática servida por Nginx y utiliza Shaka Player como reproductor DASH. Shaka Player se apoya en EME para descubrir el sistema de claves disponible en el navegador y coordinar los mensajes generados por el CDM. En el laboratorio, el cliente puede configurar

un servidor para el sistema `com.widevine.alpha` y enviar al mismo los *challenges* binarios generados durante la reproducción [4, 20].

La Fase 0 incorpora una pequeña plataforma OTT con dos usuarios de demostración. El usuario autorizado dispone de dos activos en su catálogo: un contenido Widevine público de pruebas y un contenido CENC generado localmente con una clave de laboratorio conocida. El usuario sin permisos puede autenticarse, pero el servidor no le devuelve la configuración de reproducción de ninguno de los activos. Esta distinción evita que la aplicación oficial cargue el MPD antes de verificar el *entitlement*; no obstante, la misma fase conserva de forma intencionada un reproductor externo y un endpoint `no_auth`, que se emplean para demostrar el *bypass* fuera de la plataforma.

En las fases posteriores, el cliente obtiene primero un *access token* y solicita después una sesión de reproducción. Solo tras recibir un *playback token* efímero puede pedir el manifiesto y la licencia protegidos. El cliente añade dicho token, el identificador de sesión y un identificador de dispositivo a ambas solicitudes. Además, incluye un *watermark* visible con información de usuario y sesión, cuya finalidad es aportar atribución durante la demostración, no impedir por sí mismo la captura de la señal.

3.2.2. CDN

Varnish representa la capa CDN y el frontal HTTP de cada fase. En el estado base enruta las peticiones `/content/*` hacia el origen y las peticiones de licencia hacia el servidor correspondiente. Los objetos multimedia locales pueden ser cacheados, mientras que las licencias se marcan como no cacheables. Esta separación permite analizar el valor de la CDN como acelerador de contenido y, simultáneamente, como perímetro de acceso.

La configuración de Fase 0 admite CORS amplio y conserva la ruta `/license/no_auth`. El objetivo es disponer de una evidencia reproducible de que un cliente ajeno a la plataforma puede emplear el MPD y la ruta de licencia observados durante la reproducción. En Fase 1 y Fase 2, Varnish limita las rutas expuestas a las operaciones de negocio necesarias, exige cabecera `Authorization` para manifiesto, contenido, licencia y operaciones sensibles, restringe el origen CORS al cliente correspondiente y añade un identificador de petición. Estas medidas no convierten a Varnish en el componente que toma las decisiones de autorización; su papel es reducir superficie de exposición y derivar las solicitudes al *control-plane*.

3.2.3. *Origin server*

El servidor de origen se implementa con Nginx y publica el árbol compartido `origin-content`. Este directorio contiene un ejemplo DASH sin cifrar para validar la cadena básica, los activos CENC generados localmente y sus segmentos MP4 fragmentados. El origen se mantiene como un servicio simple porque su función en el laboratorio es servir objetos, no aplicar reglas de negocio.

Esta decisión facilita la auditoría de un fallo habitual en entornos OTT: cuando el origen queda expuesto o accesible mediante una ruta alternativa, las restricciones implementadas en la CDN pierden eficacia. Por ello, Fase 0 publica el origen en el anfitrión como parte del escenario vulnerable, mientras que Fase 1 y Fase 2 lo mantienen únicamente dentro de la red Docker. La comparación permite comprobar que el ocultamiento de la topología no sustituye a la autorización, pero sí elimina un camino directo que de otro modo puede eludir el frontal de seguridad.

3.2.4. *Packager*

El empaquetado local se realiza con Shaka Packager en un contenedor. A partir de un MP4 de entrada, los scripts del laboratorio generan una inicialización MP4, segmentos `m4s` y un MPD DASH. El empaquetado no realiza *transcoding*; el activo de entrada ya se encuentra codificado. Esta separación entre codificación y empaquetado coincide con el modelo de Shaka Packager [20].

Se contemplan dos modos. El primero genera contenido CENC con señalización Widevine y claves en texto plano de laboratorio. El segundo genera un activo CENC con un KID y una clave conocidos, destinado exclusivamente a la demostración de *key leak*. Este último activo se reproduce mediante ClearKey desde Shaka Player y no llama al servidor de licencias. La prueba ilustra que, si la clave de contenido de un activo CENC se filtra, el servidor de licencias deja de ser necesario para consumir ese activo; no constituye una técnica de extracción de claves Widevine ni una emulación de su CDM.

3.2.5. *License server*

El servidor de licencias del laboratorio no implementa Widevine desde cero ni emite licencias de producción. Su función es recibir los mensajes binarios procedentes del cliente y actuar como *proxy* hacia el servicio público de pruebas utilizado por Shaka. Este enfoque permite observar un flujo Widevine funcional sin

almacenar claves de servicio, certificados de producción ni material criptográfico de terceros.

La diferencia entre fases está en quién puede llegar a ese *proxy*. En Fase 0 existe una ruta pública sin autenticación, conservada deliberadamente como vulnerabilidad. También existe una ruta de plataforma que comprueba identidad y autorización por activo antes de reenviar el *challenge*. En Fase 1 y Fase 2 el servidor de licencias deja de exponerse al anfitrión y solo acepta llamadas internas del *control-plane*. La solicitud interna incluye contexto de cuenta, dispositivo, activo y sesión; en Fase 2 se incorpora adicionalmente la puntuación de riesgo calculada para esa identidad. Esta estructura sigue el principio de que el DRM maneja el material de licencia, mientras que las reglas de acceso deben residir en la lógica de la aplicación [4, 5].

3.2.6. Proxy de seguridad

El *proxy* de seguridad se materializa progresivamente. En Fase 0, el servicio de licencias contiene una comprobación mínima de catálogo y permisos para la interfaz oficial, pero coexiste con la ruta vulnerable necesaria para el experimento. Por tanto, no debe considerarse una defensa completa.

En Fase 1 se incorpora un *control-plane* dedicado. Este componente autentica al usuario, emite un token de acceso firmado mediante HMAC, crea una sesión de reproducción y genera un token de reproducción de vida corta. Antes de servir el manifiesto o contenido y antes de reenviar una licencia, verifica que el token corresponde a una sesión activa, al dispositivo declarado y al activo solicitado. También limita el número de sesiones activas por cuenta, renueva el token mediante *heartbeat* y permite detener la sesión de forma explícita.

Fase 2 conserva estas validaciones y añade Redis para guardar sesiones, eventos, riesgos y bloqueos. El *control-plane* registra señales tales como intentos de autenticación, creación de sesión, solicitudes de contenido, *heartbeats*, solicitudes de licencia y operaciones administrativas. A partir de estas señales, aplica reglas sencillas sobre actividad desde múltiples IP, ráfagas de licencias, exceso de concurrencia y cadencia anómala. El resultado es un laboratorio capaz de comparar una autorización puntual con una decisión de acceso que evoluciona según el comportamiento observado.

3.3. Tecnologías utilizadas

3.3.1. Docker

Docker se emplea para desplegar cada fase como un conjunto de servicios aislados y conectados mediante redes internas. La plataforma permite empaquetar una aplicación y sus dependencias en contenedores, facilitando que el mismo escenario se ejecute de manera consistente en distintos equipos [30]. En este trabajo, Docker Compose define imágenes, puertos, variables de entorno, volúmenes y redes de cada topología.

La contenerización resulta especialmente útil en una evaluación de seguridad porque permite separar las fases sin modificar una instalación compartida. Por ejemplo, Fase 0 expone los puertos de origen y licencia para representar una superficie amplia, mientras que Fase 1 y Fase 2 los mantienen en redes internas. La misma base de contenido se monta como volumen de solo lectura, lo que evita duplicar activos y mantiene comparables los escenarios.

3.3.2. Python

Python se ha utilizado como herramienta auxiliar para el tratamiento de evidencias y métricas. Su biblioteca estándar y su ecosistema han facilitado automatizar la lectura de registros estructurados, normalizar marcas temporales, agrupar peticiones por cuenta o sesión y generar tablas de resultados [31]. No ha formado parte del camino crítico de reproducción implementado en los contenedores, que se ha apoyado principalmente en Node.js y JavaScript; su papel ha sido facilitar el análisis reproducible de PCAP, logs y métricas obtenidos durante la evaluación.

3.3.3. Widevine

Widevine aporta el flujo DRM real utilizado en las pruebas de reproducción protegida. El cliente carga un activo DASH de demostración con señalización Widevine, el navegador selecciona el CDM compatible a través de EME y Shaka Player reenvía el mensaje de licencia al endpoint configurado. Dicho endpoint es local al laboratorio y actúa como intermediario antes de llegar al servidor de pruebas.

Esta integración es suficiente para observar las peticiones de licencia y evaluar la política que las rodea, pero no pretende sustituir un acuerdo de producción

con Widevine. La generación y operación de licencias comerciales requieren credenciales y procesos de aprovisionamiento ajenos al alcance del laboratorio. La documentación de Widevine y de Shaka Packager sitúa precisamente el servidor de claves, el identificador de contenido y la política de licencia como piezas externas a la mera reproducción del cliente [5, 21].

3.3.4. HTTP

Todo el intercambio del laboratorio se produce sobre HTTP en la red local de Docker. Las rutas de autenticación y control transportan JSON, mientras que las solicitudes Widevine se reenvían como cuerpos binarios. Los manifiestos, inicializaciones y segmentos se entregan como objetos HTTP, respetando las cabeceras **Range** cuando procede. Esta homogeneidad permite relacionar la observación de red con las decisiones de aplicación: una misma captura puede mostrar el login, la creación de sesión, la solicitud de licencia y la descarga de segmentos.

En un despliegue de producción, estos flujos deben emplear HTTPS, y EME exige contexto seguro para la interacción con contenido cifrado [4]. El uso de HTTP y puertos locales responde a la necesidad de simplificar el laboratorio y facilitar la inspección; no debe interpretarse como una configuración de producción.

3.3.5. Linux

Los contenedores del laboratorio utilizan imágenes Linux ligeras. Nginx y Varnish se ejecutan como servicios de distribución, mientras que los servicios de aplicación se construyen sobre imágenes Node.js. Este modelo permite reproducir una topología de red interna, resolver los servicios por nombre y limitar qué puertos se publican hacia el anfitrión.

Linux también aporta un contexto natural para la instrumentación de red y la inspección de procesos de los contenedores. Sin embargo, el análisis se centra en las interfaces y registros del laboratorio, no en evaluar la seguridad del núcleo del sistema operativo ni en endurecer exhaustivamente cada imagen de contenedor.

3.3.6. Wireshark

Wireshark se selecciona como herramienta de inspección visual de tráfico para revisar intercambios HTTP, cabeceras, secuencias temporales y posibles rutas

alternativas de acceso. Su documentación incluye guías de usuario, filtros de visualización y herramientas de línea de comandos asociadas, lo que lo hace adecuado para documentar capturas de laboratorio [32].

En este TFM, Wireshark se ha empleado para contrastar el flujo observable desde el cliente oficial con el generado por los clientes externos de Fase 0. El objetivo no ha sido inspeccionar material criptográfico interno del CDM, sino identificar qué URLs, cabeceras, tokens, MPD y endpoints de licencia han quedado expuestos para un usuario que observa sus propias comunicaciones.

3.3.7. *tcpdump*

tcpdump ha complementado a Wireshark como mecanismo de captura no interactiva. Se ha utilizado para obtener ficheros PCAP en puntos concretos de la red del laboratorio y conservar evidencias revisables posteriormente. La captura se ha acotado a los puertos y contenedores de interés, evitando recoger tráfico ajeno al experimento.

La combinación de *tcpdump* para la adquisición y Wireshark para la exploración ha permitido separar el momento de la prueba del análisis posterior. Los PCAP se han tratado como evidencia de los escenarios reproducidos y no como fuente de claves DRM: el contenido de licencia Widevine puede viajar cifrado o en formato propietario y las claves no son exportadas por EME al código de la aplicación.

3.3.8. Credenciales de acceso y reproducción

Las fases endurecidas utilizan credenciales firmadas con atributos de acceso y reproducción. La implementación incluye tipo de credencial, cuenta, dispositivo, activo, identificador de sesión, ámbito y expiración. Se trata de un mecanismo deliberadamente reducido para el laboratorio, cuyo propósito es aportar contexto verificable a las peticiones y no reproducir un proveedor de identidad completo.

La distinción entre *access token* y *playback token* permite reducir el privilegio de cada credencial. El primero autentica al usuario frente al *control-plane*; el segundo queda ligado a una sesión concreta y autoriza el manifiesto, la lectura de contenido, la solicitud de licencia y el envío de *heartbeats*. Este diseño permite estudiar el efecto de la expiración, la revocación y el control de concurrencia sobre una operación especialmente sensible: el acceso a contenido protegido.

3.4. Diseño de la infraestructura OTT

3.4.1. Segmentación

El laboratorio usa MPEG-DASH, cuyo manifiesto se denomina MPD (*Media Presentation Description*), como formato de entrega. El MPD describe una representación de vídeo, su inicialización y la plantilla de segmentos que debe recuperar el reproductor. Para los activos empaquetados localmente, Shaka Packager genera un fichero de inicialización y segmentos `m4s`; el cliente obtiene estos objetos según el MPD. Esta organización reproduce la estructura básica de una distribución adaptativa, aunque el activo de laboratorio se limita a una representación de baja resolución y no persigue evaluar algoritmos de adaptación de bitrate.

La segmentación permite observar una cuestión esencial para la seguridad OTT: una reproducción no es una única descarga. Una sesión genera una sucesión de solicitudes al manifiesto, a la inicialización y a los segmentos. Por ello, autorizar solo la primera petición no basta para controlar el acceso. Las fases endurecidas tratan la sesión de reproducción como contexto persistente y la renuevan mediante *heartbeat*, mientras que Fase 0 ilustra las consecuencias de mantener rutas de distribución o licencia excesivamente abiertas.

3.4.2. Flujo de vídeo

El flujo base comienza en el cliente, que autentica al usuario cuando la fase lo requiere y solicita la configuración del activo. Para el activo Widevine, Shaka Player descarga el MPD, detecta la señalización DRM y crea una sesión EME. El CDM genera el mensaje de licencia; el cliente lo entrega al endpoint del laboratorio y, si la política de acceso lo permite, el *proxy* lo reenvía al servicio de pruebas. Tras procesar la licencia, el cliente consume los segmentos cifrados descritos por el MPD.

El activo CENC local sigue el mismo principio de entrega segmentada, aunque utiliza una clave de laboratorio configurada expresamente en el reproductor. Su función es separar dos cuestiones: el flujo normal de licencia y el impacto de disponer ya del material de descifrado. De este modo, el laboratorio no atribuye a Widevine una capacidad que no tiene, que sería la de controlar claves ya comprometidas fuera del flujo de licencia, y permite documentar ese límite de forma segura.

3.4.3. *Packaging*

Los scripts de empaquetado reciben un MP4 local y ejecutan Shaka Packager dentro de un contenedor. En el modo CENC/Widevine de laboratorio se especifican un KID, una clave de prueba, el esquema `cenc` y la señalización Widevine. El MPD resultante contiene elementos `ContentProtection`, el KID por defecto y datos PSSH, que permiten al reproductor reconocer que el activo requiere un sistema de claves compatible.

El modo de clave conocida genera un activo independiente con el mismo esquema de cifrado CENC y metadatos de sistema común. El KID y la clave forman parte explícita del experimento y se almacenan solo como valores de laboratorio. Esta separación evita confundir la prueba de impacto de una clave expuesta con la integración Widevine de demostración, cuya licencia es proporcionada por un servicio externo de pruebas.

3.4.4. *Distribución*

El contenido local se monta en el contenedor de origen como volumen de solo lectura. En Fase 0, Varnish reescribe las rutas `/content/*` hacia el origen y puede cachear objetos multimedia. La misma fase deja visibles los puertos del origen y del servicio de licencias para mostrar que una CDN sin controles de negocio no constituye una barrera suficiente.

En Fase 1 y Fase 2, Varnish entrega las solicitudes al *control-plane*. Este valida el token de reproducción y, para el contenido local, obtiene el objeto desde el origen interno. Aunque el activo Widevine público se entrega desde su ubicación de pruebas y no atraviesa el origen local, mantener el flujo local de contenido permite evaluar el patrón de acceso que seguiría una CDN privada. La convivencia de ambos activos es deliberada: uno aporta un flujo Widevine funcional y el otro permite controlar completamente la cadena CENC dentro del laboratorio.

3.5. Integración de Widevine

3.5.1. Configuración

La configuración Widevine del cliente asocia el sistema `com.widevine.alpha` con el endpoint de licencia de la fase activa. En Fase 0, la plataforma oficial usa `/platform/license`, mientras que la ruta `/license/no_auth` se conserva para

el cliente externo y la auditoría. En Fase 1 y Fase 2, el endpoint público siempre corresponde al *control-plane*; el servidor que contacta con el servicio de pruebas solo es accesible desde la red Docker.

3.5.2. Flujo de licencias

El flujo de licencia se resume en los siguientes pasos:

1. El reproductor descarga el MPD y detecta el contenido protegido mediante los metadatos CENC.
2. EME y el CDM generan un *challenge* asociado a la sesión de claves.
3. El cliente envía el *challenge* binario al endpoint de licencia configurado.
4. El componente de control valida el contexto de acceso. En Fase 0, esta comprobación puede coexistir con una ruta deliberadamente sin autenticación; en Fase 1 y Fase 2 exige token y sesión activos.
5. Si la política lo permite, el *proxy* interno reenvía el mensaje al servicio Widevine de pruebas y devuelve su respuesta binaria al cliente.
6. El CDM procesa la licencia y el reproductor continúa la reproducción de segmentos cifrados.

La propiedad importante no es solo que el paso quinto ocurra, sino que la decisión de permitirlo esté ligada a información de negocio. La Fase 0 demuestra que una ruta de licencia no autenticada elimina esa ligadura. Las fases siguientes añaden progresivamente identidad, activo, dispositivo, sesión, expiración y riesgo, que son los elementos evaluados en la propuesta de defensa en profundidad.

3.6. Configuración del entorno de captura y monitorización

3.6.1. PCAP

La adquisición de tráfico se ha realizado en dos niveles. El primero ha consistido en observar la comunicación desde el navegador durante una reproducción oficial y durante una reproducción externa de Fase 0. El segundo ha consistido en

capturar el tráfico asociado a los contenedores o a los puertos publicados, acotando las capturas a HTTP, DASH y licencias. Esta estrategia ha permitido conservar PCAP comparables para cada escenario, con marcas temporales que relacionan la solicitud de manifiesto, la emisión de licencia y la descarga de segmentos.

El análisis de las capturas ha buscado evidencias de superficie expuesta: URL de MPD, endpoint de licencia, cabeceras de autorización, redirecciones, respuestas de error y secuencia de solicitudes. No se ha interpretado ningún PCAP como mecanismo para obtener claves de Widevine, ya que el alcance se ha limitado al comportamiento observable del sistema y a los datos de control transmitidos por la aplicación.

3.6.2. *Logging*

El registro se incrementa por fases. Fase 0 conserva principalmente los logs de los contenedores y las respuestas HTTP necesarias para explicar el flujo base. Fase 1 añade trazabilidad de peticiones mediante identificadores de solicitud en Varnish y mantiene en memoria las sesiones de reproducción, sus *heartbeats* y su estado. Fase 2 formaliza la observabilidad mediante eventos JSON almacenados en Redis, incluyendo autenticación, creación y caducidad de sesión, solicitudes de licencia, solicitudes de contenido, cambios de riesgo y bloqueos.

Este esquema proporciona dos clases de evidencia. La primera permite reconstruir un incidente concreto: qué cuenta, dispositivo y sesión solicitaron una licencia o un objeto. La segunda permite observar tendencias: número de sesiones activas, frecuencia de licencias, fallos de concurrencia o actividad desde múltiples IP. La retención de eventos del laboratorio se mantiene deliberadamente acotada para simplificar el despliegue; un sistema de producción requeriría rotación, almacenamiento duradero, acceso restringido y políticas de privacidad.

3.6.3. Métricas

Las métricas se han seleccionado para responder a la hipótesis del trabajo y permitir una comparación entre fases. Se han registrado el resultado de la autorización de licencia, el número de sesiones activas por cuenta, el número de solicitudes de contenido y licencia, el tiempo de respuesta de los endpoints críticos y el número de accesos bloqueados. En Fase 2 se ha añadido la evolución de la puntuación de riesgo, las razones que la componen y el número de bloqueos automáticos o manuales.

Además de las métricas de seguridad, se ha observado el impacto operativo de

las defensas: latencia adicional de la obtención de licencia, errores de reproducción atribuibles a la expiración del token y comportamiento ante una sesión detenida o bloqueada. La evaluación del Capítulo 7 ha comparado el sistema base, donde un cliente externo puede reutilizar una ruta sin autenticación, con los sistemas endurecidos, donde la licencia queda condicionada al contexto de reproducción. Esta comparativa ha permitido distinguir de manera cuantitativa entre disponer de DRM y gobernar efectivamente el acceso al servicio.

Para favorecer la trazabilidad y la reproducción de los experimentos, el código fuente, las configuraciones Docker, los laboratorios, la memoria y las evidencias del proyecto se encuentran disponibles en un repositorio público de GitHub.¹ El repositorio incluye además documentación específica para desplegar cada fase e interpretar los artefactos obtenidos durante la evaluación.

¹Repositorio público del TFM: <https://github.com/yisusrg05/TFM>

4

Auditoría de seguridad del ecosistema Widevine

Este capítulo presenta la auditoría de la Fase 0 del laboratorio descrito en el capítulo anterior. Esta fase constituye la línea base del trabajo: dispone de reproducción Widevine funcional, un cliente OTT, una CDN local, un servidor de origen y un proxy de licencias, pero conserva deliberadamente rutas y decisiones de autorización insuficientes. El objetivo de la auditoría no es romper la criptografía de Widevine ni extraer claves del CDM, sino comprobar si un tercero puede consumir el contenido o utilizar la infraestructura fuera del contexto autorizado por la plataforma.

La distinción es esencial para interpretar los resultados. Widevine protege el contenido cifrado y procesa licencias dentro de un CDM compatible, mientras que la autenticación del usuario, la autorización por activo y el control de las sesiones pertenecen a la aplicación que integra el DRM [4, 5]. Por tanto, los hallazgos descritos a continuación afectan principalmente al ecosistema que rodea a Widevine. La auditoría evalúa si la plataforma mantiene una relación verificable entre identidad, activo, sesión, licencia y distribución, y no si el algoritmo de cifrado del DRM puede ser vulnerado.

4.1. Metodología de auditoría

La auditoría se ha realizado mediante un enfoque mixto de caja negra y caja gris. La perspectiva de caja negra ha permitido observar el sistema como lo haría un usuario o un tercero con acceso a las URLs públicas. Por su parte, la perspectiva de caja gris ha incorporado el conocimiento de la topología Docker, las rutas de Varnish y la lógica del servidor para explicar por qué ha sido posible cada comportamiento. La combinación de ambas evita limitar el análisis a una mera revisión de código y, al mismo tiempo, permite relacionar cada resultado observable con una decisión concreta de arquitectura.

El alcance se ha restringido a los componentes de Fase 0: cliente oficial, reproductor externo, laboratorio DRM, demostración de clave conocida, Varnish, Nginx y proxy de licencias. Se han utilizado exclusivamente usuarios, claves y activos de laboratorio. El contenido Widevine y el servicio de licencias utilizados corresponden a recursos públicos de prueba del ecosistema Shaka; no se han empleado contenidos comerciales, credenciales de terceros ni técnicas de extracción de claves.

La matriz experimental se ha iniciado estableciendo el comportamiento legítimo de referencia. En primer lugar, se ha comprobado la reproducción de los dos activos con el usuario autorizado y el rechazo del usuario sin permisos. A continuación, se han repetido las solicitudes desde clientes ajenos a la plataforma y mediante rutas alternativas. Este orden permite atribuir los resultados a las decisiones de seguridad de Fase 0 y no a una falta de funcionalidad del laboratorio.

4.1.1. Caja negra

El análisis de caja negra ha partido de las interfaces expuestas en el equipo anfitrión. El cliente oficial se ha publicado en el puerto 3000; el reproductor externo, el laboratorio DRM y la demostración de clave conocida se han publicado, respectivamente, en los puertos 3001, 3002 y 3003. Varnish ha actuado como CDN en el puerto 8080, mientras que el origen y el servicio de licencias también han permanecido accesibles directamente en 8081 y 8082.

Desde esta perspectiva se han planteado tres perfiles. El primero ha correspondido a un usuario autenticado y autorizado, utilizado para establecer el comportamiento legítimo. El segundo ha sido un usuario autenticado sin *entitlement*, con el que se ha comprobado si la plataforma impedía obtener la configuración del activo. El tercero ha correspondido a un cliente externo sin sesión en la plataforma, que únicamente disponía de la URL del MPD y de una dirección de

licencia observables.

Las pruebas han permitido verificar los códigos HTTP, los estados visibles del reproductor, el avance temporal del vídeo y la posibilidad de acceder a rutas alternativas. Una reproducción solo se ha considerado iniciada cuando el elemento de vídeo no estaba pausado, su tiempo de reproducción avanzaba y el navegador informaba de datos multimedia disponibles.

4.1.2. Caja gris

El análisis de caja gris ha incluido la revisión de la configuración de los contenedores, las reglas de Varnish y la lógica del servidor de Fase 0. En particular, se ha examinado qué servicios se publicaban en el anfitrión, qué rutas se enviaban al origen o al proxy de licencias, cómo se generaban los tokens de acceso y qué validaciones se ejecutaban antes de reenviar un *challenge* Widevine.

Esta revisión ha evidenciado que la plataforma mantiene dos caminos de licencia. El camino oficial, `/platform/license`, exige un token de acceso y comprueba que el usuario disponga del activo indicado. El camino alternativo, `/license/no_auth`, reenvía el mismo tipo de mensaje binario sin exigir identidad ni autorización. La coexistencia de ambos caminos es intencionada en Fase 0 y permite demostrar que un control correcto en la aplicación pierde eficacia si permanece disponible una ruta equivalente que no aplica la misma política.

La revisión de caja gris también ha permitido comprobar que los tokens de acceso son identificadores UUID almacenados en memoria. No contienen expiración, ámbito, activo, dispositivo o identificador de sesión, y permanecen aceptados hasta que el proceso se reinicia. Asimismo, el origen y el servidor de licencias se publican directamente en el anfitrión, de modo que el paso por Varnish no es obligatorio.

4.1.3. Análisis de tráfico

El tráfico se ha observado en tres niveles complementarios. En el navegador se han registrado las peticiones de manifiesto y licencia generadas por Shaka Player. En la capa HTTP se han inspeccionado los códigos de estado y las cabeceras de respuesta. Finalmente, los registros de los contenedores han permitido confirmar las solicitudes recibidas por Nginx durante la descarga del MPD, la inicialización y los segmentos locales.

En el flujo externo Widevine se ha registrado una solicitud al MPD público,

dos solicitudes al endpoint `/license/no_auth` y dos respuestas binarias de licencia de 714 y 1229 bytes. A continuación, el laboratorio ha registrado el evento `PLAYBACK_STARTED` y el elemento de vídeo ha alcanzado el estado de reproducción con datos suficientes. En el activo CENC local, los logs del origen han mostrado solicitudes al fichero de inicialización y a los segmentos `m4s`, sin que se haya producido ninguna petición a un servidor de licencias.

Las cabeceras han confirmado, además, que los objetos locales atravesaban la caché de Varnish: una segunda solicitud al MPD ha devuelto `X-Cache:HIT`. El mismo MPD también se ha podido obtener directamente desde el origen publicado en 8081. Por tanto, el análisis no solo ha evidenciado que el contenido estaba disponible, sino también que existían dos caminos de distribución: uno a través de la CDN y otro directo hacia el origen.

No se ha intentado interpretar el contenido criptográfico de las respuestas Widevine ni recuperar claves a partir del tráfico. EME no expone las claves del CDM al código JavaScript de la página, y una captura de red no debe confundirse con una vía de exportación del material protegido [4, 9]. Para obtener los resultados de este capítulo han sido suficientes la secuencia de peticiones, los tamaños de respuesta, el estado del reproductor y los logs del servidor.

4.1.4. Reproducción de ataques

Los escenarios se han reproducido de forma controlada y repetible. Cada prueba se ha iniciado con los siete contenedores de Fase 0 activos y sin reinicios. En primer lugar, se ha verificado el comportamiento legítimo; a continuación, el rechazo del usuario sin permisos y, por último, los caminos alternativos disponibles para un cliente externo.

La reproducción se ha limitado a acciones que un usuario podría realizar sobre sus propias comunicaciones: observar URLs, introducirlas en otro reproductor, reutilizar un token emitido por el laboratorio y acceder a puertos publicados localmente. En la demostración de clave conocida se han empleado un KID y una clave generados expresamente para el TFM. No se ha modificado el CDM, no se ha instrumentado Widevine L3 y no se han recuperado claves de terceros. Este límite mantiene la prueba alineada con el objetivo del trabajo: auditar la integración OTT y no desarrollar una herramienta de elusión del DRM.

4.2. Análisis del flujo de licencias

En el flujo oficial, el usuario inicia sesión mediante `/auth/login`. El servidor comprueba las credenciales de laboratorio, genera un token aleatorio y devuelve un catálogo en el que cada activo aparece como permitido o denegado. Cuando el usuario autorizado selecciona el activo Widevine, el cliente solicita su configuración mediante `/playback/config`. Shaka Player descarga después el MPD, crea una sesión EME y envía el *challenge* a `/platform/license` con el token de acceso y el identificador de activo.

El servidor vuelve a comprobar la identidad y el *entitlement* antes de reenviar el mensaje al servicio Widevine de pruebas. Este camino ha permitido una reproducción real: el elemento de vídeo se ha mantenido en ejecución, ha alcanzado un estado de datos completos y ha avanzado temporalmente. Por tanto, Fase 0 dispone de un flujo protegido funcional dentro de la interfaz oficial.

No obstante, el mismo proxy publica `/license/no_auth`. Este endpoint acepta el cuerpo binario generado por el CDM y lo reenvía sin token, usuario o activo. El reproductor externo se ha configurado para utilizar `com.widevine.alpha` con esa URL, ha obtenido respuestas de licencia válidas y ha reproducido el mismo activo sin iniciar sesión. La licencia ha seguido siendo procesada correctamente por Widevine; el fallo de seguridad se ha producido porque la aplicación ha proporcionado acceso al servicio de licencia fuera de su política de negocio.

4.2.1. Validaciones existentes

Fase 0 no carece por completo de controles. La ruta oficial valida credenciales, diferencia dos cuentas y comprueba el *entitlement* antes de devolver la configuración. El usuario autorizado ha podido reproducir tanto el activo Widevine como el activo CENC local. El usuario sin permisos ha podido autenticarse, pero ha recibido ambos activos como denegados y los botones de reproducción han permanecido deshabilitados.

El endpoint `/platform/license` repite la comprobación de autorización en cada *challenge*. Esta segunda validación es positiva porque evita confiar únicamente en la decisión adoptada al mostrar el catálogo. También se impide solicitar una licencia Widevine para un activo declarado como ClearKey.

Sin embargo, estas validaciones se aplican solo al camino oficial. La ruta `/license/no_auth` no exige token, y las rutas de contenido local tampoco requieren autorización. El identificador de activo se recibe en una cabecera controlada por el cliente y no se contrasta con el KID o el PSSH contenido en el mensa-

je Widevine. Además, no se valida dispositivo, IP, agente de usuario, sesión de reproducción, número de accesos simultáneos ni ritmo de peticiones. En consecuencia, la autorización existente es puntual y no representa una política extremo a extremo.

4.2.2. Dependencias de confianza

La prueba depende de varios componentes externos e internos. El navegador confía en Shaka Player para coordinar EME y en el CDM instalado para procesar la licencia. El MPD Widevine se descarga desde el almacenamiento público de demostración de Shaka, y el proxy local confía en el servicio CWIP para emitir la respuesta. La CDN local confía en el origen Nginx para entregar los objetos, mientras que el cliente confía en que las URLs proporcionadas por el backend corresponden al activo autorizado.

Estas dependencias delimitan lo que demuestra el experimento. El éxito del escenario externo acredita que el proxy de licencias local acepta solicitudes sin identidad y que Widevine reproduce cuando recibe una licencia válida. No acredita que se haya comprometido el servicio CWIP, el CDM o la criptografía de Widevine.

La configuración usa además HTTP y permite CORS desde cualquier origen. CORS no constituye un mecanismo de autenticación y solo limita determinados comportamientos de navegadores; un cliente nativo o una herramienta de línea de comandos no queda sometido a esa política. Por ello, restringir CORS sería una medida de reducción de superficie, pero no resolvería por sí sola la ausencia de autorización en la licencia o el contenido.

4.3. Evaluación de vulnerabilidades

La Tabla [4.1](#) resume los principales hallazgos. La severidad se ha asignado cualitativamente atendiendo al impacto sobre la confidencialidad del contenido, la facilidad de explotación y la necesidad de disponer de credenciales previas.

Tabla 4.1: Resumen de hallazgos de seguridad en Fase 0

ID	Hallazgo	Impacto principal	Severidad
F0-01	Licencia pública sin autenticación	Reproducción Widevine fuera de la plataforma sin identidad ni permiso.	Crítica
F0-02	Exposición de origen y contenido	Elusión del frontal CDN y descarga de MPD y segmentos sin autorización.	Alta
F0-03	Sesión sin expiración ni contexto	Reutilización de credenciales durante toda la vida del proceso.	Alta
F0-04	Ausencia de control de concurrencia	Uso simultáneo y compartición de cuenta sin detección.	Alta
F0-05	CORS y transporte permisivos	Facilita integración desde terceros y expone datos de control en HTTP.	Media

4.3.1. CDN *leeching*

La CDN permite recuperar el MPD y los segmentos locales sin token. El conocimiento de una ruta como `/content/dash-known-key/stream.mpd` resulta suficiente para que cualquier cliente alcance el activo. Varnish cachea el objeto y lo sirve con `Access-Control-Allow-Origin:*`, por lo que la infraestructura puede consumir ancho de banda aunque la solicitud no proceda de la aplicación oficial.

El origen también está publicado en 8081. El mismo MPD obtenido mediante la CDN se ha podido descargar directamente desde Nginx, evitando las reglas de Varnish. Este resultado reproduce una condición frecuente de *bypass*: proteger únicamente el frontal no es efectivo si permanece disponible una ruta alternativa hacia el origen. Los mecanismos de URLs o cookies firmadas descritos por proveedores de CDN persiguen precisamente que conocer una ruta no sea suficiente para acceder al objeto [11, 12].

4.3.2. Reutilización de sesiones

Fase 0 no crea una sesión de reproducción diferenciada. El token de login es la única referencia de estado y puede reutilizarse para pedir configuraciones

y licencias oficiales mientras permanezca en memoria. No existe *logout*, lista de revocación, expiración temporal, nonce de un solo uso ni asociación entre token y una reproducción concreta.

Si el token se filtra, otro cliente puede presentar el mismo valor y actuar como el usuario hasta que el proceso se reinicie. El servidor tampoco registra un cambio de IP o dispositivo que permita detectar el traslado de la credencial. Este comportamiento no rompe la firma de un token ni el protocolo Widevine; explota la ausencia de ciclo de vida y contexto alrededor de la credencial. Los mecanismos implementados en las fases endurecidas han separado el token de autenticación del token de reproducción y han reducido la ventana de reutilización.

4.3.3. *Credential sharing*

La compartición de credenciales es viable porque la cuenta no se vincula a dispositivos y el servidor no conserva información sobre ubicaciones o patrones de acceso. Varias personas pueden utilizar el mismo correo y contraseña y obtener tokens distintos sin que el sistema relacione esas autenticaciones entre sí. El DRM continuará emitiendo licencias válidas porque cada petición llega desde un navegador compatible y el plano de negocio no detecta que la identidad se está compartiendo.

En el laboratorio no se han simulado desplazamientos geográficos reales ni poblaciones masivas de clientes. No obstante, la revisión de caja gris confirma que no existe ninguna función capaz de distinguir esos casos. Por ello, el hallazgo se fundamenta en la ausencia verificable del control y no en una inferencia sobre la criptografía. La compartición de cuentas pertenece al plano de política de servicio y requiere señales adicionales de cuenta, dispositivo, red y concurrencia [15, 14].

4.3.4. **Abuso de concurrencia**

El servidor no mantiene sesiones activas ni contabiliza reproducciones simultáneas. Cada login genera un UUID independiente y no existe un límite por cuenta. Las solicitudes de licencia tampoco crean un registro de reproducción que pueda detenerse o expirar mediante *heartbeat*. En consecuencia, una misma identidad puede iniciar varias reproducciones y obtener varias licencias sin recibir una respuesta distinta.

Esta carencia amplifica los efectos de la compartición de credenciales y del robo de tokens. Incluso si todas las licencias fueran criptográficamente válidas, la plataforma no podría determinar si corresponden al número de streams permitido

por el plan. La defensa requiere un estado de sesión independiente del DRM, mantenido por la aplicación y consultado en los puntos de contenido y licencia.

4.3.5. Ausencia de validaciones contextuales

La decisión de Fase 0 se reduce a dos condiciones: que el token exista en memoria y que el activo figure en la lista de permisos. No se verifica que la petición conserve la misma IP, dispositivo o agente de usuario que el login. Tampoco se comprueba una sesión de reproducción activa, el ritmo de licencias, la relación entre KID y activo, el estado de riesgo o la concurrencia.

La ruta pública elimina incluso esas dos condiciones. El navegador externo ha podido obtener dos respuestas de licencia y comenzar la reproducción sin enviar credenciales. Esta es la evidencia más directa de la hipótesis del TFM: el CDM y Widevine pueden funcionar correctamente y, al mismo tiempo, la plataforma puede autorizar de forma insuficiente el acceso al servicio.

La demostración de clave conocida añade un límite diferente. Al introducir manualmente el KID y la clave de laboratorio, se reproduce el activo CENC sin configurar un servidor de licencias. Esta prueba no constituye un hallazgo contra Widevine, puesto que la clave no se ha extraído del CDM. Ilustra que, una vez comprometido el material de contenido, las validaciones del servidor de licencias dejan de intervenir para ese activo.

4.4. Escenarios de ataque reproducidos

Los escenarios se han diseñado para responder a preguntas concretas y evitar pruebas ambiguas. En primer lugar, se ha verificado que el sistema pudiera distinguir un usuario permitido de otro sin *entitlement*. A continuación, se ha comprobado si esa decisión seguía siendo efectiva al abandonar el cliente oficial. Finalmente, se han evaluado las rutas directas y el efecto de disponer de una clave conocida.

4.4.1. Procedimiento

El procedimiento seguido ha sido el siguiente:

1. Levantar los siete contenedores de Fase 0 y comprobar que cliente, CDN, origen y proxy respondían en sus puertos previstos.

2. Iniciar sesión con el usuario autorizado, obtener el catálogo y reproducir el activo Widevine mediante `/platform/license`.
3. Reproducir desde la misma plataforma el activo CENC local configurado con ClearKey.
4. Iniciar sesión con el usuario sin permisos y comprobar el estado de ambos controles de reproducción.
5. Abrir el reproductor externo sin iniciar sesión, mantener el MPD público y configurar `/license/no_auth` como servidor Widevine.
6. Repetir el escenario en el laboratorio DRM, registrar soporte EME, peticiones y tamaños de las respuestas de licencia.
7. Solicitar el MPD local tanto a través de Varnish como directamente al origen publicado en 8081.
8. Introducir el KID y la clave de laboratorio en la demostración ClearKey y reproducir sin servidor de licencias.
9. Solicitar al laboratorio que intentara mostrar claves Widevine y registrar la imposibilidad de exportarlas mediante EME.

Cada resultado se ha contrastado con el estado del elemento de vídeo, la respuesta visible de la aplicación, las cabeceras HTTP y los logs de los contenedores. No se ha considerado exitoso ningún escenario basándose únicamente en mensajes generados por la propia interfaz.

4.4.2. Resultados

La Tabla 4.2 recoge el resultado observado de cada escenario.

Tabla 4.2: Resultados de los escenarios reproducidos en Fase 0

ID	Escenario	Resultado observado	Conclusión
E0-01	Reproducción oficial autorizada	Widevine y ClearKey reproducen después de validar identidad y activo.	Control válido
E0-02	Usuario sin <i>entitlement</i>	Login correcto, catálogo denegado y botones deshabilitados.	Control válido
E0-03	Reproductor Widevine externo	Reproducción iniciada con MPD y <code>no_auth</code> , sin login.	Vulnerable
E0-04	Acceso directo al origen	El MPD local responde por 8081 sin pasar por la CDN.	Vulnerable
E0-05	Contenido local por CDN	MPD y segmentos accesibles sin token; la caché devuelve HIT.	Vulnerable
E0-06	Reproducción con clave conocida	El activo CENC reproduce sin ninguna solicitud de licencia.	Impacto confirmado
E0-07	Exportación de claves Widevine	El navegador no expone claves del CDM mediante EME.	No reproducido

El resultado E0-03 es el más relevante. El vídeo externo se ha reproducido en su totalidad. El laboratorio ha reproducido el mismo comportamiento y ha registrado dos respuestas de licencia. Por tanto, no se ha tratado de una simulación visual, sino de una reproducción Widevine real fuera del cliente oficial.

E0-01 y E0-02 demuestran que la existencia de la vulnerabilidad no implica que toda la lógica de la plataforma sea inexistente. La autorización por catálogo funciona en el camino previsto. El problema es que puede eludirse seleccionando una ruta alternativa equivalente. Este resultado refuerza la necesidad de aplicar la misma política en todos los caminos capaces de entregar contenido o licencias.

4.4.3. Evidencias

Las principales evidencias obtenidas han sido las siguientes:

- Estado Reproduciendo:`SintelWidevinepublico` en el cliente autorizado

y avance real del elemento de vídeo.

- Dos botones deshabilitados y activos marcados como denegados para el usuario sin permisos.
- Evento `LICENSE_REQUEST` dirigido a `/license/no_auth` desde un cliente sin sesión.
- Respuestas binarias de licencia de 714 y 1229 bytes, seguidas de `PLAYBACK_STARTED`.
- Elemento de vídeo externo sin pausar, con `readyState` completo y tiempo de reproducción creciente.
- Respuestas HTTP 200 para el MPD local tanto mediante 8080 como mediante 8081.
- Cabecera `X-Cache:HIT` en la entrega del MPD a través de Varnish.
- Solicitudes registradas por Nginx para `video_init.mp4` y segmentos `video_N.m4s` durante la prueba ClearKey.
- Evento `PLAYBACK_STARTED_WITH_KNOWN_KEY` sin endpoint de licencias configurado.
- Evento `KEY_EXTRACTION_BLOCKED`, coherente con la imposibilidad de exportar claves Widevine mediante EME.

La Figura 4.1 documenta el comportamiento de referencia del sistema. El usuario autorizado aparece identificado en el cliente oficial, los dos activos figuran como permitidos y el reproductor muestra el contenido Widevine en curso. Esta prueba establece la línea base frente a la que se compara el acceso no autorizado.

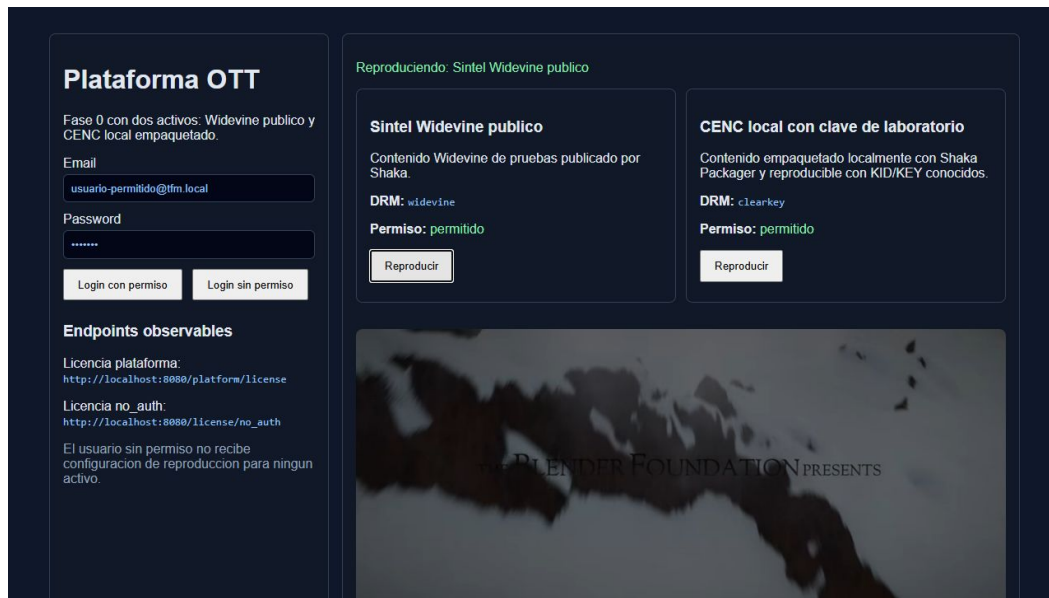


Figura 4.1: Reproducción del activo Widevine desde el cliente oficial con un usuario autorizado.

La evidencia central de la auditoría se muestra en la Figura 4.2. El laboratorio externo utiliza directamente el endpoint `/license/no_auth`, obtiene dos respuestas binarias de licencia y alcanza el evento `PLAYBACK_STARTED` sin que exista una sesión de usuario. La imagen conserva simultáneamente la reproducción y el registro de eventos, por lo que permite relacionar el resultado visible con la ruta empleada.

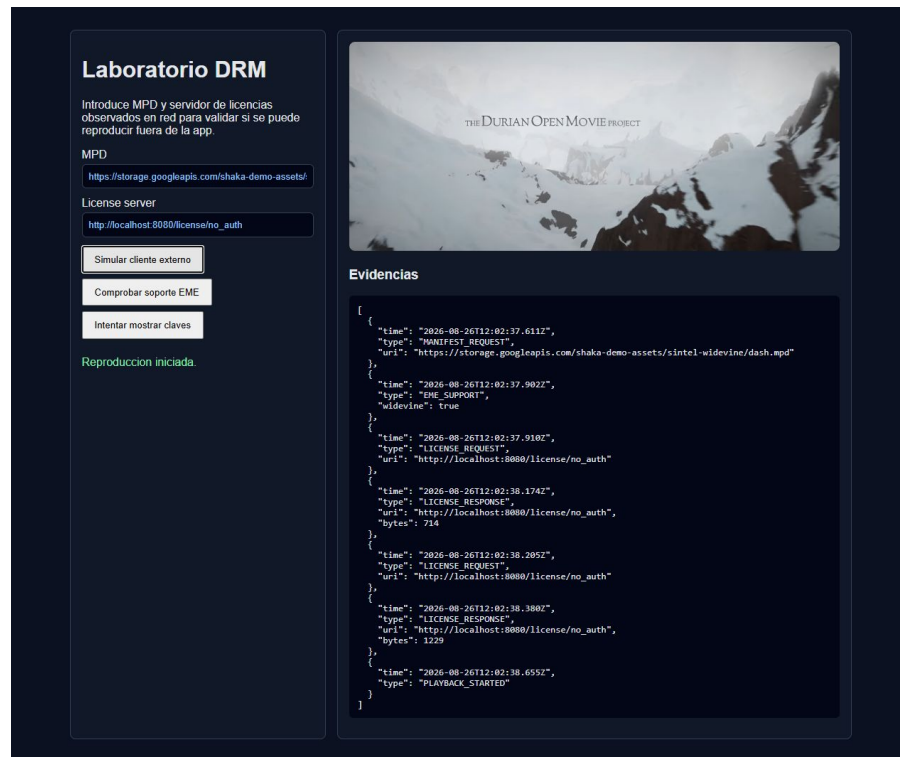


Figura 4.2: Reproducción Widevine desde el laboratorio externo mediante el endpoint de licencia sin autenticación.

Por último, la Figura 4.3 recoge la prueba ClearKey. El reproductor utiliza el MPD, el KID y una clave generada exclusivamente para el laboratorio, e inicia el vídeo sin configurar un servidor de licencias. Esta demostración no representa una extracción de claves Widevine; ilustra el impacto que tendría una clave ya conocida fuera del perímetro del DRM.

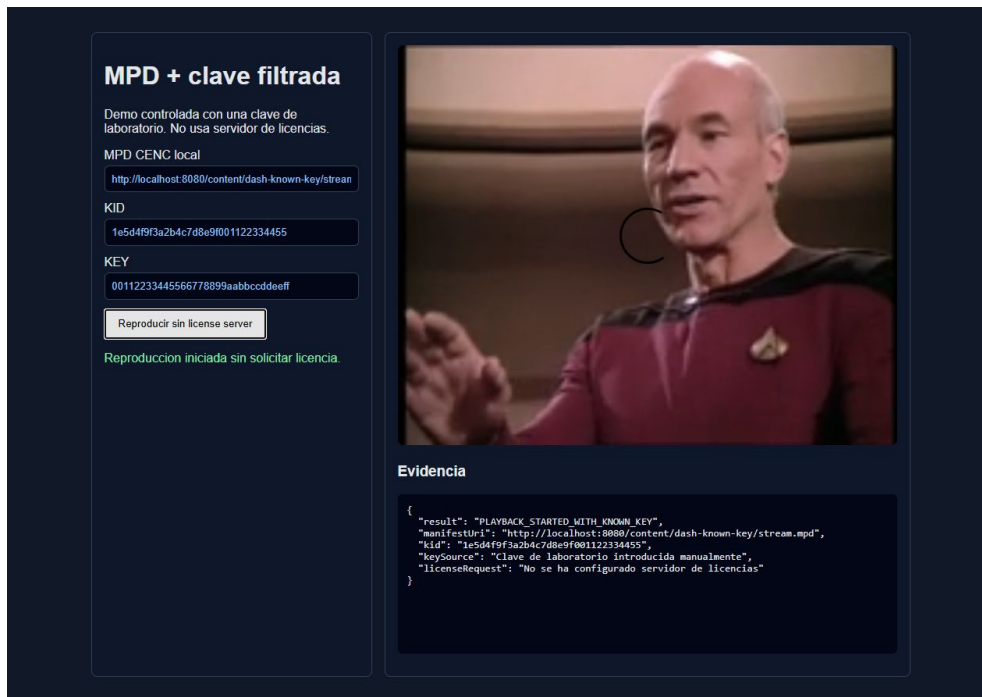


Figura 4.3: Reproducción del activo CENC local con una clave ClearKey conocida y sin servidor de licencias.

Para asegurar la trazabilidad, las pruebas principales se han conservado en el repositorio de GitHub de este TFM, y el Anexo A incluye una explicación detallada de todas las evidencias de esta Fase 0.

4.5. Limitaciones del modelo DRM tradicional

Los resultados confirman que Widevine cumple el cometido evaluado dentro del CDM: el navegador identifica el sistema de claves, genera un *challenge*, procesa una licencia binaria y reproduce el contenido cifrado sin exponer la clave a JavaScript. La auditoría no ha evidenciado una ruptura criptográfica de Widevine ni un método de extracción de claves. La debilidad se encuentra en la política que decide quién puede alcanzar el servicio de licencia y bajo qué contexto.

La ruta `/license/no_auth` demuestra que una licencia técnicamente válida puede ser emitida dentro de una reproducción ilegítima para la plataforma. Del mismo modo, el acceso directo al origen muestra que el cifrado no evita el consumo de ancho de banda ni el uso de la infraestructura por terceros. Finalmente, la ausencia de sesiones y concurrencia permite que credenciales válidas se utilicen

de una forma incompatible con la política de negocio sin que el DRM perciba ninguna anomalía.

La demostración ClearKey establece otro límite: ningún servidor de licencias puede revocar una clave que ya se encuentra fuera de su perímetro de control. En un sistema real, la robustez del CDM, la rotación de claves, las políticas de licencia y mecanismos complementarios como *watermarking* ayudan a reducir el impacto, pero no convierten al cliente en una frontera absoluta. La literatura sobre Widevine L3 confirma además que el nivel de seguridad del dispositivo condiciona fuertemente la resistencia del extremo cliente [9, 24].

El laboratorio presenta limitaciones que deben conservarse al generalizar los resultados. Utiliza vídeo bajo demanda en lugar de ingesta en directo, un único navegador, usuarios de demostración, HTTP local y un servicio Widevine público. No reproduce carga masiva, diversidad geográfica, múltiples CDN ni políticas comerciales de licencia. Tampoco demuestra leeching completo de segmentos Widevine a través de la CDN local, puesto que el activo Widevine se aloja externamente. Estas limitaciones reducen la validez externa del experimento, pero no alteran el hallazgo central: si existe un camino de licencia sin autorización, la protección criptográfica puede seguir funcionando mientras el control de acceso de negocio queda anulado.

La auditoría permite derivar cinco requisitos para la arquitectura defensiva: eliminar rutas de licencia sin autenticación; impedir el acceso directo a origen y servidor de licencias; separar identidad y sesión de reproducción; emitir credenciales efímeras ligadas a usuario, activo y dispositivo; y registrar señales que permitan limitar concurrencia, detectar abuso y revocar acceso. El capítulo siguiente desarrolla estos requisitos como una arquitectura de defensa en profundidad, mientras que el capítulo de implementación describe su materialización progresiva en Fase 1 y Fase 2.

5

Arquitectura de defensa en profundidad propuesta

La auditoría de Fase 0 ha evidenciado que el problema principal no se encuentra en el cifrado de Widevine, sino en los caminos que permiten alcanzar el contenido y el servicio de licencia sin conservar el contexto de autorización. A partir de este resultado se propone una arquitectura incremental: Fase 1 introduce controles preventivos alrededor del DRM y Fase 2 añade persistencia, detección y respuesta. La finalidad no es reemplazar Widevine, sino impedir que una licencia técnicamente válida se obtenga fuera de una reproducción autorizada y aportar capacidad de reacción cuando el uso deja de ser coherente con la política del servicio.

Este capítulo presenta las decisiones de diseño de ambas fases. La implementación concreta se describe en el capítulo siguiente y la comparación experimental con Fase 0 se ha realizado en el Capítulo 7. Esta separación evita convertir la descripción arquitectónica en una enumeración de pruebas y permite justificar cada control por la debilidad que pretende corregir.

5.1. Objetivos de la arquitectura defensiva

La propuesta parte de cinco requisitos derivados directamente de los hallazgos F0-01 a F0-05. En primer lugar, todo camino capaz de entregar el manifiesto, el contenido local o una licencia debe exigir autorización; eliminar la ruta `/license/no_auth` no sería suficiente si permaneciera otro acceso equivalente. En segundo lugar, el origen y el servidor de licencias no deben estar publicados directamente en el anfitrión. En tercer lugar, la identidad autenticada debe transformarse en una sesión de reproducción específica y efímera. En cuarto lugar, las peticiones deben vincularse a cuenta, dispositivo, instancia de cliente, activo y sesión, de modo que una credencial válida para un contexto no pueda trasladarse sin más a otro. Finalmente, el sistema debe conservar señales suficientes para detectar patrones anómalos y revocar el acceso.

Estos requisitos se traducen en tres propiedades verificables. La primera es *consistencia*: manifiesto, contenido, heartbeat y licencia deben referirse a la misma sesión. La segunda es *temporalidad*: el permiso de reproducción debe caducar si el cliente deja de mantener la sesión. La tercera es *capacidad de respuesta*: una cuenta o dispositivo bloqueado no debe poder continuar utilizando una credencial emitida con anterioridad.

La arquitectura se mantiene deliberadamente acotada. No pretende construir un proveedor de identidad, una CDN comercial o una plataforma antifraude basada en aprendizaje automático. Su objetivo es materializar controles observables y reproducibles con los que comparar las tres fases del laboratorio.

5.2. Modelo *Defense in Depth* aplicado a OTT

La defensa en profundidad se aplica como una composición de capas con responsabilidades distintas. El cliente inicia el flujo, pero no decide por sí mismo si el acceso es válido. Varnish reduce la superficie HTTP y normaliza la información de entrada. El *control-plane* aplica identidad, *entitlement*, sesión y contexto. El estado de Fase 2 permite relacionar peticiones separadas en el tiempo. Finalmente, el origen y el servidor de licencias permanecen internos y reciben únicamente solicitudes ya autorizadas.

La Figura 5.1 resume esta distribución. Fase 1 concentra la prevención: elimina el bypass, protege manifiesto, contenido local y licencia, y limita la concurrencia. Fase 2 incorpora memoria y respuesta: registra eventos, mantiene contadores, calcula riesgo y aplica bloqueos. Widevine continúa ocupándose del procesamiento de la licencia y de la protección de las claves dentro del CDM; las capas anteriores

determinan si el flujo que llega hasta él es legítimo.

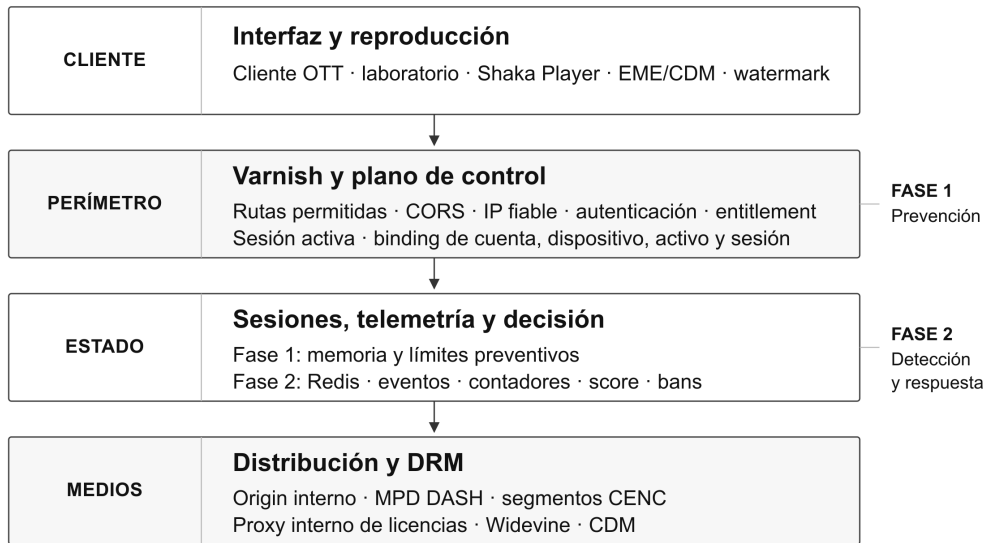


Figura 5.1: Capas de la arquitectura defensiva y responsabilidades de Fase 1 y Fase 2.

La Tabla 5.1 relaciona los problemas principales de Fase 0 con los controles propuestos. Esta trazabilidad es importante porque evita atribuir a una única medida una protección que depende de varias capas.

Tabla 5.1: Trazabilidad entre hallazgos y controles defensivos

Hallazgo	Control principal	Fase
Licencia pública	Único endpoint público de licencia, protegido por una sesión de reproducción.	Fase 1
Origen y contenido expuestos	Servicios internos y autorización sobre manifiesto, segmentos y licencia.	Fase 1
Sesión sin caducidad ni contexto	Credencial efímera ligada a cuenta, dispositivo, activo e instancia.	Fase 1
Sin control de concurrencia	Límite preventivo y registro de rechazos reiterados.	Fases 1 y 2
CORS y HTTP per-misivos	Orígenes web restringidos; TLS reservado al despliegue de producción.	Fase 1 / producción

5.3. Diseño del acceso protegido a manifiestos y licencias

La Fase 0 permitía obtener el MPD y alcanzar el proxy Widevine mediante caminos que no compartían una política común. La propuesta unifica ambos recursos sensibles detrás del *control-plane*. El cliente ya no recibe directamente el manifiesto público como configuración de reproducción, sino una URL local de la forma `/manifest/:assetId`. De igual modo, el único endpoint de licencia expuesto es `/license`; la ruta `no_auth` desaparece del perímetro.

Esta decisión no convierte el MPD en una credencial secreta. Su función es asegurar que la plataforma compruebe el contexto antes de entregar la descripción de la presentación y de iniciar el flujo EME. En el activo Widevine público, los segmentos continúan alojados en la infraestructura de demostración de Shaka. Para los activos locales, la ruta `/content/*` aplica además una relación entre el activo del token y el prefijo permitido del origen.

5.3.1. Interceptación

Varnish constituye el único punto de entrada hacia el plano de control. Rechaza rutas no contempladas por la fase, exige la cabecera **Authorization** en manifiesto, contenido, licencia, heartbeat y parada de sesión, y genera un identificador de petición. Además, sustituye cualquier **X-Forwarded-For** aportado por el cliente por la dirección observada en la conexión. Con ello se evita que una señal de red utilizada para correlación pueda ser elegida libremente desde el navegador.

La interceptación también se produce en Shaka Player. El cliente añade el token de reproducción y las cabeceras de sesión, dispositivo e instancia a las solicitudes protegidas de manifiesto, contenido local y licencia. Se mantiene así una política común sobre recursos que Fase 0 trataba de manera independiente.

5.3.2. Validación

El *control-plane* valida primero la firma y expiración de la credencial. Después comprueba que su tipo sea de reproducción y recupera la sesión indicada. La sesión debe existir, permanecer activa y coincidir con la cuenta, dispositivo, instancia de cliente y activo incluidos en el token. Finalmente, los identificadores declarados en las cabeceras deben concordar con el token y el activo solicitado debe coincidir con la ruta.

La Figura 5.2 representa el flujo completo. La autenticación genera una credencial de acceso; esta permite crear una sesión y obtener una credencial de reproducción de corta duración. Solo la segunda se utiliza para solicitar el MPD y la licencia. El heartbeat renueva el permiso mientras la sesión permanece activa.

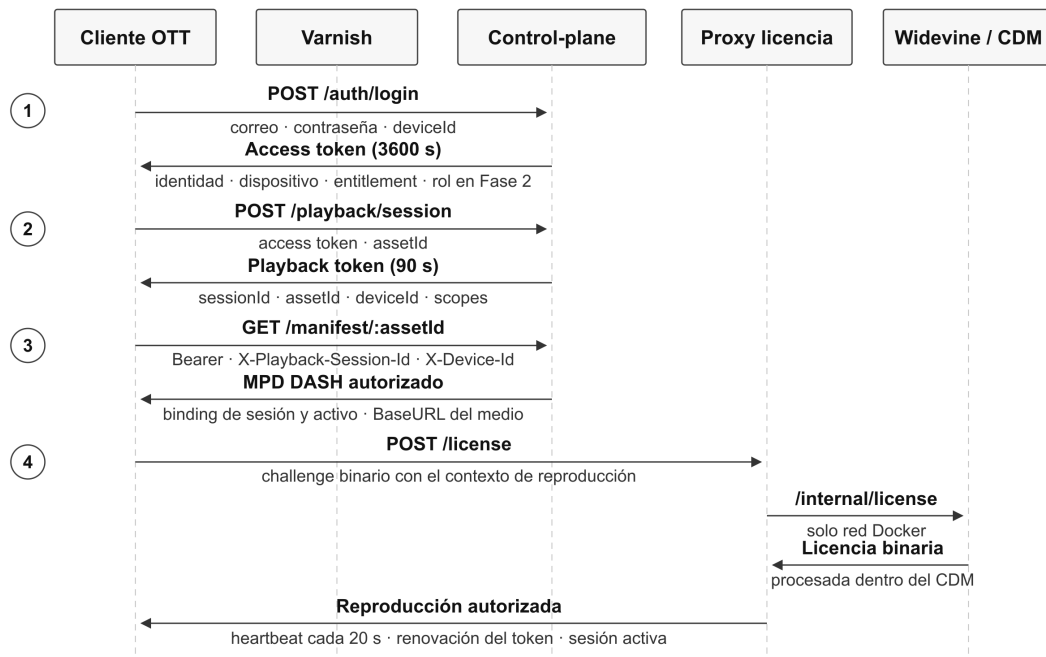


Figura 5.2: Secuencia de autenticación, sesión, manifiesto y licencia en las fases endurecidas.

La validación se repite en cada punto sensible; no se asume que una decisión adoptada al crear la sesión siga siendo válida indefinidamente. Esta propiedad permite que una sesión detenida, expirada o bloqueada deje de servir aunque el cliente conserve una copia del token.

5.3.3. Política de acceso

La política distingue autenticación, autorización y reproducción. Un usuario puede presentar credenciales correctas y, aun así, carecer del *entitlement* del activo. En ese caso obtiene identidad, pero la creación de la sesión se rechaza. Del mismo modo, disponer de un token de acceso no permite solicitar directamente manifiesto o licencia porque ambos endpoints exigen una credencial de tipo reproducción.

Las decisiones principales producen respuestas diferenciadas: ausencia o alteración de token implica rechazo de autenticación; un activo distinto al autorizado produce un error de autorización; una cabecera de sesión incoherente se trata como conflicto de contexto; y una segunda reproducción simultánea supera el límite de concurrencia. La codificación exacta de estas respuestas se describe en

el capítulo de implementación y su comportamiento se ha evaluado en el Capítulo 7.

5.3.4. CDN leeching y clave conocida

La protección del servidor de licencias no basta cuando una clave ya ha salido del entorno previsto. En ese escenario, un tercero puede intentar obtener el MPD y los segmentos cifrados para reproducirlos en otro cliente o utilizarlos como fuente de un servicio de *restreaming*. Por este motivo, el activo CENC local se publica en las fases endurecidas mediante el mismo plano autorizado: tanto el manifiesto como cada objeto de contenido exigen token, sesión, activo e instancia coherentes.

CORS se conserva como una limitación adicional para navegadores con un origen no permitido, pero no se considera un mecanismo de autenticación. Un cliente nativo o un backend IPTV no está obligado a aplicar la política del navegador y, si dispone de una sesión válida, puede realizar las solicitudes directamente. El control efectivo reside, por tanto, en la validación de servidor y no en la cabecera `Origin`.

Existe además un caso límite deliberadamente aceptado. Una única sesión legítima que presente un token vigente y la clave conocida puede descargar y reproducir un canal sin solicitar una nueva licencia. Bloquearla de forma inmediata produciría falsos positivos ante licencias ya disponibles en el dispositivo. Fase 2 trata la combinación de manifiesto y tres objetos de contenido sin petición de licencia como una señal heurística: añade 20 puntos una sola vez por sesión, pero no genera por sí sola un ban. La concurrencia, el cruce de activo, la copia desde otra instancia y el uso posterior a la parada continúan siendo rechazados de forma independiente.

5.4. Sistema de autenticación y autorización

5.4.1. Credenciales de acceso y reproducción

La arquitectura utiliza dos clases de credencial. El token de acceso representa la autenticación de la cuenta e incluye dispositivo, plan, permisos y, en Fase 2, roles. El token de reproducción representa una única sesión e incorpora cuenta, dispositivo, activo, `sessionId`, `clientInstanceId` y ámbitos para manifiesto, contenido, licencia y heartbeat.

Ambas credenciales se firman y disponen de expiración. El formato se ha simplificado para el laboratorio porque el aspecto evaluado no es un estándar concreto de identidad, sino la separación de privilegios, la vinculación con el contexto de reproducción y la posibilidad de revocar el acceso.

5.4.2. Expiración y renovación

La credencial de acceso tiene una vida mayor que la de reproducción. El diseño permite mantener una sesión de usuario razonable sin conceder durante el mismo periodo el acceso al plano de medios. El token de reproducción caduca rápidamente y se renueva mediante heartbeat. Si la aplicación deja de enviar esa señal, la sesión pasa a estado expirado y la siguiente validación falla.

La renovación no amplía una sesión que haya sido detenida o bloqueada. Antes de emitir el nuevo token se vuelve a recuperar el estado y, en Fase 2, se consulta también si existen bans de cuenta o dispositivo. Por tanto, la expiración corta limita la ventana de reutilización y el estado del servidor conserva la capacidad de revocar antes de que se alcance dicha expiración.

5.4.3. Validación contextual

El contexto mínimo se compone de cuenta, dispositivo, instancia de cliente, activo y sesión. La IP se utiliza como señal de correlación, no como identidad principal: puede cambiar por movilidad, NAT o infraestructura intermedia y no sería prudente bloquear automáticamente una cuenta por una única variación. La arquitectura tampoco afirma realizar geolocalización. Fase 2 conserva las IP observadas durante una ventana temporal y eleva el riesgo cuando una misma cuenta acumula actividad desde varias direcciones.

El identificador de dispositivo y el de instancia se generan en el cliente de laboratorio y se incluyen en las credenciales. Ninguno constituye una atestación criptográfica de hardware; su utilidad es hacer reproducible el binding, diferenciar una ejecución concreta y permitir demostrar el bloqueo de un sujeto. Un sistema comercial requeriría señales más resistentes a manipulación, pero esa ampliación no es necesaria para comprobar el efecto de la política propuesta.

5.5. Controles de concurrencia y ciclo de vida

5.5.1. Límite de sesiones

Antes de crear una sesión, el *control-plane* cuenta las sesiones activas de la cuenta. El laboratorio fija el máximo en una reproducción simultánea. Si ya existe una, la segunda solicitud se rechaza sin emitir otro token de reproducción. El control se sitúa en la aplicación y no en Widevine, porque el DRM no conoce el número de reproducciones permitido por el plan contratado.

Fase 1 mantiene este estado en memoria. Fase 2 lo conserva en Redis, lo que permite que las comprobaciones de distintas peticiones consulten una fuente común y que la interfaz administrativa muestre sesiones recientes. En ambos casos, el estado incluye inicio, último heartbeat, cuenta, dispositivo, instancia de cliente, activo y situación de la sesión.

5.5.2. Correlación de IP y dispositivo

La dirección observada por Varnish se asocia a los eventos de autenticación y creación de sesión. Fase 2 mantiene durante diez minutos el conjunto de IP recientes por cuenta y lo utiliza como una señal de riesgo. El dispositivo y la instancia permanecen en el token y en la sesión, por lo que trasladar únicamente la credencial y modificar una cabecera no produce un contexto coherente.

Esta correlación no equivale a afirmar que cada IP identifica una persona ni que el `deviceId` sea imposible de copiar. La defensa surge de combinar señales y de repetir la validación, no de confiar de manera absoluta en un único atributo controlado parcialmente por el cliente.

5.5.3. Expiración y finalización

La sesión puede terminar de tres maneras. La aplicación puede detenerla explícitamente mediante `/playback/stop`; puede expirar al superar la gracia sin heartbeat; o puede quedar inutilizable al aparecer un ban aplicable. La separación entre estado de sesión y token permite que conservar una credencial no sea suficiente para restaurar ninguna de estas situaciones.

5.6. Detección de anomalías

5.6.1. Patrones sospechosos

Fase 2 observa señales que pueden relacionarse con abuso: actividad de una cuenta desde varias IP, rechazos repetidos de concurrencia, ráfagas de licencias, tasas elevadas de contenido, contenido consumido sin una licencia observada en la sesión, heartbeats con una cadencia superior a la del cliente y fallos de autenticación concentrados en un dispositivo. Ninguna de estas señales demuestra por sí sola fraude; su función es producir una decisión reproducible dentro del experimento.

Los eventos conservan el tipo de operación, identificadores relevantes y marca temporal. Para las solicitudes de manifiesto y licencia se registra además la sesión y el riesgo calculado. Las claves Widevine y los tokens completos quedan fuera del registro, reduciendo el riesgo de que la propia observabilidad se convierta en una fuente de credenciales.

5.6.2. Límites y contadores temporales

Fase 1 incorpora límites preventivos por IP y ventana para autenticación, creación de sesión, manifiesto, heartbeat, contenido y licencia. Fase 2 utiliza contadores persistidos con TTL como señales de comportamiento. Esta segunda estrategia no se limita a devolver un error por volumen: acumula contexto y puede terminar en un ban.

Los umbrales se han elegido para facilitar escenarios de laboratorio y no proceden de una calibración con tráfico real. Por ello, la arquitectura separa su significado de la cifra concreta. El diseño relevante es la existencia de ventanas, contadores y una acción posterior; la adecuación estadística de cada umbral pertenece a una evaluación con una población de usuarios que queda fuera del alcance del prototipo.

5.6.3. Correlación básica

La correlación se realiza por cuenta, dispositivo, sesión e IP observada. Cada señal puede aumentar una puntuación y añadir una razón legible. Cuando la puntuación de cuenta alcanza el umbral configurado se crea un ban automático. Los fallos de autenticación siguen un camino más directo: al alcanzar la ráfaga

definida se bloquea temporalmente el dispositivo, incluso aunque no exista una sesión autenticada desde la que calcular riesgo de cuenta.

La Figura 5.3 resume el recorrido desde la señal hasta la respuesta. Redis mantiene el estado intermedio y el plano administrativo permanece separado del plano de reproducción, de modo que un operador pueda inspeccionar o retirar un bloqueo sin depender de que el dispositivo bloqueado siga autorizado.

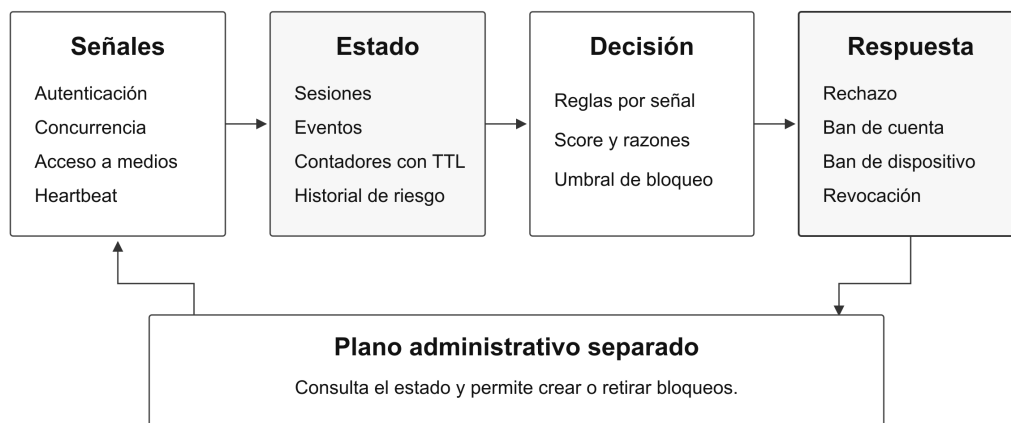


Figura 5.3: Flujo de eventos, riesgo y respuesta activa en Fase 2.

5.7. Bloqueo y mitigación

5.7.1. Bans de cuenta y dispositivo

La propuesta evita el baneo automático por IP como mecanismo principal. Una dirección puede representar varios usuarios o cambiar sin que exista abuso. Los sujetos bloqueables son la cuenta y el dispositivo: la primera agrupa el comportamiento autenticado y el segundo permite responder a una ráfaga de fallos previa al login. Las IP permanecen como contexto de detección.

Los bans son temporales y contienen motivo, creador, instante de creación y expiración. Pueden generarse automáticamente o desde el plano administrativo. La validación de bans se ejecuta tanto con tokens de acceso como con tokens de reproducción, por lo que afecta al login posterior y a las operaciones de una sesión ya creada.

5.7.2. Revocación y recuperación

El sistema diferencia el plano de reproducción del plano administrativo. Los endpoints de observabilidad y gestión requieren un token de acceso con rol `admin`, pero no dependen de que el dispositivo mantenga permiso para reproducir. Así se evita que aplicar un ban deje al operador sin capacidad para observarlo o retirarlo.

La revocación no borra la evidencia: la creación y la retirada producen eventos separados. Una vez eliminado el ban, el cliente debe volver a superar la validación y renovar el token de reproducción antes de continuar. Esta recuperación explícita permite demostrar que el bloqueo no consiste únicamente en deshabilitar un botón de la interfaz.

5.7.3. Invalidación de sesión

Detener una sesión cambia su estado y provoca que cualquier token asociado sea rechazado. La expiración por ausencia de heartbeat produce el mismo efecto. Ante un ban, la sesión puede permanecer registrada para trazabilidad, pero deja de ser utilizable porque la comprobación del sujeto bloqueado precede a la entrega de manifiesto, contenido o licencia.

5.8. Arquitectura final integrada

La arquitectura final combina prevención, detección y respuesta sin trasladar al DRM responsabilidades que pertenecen a la plataforma. Fase 1 elimina caminos alternativos, internaliza origen y licencia, separa identidad de reproducción y exige un contexto coherente. Fase 2 añade persistencia efímera, eventos estructurados, reglas de riesgo, bans y una interfaz administrativa.

El resultado conserva una propiedad esencial para el TFM: las tres fases siguen siendo comparables. Fase 0 permite observar el flujo vulnerable; Fase 1 muestra el efecto de aplicar autorización extremo a extremo; y Fase 2 permite estudiar qué aporta la memoria del comportamiento una vez corregido el bypass básico. El capítulo siguiente describe cómo se ha materializado esta arquitectura en Docker, Node.js, Varnish, Redis, Shaka Player y los laboratorios de auditoría.

6

Implementación

Este capítulo describe la materialización de la arquitectura propuesta en Fase 1 y Fase 2. Ambas reutilizan el activo Widevine y los servicios de medios del entorno experimental, pero modifican de forma sustancial el camino de acceso: Varnish dirige las operaciones al *control-plane*, el origen y el servidor de licencias dejan de publicarse en el anfitrión y el cliente necesita una sesión activa para cargar el manifiesto, recuperar contenido local o solicitar una licencia.

La exposición se centra en decisiones comprobables en el código y en la configuración desplegada. Los valores indicados corresponden al prototipo ejecutado; no se presentan como parámetros universales para un servicio OTT. Las pruebas comparativas, los códigos observados y el impacto de los controles se han desarrollado en el Capítulo 7.

6.1. Despliegue con Docker

6.1.1. Contenedores

Fase 1 se despliega mediante siete contenedores: Varnish, *control-plane*, origen Nginx, servidor interno de licencias, cliente OTT, laboratorio de auditoría y laboratorio de CDN leeching. Fase 2 conserva esos componentes y añade Re-

dis, alcanzando ocho servicios. El cliente y el laboratorio general son aplicaciones estáticas servidas por Nginx; el plano de control, el proxy de licencia y el laboratorio de CDN leeching se implementan con Node.js y Express.

Tabla 6.1: Contenedores de las fases endurecidas

Servicio	Función implementada	Fases
cdn	Frontal Varnish, lista de rutas, CORS, normalización de IP y derivación al plano de control.	1 y 2
control-plane	Login, autorización, sesiones, tokens, manifiesto, contenido, licencia y, en Fase 2, riesgo y administración.	1 y 2
origin	Publicación interna y de solo lectura del árbol <code>origin-content</code> .	1 y 2
license-server	Proxy interno del challenge hacia el servicio Widevine de pruebas.	1 y 2
ott-client	Cliente oficial con Shaka Player, watermark y controles de sesión.	1 y 2
audit-lab	Reproducción controlada, inspección de atributos y escenarios reproducibles.	1 y 2
key-leak-lab	Cliente externo controlado para Key Leak, CORS, concurrencia y reutilización contextual.	1 y 2
redis	Sesiones, eventos, riesgo, contadores y bans sin persistencia en disco.	2

Las imágenes de Nginx y Redis proceden de versiones ligeras basadas en Alpine. Varnish utiliza la imagen 7.5 y reserva 256 MB para su almacenamiento en memoria. Los servicios Node.js se construyen desde los directorios correspondientes, lo que permite modificar su lógica sin mezclarla con la imagen de distribución.

6.1.2. Redes

Cada fase define una red Docker independiente: `tfm-fase1-net` y `tfm-fase2-net`. Los nombres de servicio actúan como resolución interna, de modo que el *control-plane* alcanza `http://origin` y `http://license-server:8080` sin

publicar esos puertos. En Fase 2, Redis solo escucha dentro de la misma red como `redis:6379`.

La ausencia de puertos del anfitrión para origen y licencia corrige la ruta directa observada en Fase 0. No se trata únicamente de ocultar direcciones: Varnish remite todas las operaciones públicas al *control-plane*, que vuelve a aplicar la política antes de acceder a los servicios internos. Los activos compartidos se montan en Nginx como volumen de solo lectura.

6.1.3. Servicios y puertos

La Tabla 6.2 recoge las interfaces publicadas. Los puertos de cliente y laboratorio se separan para que las acciones de auditoría no tengan que incorporarse a la aplicación oficial.

Tabla 6.2: Interfaces publicadas en el anfitrión

Fase	Interfaz	Dirección publicada
Fase 1	Cliente oficial	<code>localhost:9300</code>
	Laboratorio de auditoría	<code>localhost:9301</code>
	Laboratorio de CDN <i>leeching</i>	<code>localhost:9302</code>
	CDN / API	<code>localhost:9080</code>
Fase 2	Cliente oficial	<code>localhost:9400</code>
	Laboratorio de auditoría	<code>localhost:9401</code>
	Laboratorio de CDN <i>leeching</i>	<code>localhost:9402</code>
	CDN / API	<code>localhost:9180</code>

El *control-plane*, el origen, el servidor de licencias y Redis no se publican directamente. Los secretos HMAC y las credenciales de los dos usuarios se suministran como valores de demostración. Al igual que el uso de HTTP local, esta configuración facilita la reproducción del laboratorio, pero debería sustituirse por TLS, gestión de secretos e identidad externa en un despliegue real.

6.2. Desarrollo del plano de control y del proxy protegido

6.2.1. Arquitectura software

El *control-plane* es una aplicación Express que combina tres responsabilidades acotadas. La primera es emitir y verificar credenciales. La segunda es mantener el ciclo de vida de las sesiones. La tercera es actuar como proxy autorizado hacia el manifiesto, el origen local y el servidor interno de licencias. Fase 1 conserva sesiones y límites en estructuras **Map** del proceso; Fase 2 reemplaza el estado operativo por claves y estructuras de Redis.

El manifiesto Widevine se obtiene desde la URL pública de Sintel únicamente después de validar la reproducción. El servicio inserta un elemento **BaseURL** con el directorio externo de los medios y devuelve el MPD como **application/dash+xml**. Esta reescritura permite que Shaka resuelva correctamente los segmentos aunque el MPD se haya solicitado a la URL local protegida. En consecuencia, el control implementado gobierna la entrega del manifiesto y el inicio del flujo; los segmentos del activo público continúan alojados fuera del laboratorio.

Para el contenido local, el manifiesto CENC se obtiene del origen, se normaliza como una presentación estática de 60 s y se reescribe para que cada **BaseURL** apunte a **/content/dash-known-key/**. Esta ruta elimina el prefijo público, contrasta la ruta con el activo del token y solo entonces reenvía la solicitud a Nginx. La respuesta conserva cabeceras como **Content-Type**, **Content-Length**, rangos y ETag. El mapeo explícito impide usar un token de un activo para recorrer otro prefijo del origen y obliga a autorizar también las inicializaciones y los segmentos.

La licencia sigue una cadena diferente. Express recibe el cuerpo binario con un límite de 2 MB, valida la sesión y envía el challenge a **/internal/license**. La llamada interna añade cuenta, dispositivo, activo y sesión como cabeceras. El servidor de licencias exige la presencia de ese contexto antes de reenviar el cuerpo al servicio CWIP de pruebas y marca la respuesta como no cacheable.

6.2.2. *Endpoints*

La interfaz HTTP se mantiene prácticamente igual entre las fases. Fase 2 añade los endpoints administrativos y eventos asociados. La Tabla 6.3 resume el contrato implementado.

Tabla 6.3: Endpoints principales de Fase 1 y Fase 2

Método	Ruta	Credencial y función	Fases
GET	/health	Estado del servicio y sesiones conocidas.	1 y 2
POST	/auth/login	Credenciales de usuario; emite el token de acceso.	1 y 2
POST	/playback/session	Token de acceso; valida entitlement y concurrencia.	1 y 2
GET	/manifest/:assetId	Token de reproducción; entrega el MPD asociado.	1 y 2
GET/HEAD	/content/*	Token de reproducción; proxy al prefijo local autorizado.	1 y 2
POST	/license	Token de reproducción; proxy del challenge Widevine.	1 y 2
POST	/playback/heartbeat	Token de reproducción; mantiene sesión y rota token.	1 y 2
POST	/playback/stop	Token de reproducción; detiene la sesión.	1 y 2
GET	/admin/overview	Token de acceso con rol admin; resumen operativo.	2
GET	/admin/events	Token de acceso con rol admin; eventos recientes.	2
POST	/admin/bans	Crea un ban de cuenta o dispositivo.	2
POST	/admin/bans/clear	Retira un ban existente.	2

La ruta heredada `/license/no_auth` no aparece en la lista permitida de Varnish ni existe en Express. De esta forma no hay un segundo camino público que evite las comprobaciones del endpoint protegido.

6.2.3. *Middleware* y perímetro

Varnish responde a las preflight CORS y solo admite los orígenes de cliente y laboratorio de su fase. Las cabeceras autorizadas se limitan a autorización, tipo de contenido, rangos, sesión, dispositivo e instancia de cliente. En Fase 2 se expone además **X-Risk-Score**. Todas las respuestas del frontal se marcan como **no-store**, puesto que el prototipo prioriza la consistencia de las decisiones sobre una evaluación de caché. CORS limita un reproductor ejecutado desde un origen web ajeno, pero no autentica un cliente nativo o un backend; estos siguen dependiendo de la validación del token.

Antes de enviar la petición al backend, Varnish comprueba que la ruta pertenezca al conjunto esperado y exige **Authorization** en las operaciones protegidas. Sobrescribe **X-Forwarded-For** con la IP de la conexión, añade **X-Request-Id** y fuerza el paso directo al backend. Express aplica a continuación la comprobación criptográfica y contextual; por tanto, la presencia de la cabecera en Varnish no se considera prueba suficiente de autorización.

6.3. Implementación de validaciones

6.3.1. Credenciales de acceso y reproducción

Las credenciales se serializan y firman localmente. Antes de interpretar sus atributos, el servidor comprueba su integridad y rechaza cualquier valor alterado.

La credencial de acceso identifica la cuenta, el dispositivo y sus permisos; la de reproducción incorpora el activo, la sesión, la instancia de cliente y las operaciones permitidas. Ambas utilizan secretos distintos para impedir que una credencial de acceso pueda aceptarse como autorización de reproducción.

La Tabla 6.4 muestra los parámetros comunes configurados en Docker Compose.

Tabla 6.4: Parámetros de credenciales y sesiones

Parámetro	Valor	Finalidad
Expiración del token de acceso	3600 s	Mantener la identidad de la sesión de usuario.
Expiración del token de reproducción	90 s	Reducir la ventana de reutilización.
Gracia de heartbeat	45 s	Expirar sesiones que dejan de mantenerse.
Periodo del heartbeat del cliente	20 s	Renovar antes de la expiración o la gracia.
Máximo de sesiones activas	1 por cuenta	Reproducir el control de concurrencia.
Retención de eventos en Fase 2	300 eventos	Acotar el estado de observabilidad.
Umbral de auto-ban de cuenta	100 puntos	Activar respuesta por riesgo acumulado.

6.3.2. Validación de IP, dispositivo, instancia, activo y sesión

La implementación no realiza geolocalización. La IP fiable proporcionada por Varnish se utiliza para rate limiting en Fase 1 y para correlación temporal en Fase 2. El `deviceId` se genera en el navegador y se incluye en el login. Al crear la reproducción se genera también un `clientInstanceId`, que queda ligado al token y a la sesión. Cada validación compara cuenta, dispositivo, instancia y activo del token con el objeto de sesión almacenado.

El manifiesto contrasta también el `assetId` de la ruta, mientras que manifiesto, contenido y licencia comparan las cabeceras `X-Playback-Session-Id` y `X-Client-Instance-Id`. Un activo diferente se rechaza como falta de autorización, una sesión diferente como conflicto y una instancia copiada como credencial no válida. Esta distinción facilita el análisis posterior sin revelar el token utilizado.

Fase 2 ejecuta además la comprobación de bans en las operaciones que emplean credenciales de acceso o reproducción. El plano administrativo usa una validación separada de rol. Esta separación es intencional: un operador debe poder consultar y retirar un ban aunque su dispositivo no pueda utilizar el plano de reproducción.

6.3.3. Control de sesiones, concurrencia y expiración

La creación de sesión elimina primero las sesiones que han superado la gracia de heartbeat. Después comprueba el entitlement, exige la instancia de cliente y cuenta las sesiones activas de la cuenta. Si no se supera el máximo, genera un UUID, almacena el contexto y emite el token de reproducción.

Fase 1 guarda los objetos en memoria. Fase 2 emplea claves `session:<uuid>` y un conjunto `account-sessions:<accountId>`. Las sesiones tienen un TTL operativo de 900 s y el índice por cuenta, de 24 horas. El proceso revisa cada diez segundos las sesiones conocidas y marca como `expired` las que superan los 45 s sin heartbeat. La parada explícita utiliza el estado `stopped`.

El cliente llama al heartbeat cada 20 s. La respuesta contiene un nuevo token de reproducción de 90 s, por lo que la rotación se realiza antes de la expiración. La renovación no cambia el identificador de sesión ni el activo y vuelve a comprobar el estado almacenado.

Fase 1 incorpora además límites en memoria por IP y minuto: 20 intentos de autenticación, 60 creaciones de sesión, 120 manifiestos, 180 heartbeats, 600 solicitudes de contenido y 120 licencias. Fase 2 no replica este limitador duro como módulo independiente; utiliza contadores persistentes para elevar riesgo y terminar en bloqueo. Esta diferencia se conserva expresamente en la descripción para no afirmar que ambas fases aplican exactamente la misma semántica de rate limiting.

6.4. Sistema de *logging* y monitorización

Fase 2 registra cada evento como JSON en una lista Redis limitada a 300 elementos. El productor añade UUID, tipo y marca temporal a los campos específicos de la operación. Entre los tipos implementados se encuentran `auth.success`, `auth.failure`, `auth.blocked`, `playback.session_created`, `playback.concurrency_rejected`, `playback.client_instance_rejected`, `manifest.request`, `content.request`, `license.request`, `key_leak.pattern_detected`, `playback.heartbeat`, `playback.stopped`, `playback.expired`, `risk.incremented`, `ban.created` y `ban.cleared`.

La puntuación de riesgo se almacena durante 24 horas junto con un máximo de ocho razones distintas. Un login desde menos de tres IP recientes reduce cinco puntos, mientras que los patrones de la Tabla 6.5 incrementan el valor o crean un ban directo.

Tabla 6.5: Reglas implementadas en Fase 2

Señal	Ventana / umbral	Acción
Actividad multi-IP	3 IP en 10 min	+30 puntos de cuenta
Concurrencia rechazada	2 en 10 min	+25 puntos
Solicitudes de licencia	6 en 5 min	+20 puntos
Solicitudes de contenido	40 en 1 min	+15 puntos
Contenido sin licencia	3 objetos tras el manifest, una vez por sesión	+20 puntos
Heartbeats	8 en 1 min	+10 puntos
Fallos de autenticación	8 en 5 min	Ban de dispositivo de 15 min
Riesgo acumulado	≥ 100 puntos	Ban de cuenta de 30 min

Los contadores utilizan TTL y se incrementan de forma atómica en Redis. Los bans se guardan como `ban:account:<id>` o `ban:device:<id>` e incluyen razón, creador y expiración. La API administrativa permite crear bans con una duración indicada, listar el estado relevante para la cuenta autenticada y retirar un bloqueo.

El panel de Fase 2 representa los eventos mediante nodos de texto, no mediante HTML interpretado. Además, las interfaces muestran identificadores abreviados y atributos seleccionados, pero no imprimen `accessToken` ni `playbackToken` completos. Estas decisiones evitan que la propia instrumentación exponga credenciales reutilizables.

6.5. Clientes y laboratorios de auditoría

Los clientes oficiales mantienen separado el flujo de usuario de las acciones de auditoría. En ambos casos, el usuario autorizado inicia sesión, crea una reproducción y deja que Shaka Player añada el token y la instancia a las solicitudes de manifiesto, contenido local y licencia. Un watermark superpuesto utiliza correo enmascarado, prefijo de sesión y fase; aporta atribución visual, pero no se considera una medida criptográfica.

El laboratorio de Fase 1 automatiza login, creación de sesión y obtención del token a partir de usuario y contraseña. Muestra únicamente tipo, activo, expiración y scopes decodificados. También permite reproducir y ejecutar cuatro casos negativos: ausencia de token, token alterado, sesión distinta y activo distinto. La Figura 6.1 presenta en paralelo el cliente legítimo y el laboratorio, ambos utilizando las mismas rutas protegidas.

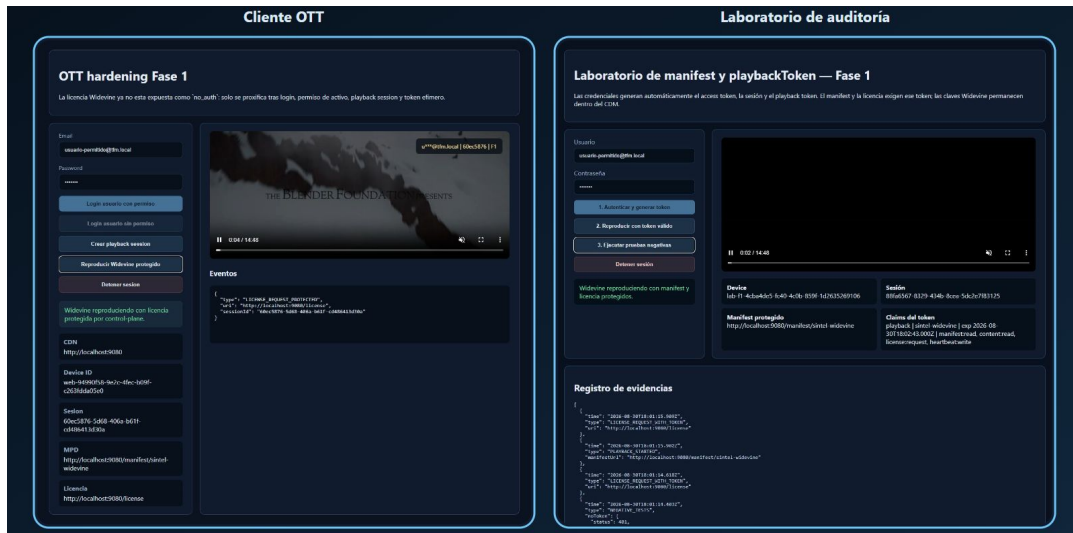


Figura 6.1: Cliente oficial y laboratorio de Fase 1 utilizando manifiesto y licencia protegidos.

El cliente de Fase 2 añade un panel de observabilidad que consulta `/admin/overview`. La respuesta integra sesiones, score, razones, bans y los veinte eventos más recientes. Los botones administrativos crean o retiran un ban del dispositivo visible. La Figura 6.2 muestra la reproducción legítima y la aparición de eventos de sesión, manifiesto y licencia; su análisis cuantitativo se presenta en el capítulo de evaluación.

OTT hardening Fase 2

Widevine protegido por control-plane, Redis, eventos, score de riesgo, baneos y observabilidad.

Email

usuario-permitido@tfm.local

Password

Login usuario con permiso

Login usuario sin permiso

Crear playback session

Reproducir Widevine protegido

Actualizar observabilidad

Banear este device 15 min

Quitar ban del device

Detener sesion

Widevine reproduciendo con licencia protegida y observabilidad.

u***@tfm.local | 87039446 | risk:0

THE DURIAN OPEN MOVIE PROJECT

0:11 / 14:48

Device	Sesion	Riesgo	Activo
web-066cfa5e-a631-469d-8d9d-51b97b807352	87039446-2883-4772-bfb4-c2c3d7a25423	0	1

Observabilidad

Resumen

Cuenta acc-allowed-001 | sesiones registradas: 1 | score: 0

Eventos recientes

license.request

2026-08-30T18:02:50.360Z

{ "sessionId": "87039446-2883-4772-bfb4-c2c3d7a25423", "accountId": "acc-allowed-001", "risk": { "accountId": "acc-allowed-001", "score": 0, "reasons": [] }, "lastUpdatedAt": "2026-08-30T18:02:48.952Z", "id": "65fa9a8a-35f0-48e9-99fc-e25ed343f0fd", "type": "license.request", "timestamp": "2026-08-30T18:02:50.360Z" }

license.request

2026-08-30T18:02:50.158Z

{ "sessionId": "87039446-2883-4772-bfb4-c2c3d7a25423", "accountId": "acc-allowed-001", "risk": { "accountId": "acc-allowed-001", "score": 0, "reasons": [] }, "lastUpdatedAt": "2026-08-30T18:02:48.952Z", "id": "22cbc3f0-27c3-4cd8-b66f-a4d28d517059", "type": "license.request", "timestamp": "2026-08-30T18:02:50.158Z" }

...

Bans y riesgo

Razones de riesgo

Sin razones activas.

Bans activos

Sin bans activos.

Figura 6.2: Cliente de Fase 2 con reproducción activa y panel de observabilidad.

El laboratorio de Fase 2 conserva las pruebas del manifiesto y añade dos secuencias de bloqueo reproducibles. La primera envía ocho logins con contraseña incorrecta y el mismo `deviceId`, lo que activa el ban temporal de dispositivo. La segunda mantiene una sesión legítima y solicita cinco sesiones adicionales:

todas reciben 409CONCURRENCY_LIMIT; el score observado progresa como 0, 25, 50, 75 y 100, y se activa el ban de cuenta. En ambos casos se detiene el heartbeat, se descarga el reproductor y se vuelve a solicitar el manifiesto con el token existente. La Figura 6.3 documenta el segundo escenario: además del estado visible, el registro local conserva la progresión, el motivo del ban y el rechazo 401 del manifiesto.

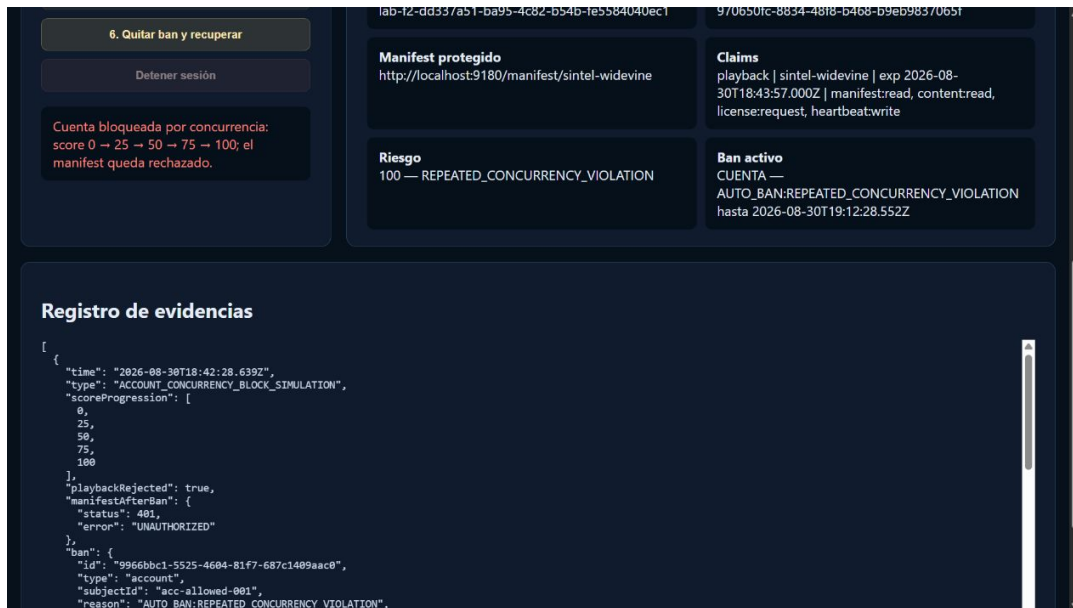


Figura 6.3: Laboratorio de Fase 2 tras alcanzar 100 puntos por rechazos de concurrencia y activar el ban automático de cuenta.

La misma interfaz permite retirar cualquiera de los dos bans con el token administrativo y volver a reproducir. Para ello renueva la credencial mediante heartbeat o crea una nueva sesión si la anterior ha expirado durante el bloqueo. La retirada no reinicia el score, que permanece como rastro del comportamiento observado. Estas secuencias han proporcionado mecanismos controlados para el Capítulo 7, pero no han sustituido a las evidencias de red y servidor utilizadas para contrastar el resultado visible.

Se ha incorporado además un laboratorio específico de CDN leeching, accesible en `localhost:9302` y `localhost:9402`. Su backend actúa como un cliente externo no sujeto a CORS: autentica al usuario de prueba, crea una sesión y vuelve a servir al navegador el manifiesto y los segmentos obtenidos con la clave CENC conocida. La interfaz permite contrastar la ausencia de token, el origen web no permitido, una única reproducción válida, una segunda sesión, la copia del token desde otra instancia, el cruce de activo y el uso posterior a la parada. En Fase 2 muestra también el score y la razón generada al consumir contenido

sin que se haya observado una solicitud de licencia. Las capturas y el análisis se presentan en el capítulo de evaluación.

6.6. Integración completa del entorno

El flujo integrado comienza en uno de los frontales web y converge en Varnish. Tras el login, el cliente conserva el token de acceso únicamente para crear sesiones y, en Fase 2, consultar administración. La sesión devuelve las URLs locales de manifiesto y licencia y un token efímero. Shaka Player introduce el contexto en sus filtros de red; el *control-plane* valida y actualiza la sesión; y el servidor interno de licencia reenvía el challenge sin quedar expuesto al anfitrión. El laboratorio externo utiliza exactamente esas rutas, por lo que no dispone de un acceso privilegiado al origen.

En Fase 2, cada paso relevante produce además un evento y puede modificar el riesgo. La respuesta activa se aplica antes de alcanzar manifiesto, contenido, heartbeat o licencia. El plano administrativo permanece disponible para observar y revertir el estado. Redis se configura sin RDB ni AOF para que reiniciar el contenedor deje el laboratorio en un estado conocido; la persistencia duradera y la alta disponibilidad no forman parte de esta implementación.

Con esta integración han quedado implementadas las dos etapas de mejora: Fase 1 corrige los caminos de acceso y crea una sesión verificable; Fase 2 añade memoria del comportamiento y capacidad de bloqueo. En el Capítulo 7 se ha evaluado si estos mecanismos producen las respuestas esperadas frente a los escenarios de Fase 0, qué evidencias dejan y qué coste funcional introducen.

7

Evaluación y resultados

En este capítulo se ha llevado a cabo una evaluación comparativa de la línea base y de las dos etapas de mejora. La pregunta experimental no se ha limitado a comprobar si Widevine permite la reproducción, sino que ha tratado de determinar qué ocurre cuando una petición abandona el flujo legítimo, si la decisión queda asociada a una sesión y qué respuesta ofrece el sistema cuando el comportamiento se repite. Para evitar que la descripción de la implementación sustituya a la evidencia, los resultados se han obtenido mediante una batería automatizada, reproducciones reales en el navegador, capturas PCAP y exportaciones del estado de Redis.

La ejecución final ha comprendido una batería general de 65 comprobaciones funcionales: 12 sobre Fase 0, 21 sobre Fase 1 y 32 sobre Fase 2. Todas han producido el resultado esperado. Posteriormente se ha ejecutado una batería complementaria de 27 comprobaciones, centrada en Key Leak y CDN leeching, con 3 casos en Fase 0, 11 en Fase 1 y 13 en Fase 2. El resultado agregado ha sido de 92 casos superados sobre 92. Las 240 observaciones temporales originales, con 20 repeticiones por combinación de fase y operación, se han mantenido sin cambios porque los nuevos escenarios evalúan controles distintos y no alteran esa comparación. Los datos originales se han conservado en **evidencias**, junto con los scripts de repetición, las capturas, los logs, los resúmenes HTTP sin tokens y las huellas SHA-256.

7.1. Escenarios de evaluación

La evaluación se ha desarrollado siguiendo siempre el mismo orden. En primer lugar, se ha comprobado la disponibilidad del entorno y el funcionamiento de una reproducción legítima. A continuación, se han introducido credenciales o contextos incorrectos y, por último, se han ejecutado los escenarios con estado: concurrencia, detención de sesión, score y bans. Fase 1 se ha reiniciado antes de cada bloque que requería un estado vacío, mientras que Fase 2 ha partido de una instancia de Redis limpia. Esta separación ha evitado que una sesión anterior o un contador de riesgo contaminase el caso siguiente.

La Tabla [7.1](#) resume la matriz. El símbolo “—” indica que el concepto no aplica en la fase correspondiente.

Tabla 7.1: Escenarios de la batería comparativa

Escenario	Fase 0	Fase 1	Fase 2
Usuario autorizado	Reproducción permitida	Reproducción permitida	Reproducción permitida
Usuario sin <i>entitlement</i>	Configuración 403	Sesión 403	Sesión 403
Ruta <code>license/no_auth</code>	Disponible	401/404	401/404
Origen y licencia internos	Puertos publicados	Sin publicación	Sin publicación
Token ausente o alterado	Solo controla ruta oficial	401	401
Activo o sesión distintos	Sin sesión de playback	403/409	403/409
Token después de detener	—	401	401
Segunda reproducción	Sin límite	409	409 y señal de riesgo
Key Leak sin token	MPD y contenido 200	401	401
Sesión única, token y clave	Reproducción posible	Reproducción posible	Posible y +20 de riesgo
Token copiado a otra instancia	Sin binding	401	401 y evento
Abuso reiterado	Sin detección	Rechazo puntual	Score, eventos y ban
Recuperación administrativa	—	—	Retirada de ban

Se emplearon dos cuentas: una con acceso a `sintel-widevine` y `local-cenc-clearkey`, y otra sin activos permitidos. En las fases endurecidas cada login incluyó un `deviceId` y cada sesión un `clientInstanceId`; la sesión resultante se utilizó para manifest, heartbeat, contenido y licencia. Las pruebas negativas alteraron una sola condición cada vez: credencial, activo, identificador de sesión, instancia o estado de la reproducción.

La evidencia de red se ha obtenido dentro del espacio de red del contenedor CDN. En la batería general de Fase 1 se han conservado 62 paquetes que contienen 17 solicitudes y 17 respuestas HTTP. En Fase 2 se han generado dos PCAP: 111

paquetes para concurrencia y 83 para fallos de autenticación. El archivo PCAPNG resultante de su agregación contiene 194 paquetes. La ampliación de CDN leeching se ha capturado de forma independiente: 174 paquetes con 14 solicitudes y 14 respuestas en Fase 1, y 220 paquetes con 18 solicitudes y 18 respuestas en Fase 2. Los resúmenes JSON se han generado con TShark sin exportar **Authorization**; los PCAP brutos sí pueden contener tokens efímeros del laboratorio. Al igual que en el Capítulo 4, no se ha intentado interpretar una licencia Widevine ni obtener claves a partir de la captura.

Durante la primera ejecución se ha detectado una desviación relevante. Se ha comprobado que Fase 2 no contrastaba la cabecera **X-Playback-Session-Id** en **/license**; en una prueba con una sesión distinta, la petición ha alcanzado el proxy interno y ha finalizado con un HTTP 500. La suite inicial ha superado 64 de los 65 casos. A partir de este resultado, se ha añadido la comprobación antes del reenvío y se ha repetido la batería completa: la misma solicitud ha pasado a devolver **409SESSION_MISMATCH** y el resultado final ha sido de 65 casos superados sobre 65. La incidencia y ambas observaciones se conservan en `evidencias/resultados/incidencia_binding_licencia_fase2.json`.

7.2. Métricas utilizadas

Se han registrado métricas funcionales y operativas. Entre las primeras se han incluido el código HTTP, el motivo estructurado, la disponibilidad de caminos alternativos, la creación de eventos, el score y la presencia de bans. En la ampliación se han añadido los bytes de contenido entregados, el número de objetos recuperados, la ausencia de solicitudes de licencia y el binding de instancia. Las métricas operativas originales han recogido el tiempo de respuesta de los endpoints locales y una instantánea del consumo de los contenedores. Una reproducción solo se ha considerado válida cuando el navegador ha cargado el manifest y ha mostrado el vídeo en ejecución; cuando el activo ha requerido licencia, también se ha comprobado su obtención. Por tanto, no se ha aceptado únicamente el mensaje de la interfaz.

La duración HTTP se ha medido desde el inicio de la solicitud hasta la lectura completa de la respuesta. Se han realizado 20 repeticiones por operación. Este tamaño se ha elegido para mantener la prueba de Fase 1 dentro de su límite de 20 logins por minuto. En Fase 2 se ha limpiado Redis antes de cada ciclo temporal: de otro modo, medir veinte heartbeats consecutivos habría activado deliberadamente la regla antifraude y habría mezclado el rendimiento con la respuesta de seguridad.

7.2.1. Tiempo de respuesta

La Tabla 7.2 presenta mediana y percentil 95. En Fase 0, “autorizar reproducción” corresponde a `/playback/config`; en las fases restantes corresponde a crear `/playback/session`. Manifest, heartbeat y parada no tienen equivalente en la línea base porque Fase 0 no mantiene una sesión de reproducción diferenciada.

Tabla 7.2: Tiempos de respuesta observados, 20 muestras por operación

Fase	Operación	Mediana (ms)	P95 (ms)
0	Login	15,100	17,779
0	Autorizar reproducción	15,639	24,857
1	Login	15,489	26,231
1	Autorizar reproducción	15,072	24,332
1	Manifest	44,166	79,107
1	Heartbeat	9,347	20,541
1	Detener sesión	15,639	27,150
2	Login	18,659	32,617
2	Autorizar reproducción	11,293	31,528
2	Manifest	84,614	125,130
2	Heartbeat	19,595	32,182
2	Detener sesión	29,335	32,360

En el login de Fase 1 se ha observado una mediana prácticamente igual a la de Fase 0. En Fase 2, dicha mediana se ha incrementado un 20,5 % con respecto a Fase 1. El heartbeat ha pasado de 9,347 a 19,595 ms y la parada, de 15,639 a 29,335 ms. Estas diferencias resultan coherentes con el acceso adicional a Redis y la creación de eventos, aunque no constituyen un benchmark de producción, ya que las mediciones se han realizado en un único equipo y con tráfico secuencial.

El manifest ha presentado una variabilidad mayor. En Fase 2 se ha obtenido una mediana de 84,614 ms, frente a los 44,166 ms de Fase 1. Ambas rutas descargan el MPD desde la infraestructura pública de Shaka antes de responder, por lo que la cifra combina red externa, reescritura local, planificación de Docker y, en Fase 2, persistencia y observabilidad. En consecuencia, no resulta adecuado atribuir toda la diferencia a Redis.

7.2.2. Número de accesos bloqueados

Se han contabilizado únicamente las respuestas 4xx esperadas; un error 5xx se consideraría un fallo de la prueba. En Fase 0 se han registrado tres rechazos válidos en su camino oficial: credenciales incorrectas, usuario sin permiso y licencia oficial sin token. Sin embargo, el origen y la ruta alternativa de licencia se han mantenido accesibles. El PCAP del bypass conserva dos `POST/license/no_auth` con HTTP 200 sin login ni configuración previa.

En Fase 1 se han registrado 14 rechazos esperados y en Fase 2, 17. La cifra mayor no significa por sí sola que Fase 2 sea “un 21 % más segura”, pues contiene tres escenarios adicionales. Su significado es funcional: todas las variantes negativas de las fases endurecidas han finalizado con 401, 403, 404 o 409, sin errores 5xx en la regresión final de ambas fases. Entre ellas se encuentran el token alterado, el activo distinto, la sesión distinta, el contenido no asociado, la ruta heredada y el token posterior a la parada.

7.2.3. Detección de abuso

El rechazo de concurrencia permite distinguir entre prevención y detección. En Fase 1 se ha obtenido un 409 ante una segunda sesión, pero no se ha conservado ninguna puntuación. En Fase 2 se ha mantenido una sesión válida y se han enviado cinco solicitudes adicionales. Las cinco han devuelto `CONCURRENCY_LIMIT`; el score observado después de cada una ha sido 0, 25, 50, 75 y 100. El primer rechazo únicamente ha iniciado el contador y los cuatro siguientes han añadido 25 puntos. Al alcanzar los 100 puntos, se ha creado un ban de cuenta de 30 minutos con la razón `AUTO_BAN:REPEATED_CONCURRENCY_VIOLATION`.

La exportación de Redis conserva cinco eventos `playback.concurrency_rejected`, cuatro `risk.incremented`, un `ban.created`, un `ban.cleared` y la parada final. Mientras el ban se ha mantenido activo, el manifest ha respondido con un 401. La Figura 6.3, incluida en el capítulo anterior, muestra este estado; el PCAP registra seis `POST/playback/session`: una creación 201 y cinco conflictos 409.

En el segundo escenario se han enviado ocho contraseñas incorrectas desde el mismo dispositivo. Todas han recibido un 401, tras lo cual se ha creado un ban `AUTH_FAILURE_BURST` de 15 minutos y el manifest posterior ha quedado rechazado. Al tratarse de una respuesta directa sobre el dispositivo, el score de la cuenta se ha mantenido en cero. La Figura 7.1 muestra el resultado visible y la secuencia registrada.

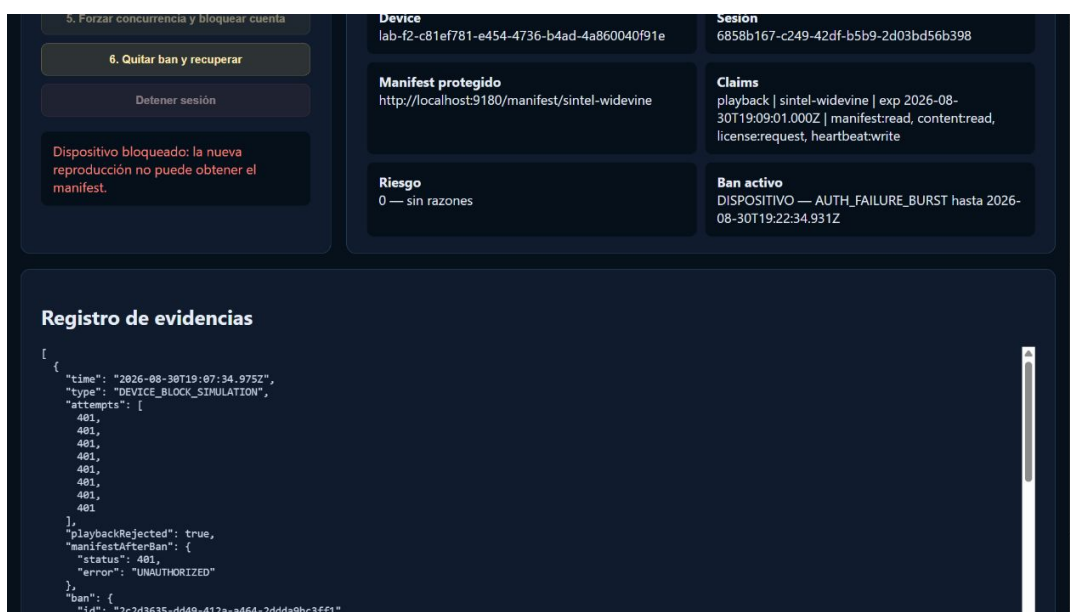


Figura 7.1: Evaluación de Fase 2 tras ocho fallos de autenticación y activación del ban de dispositivo.

En ambos escenarios, el plano administrativo se ha mantenido operativo y ha permitido retirar el ban, renovar la credencial o crear una nueva sesión cuando la anterior había expirado. El score de la cuenta no se ha reiniciado al retirar el ban de concurrencia, sino que se ha mantenido en 100 como evidencia del comportamiento previo.

7.2.4. Impacto en rendimiento

La instantánea final de los contenedores ha mostrado un consumo de 29,16 MiB para el plano de control de Fase 1 y de 38,83 MiB para el de Fase 2. Redis ha utilizado 3,61 MiB adicionales. Los contenedores Varnish se han situado en 96,11 y 99,37 MiB. Los laboratorios externos han consumido 24,80 MiB en Fase 1 y 32,38 MiB en Fase 2, aunque no forman parte del camino de entrega y se han excluido de la comparación del plano de control. El uso puntual de CPU se ha situado por debajo del 1% en los componentes citados. Estos valores describen el laboratorio en reposo después de la campaña; no deben interpretarse como capacidad máxima ni como una prueba de carga.

El coste funcional se ha mantenido reducido en términos absolutos: ninguna mediana local ha superado los 30 ms, salvo la obtención del manifest, condicionada por el upstream externo. Fase 2 sí ha mostrado una latencia mayor en login,

heartbeat y parada, además de aproximadamente 13,3 MiB sobre la suma del plano de control y Redis frente al plano de control de Fase 1. A cambio, incorpora estado compartido, eventos, score y respuesta administrativa, capacidades ausentes en las fases anteriores.

7.3. Evaluación del sistema base

Fase 0 ha superado sus comprobaciones legítimas. El usuario autorizado ha reproducido correctamente el contenido Widevine y el activo CENC local, mientras que el usuario sin *entitlement* ha recibido un 403 al solicitar la configuración. La ruta oficial de licencia ha devuelto un 401 en ausencia de token. Por tanto, la línea base no es un sistema sin controles, sino un sistema cuyos controles no cubren todos los caminos equivalentes.

La principal diferencia se ha evidenciado al abandonar el cliente oficial. El origen ha respondido con un HTTP 200 en `localhost:8081` y el servidor de licencias ha publicado su health en `localhost:8082`. Una solicitud binaria artificial a `/license/no_auth` ha alcanzado la lógica de proxy sin recibir un 401, 403 o 404; el challenge inválido ha finalizado con un 500 y la respuesta ha conservado `X-Phase0-Weakness`. Más relevante aún, el PCAP de reproducción real ya analizado en el Capítulo 4 contiene una preflight y dos `POST/license/no_auth` con respuesta 200, sin login ni `/playback/config`. Esta combinación permite evitar la confusión entre una “ruta alcanzable” y un “challenge Widevine válido”.

La demostración ClearKey también ha confirmado que el MPD, las inicializaciones y los primeros segmentos podían recuperarse sin URI de licencia cuando la clave de laboratorio era conocida. Este resultado no rompe Widevine; muestra que cifrar el activo no compensa la exposición simultánea del contenido y del material necesario para descifrar otro activo CENC controlado.

En conjunto, Fase 0 ha bloqueado el uso no permitido dentro de la interfaz oficial, pero no ha gobernado de extremo a extremo el acceso al origen y al proxy de licencia. Tampoco ha permitido revocar una reproducción concreta, limitar la concurrencia ni correlacionar eventos por sesión.

7.4. Evaluación del sistema propuesto

Fase 1 ha mantenido la reproducción Widevine real y ha eliminado los caminos alternativos. El PCAP de evaluación contiene el login, la creación de la sesión,

el manifest, dos solicitudes de licencia y la parada. Los cuatro casos negativos del laboratorio han producido 401, 401, 409 y 403, como recoge la Figura 7.2. La distribución completa del PCAP ha sido de seis respuestas 200, una 201, seis preflight 204, dos 401, una 403 y una 409.

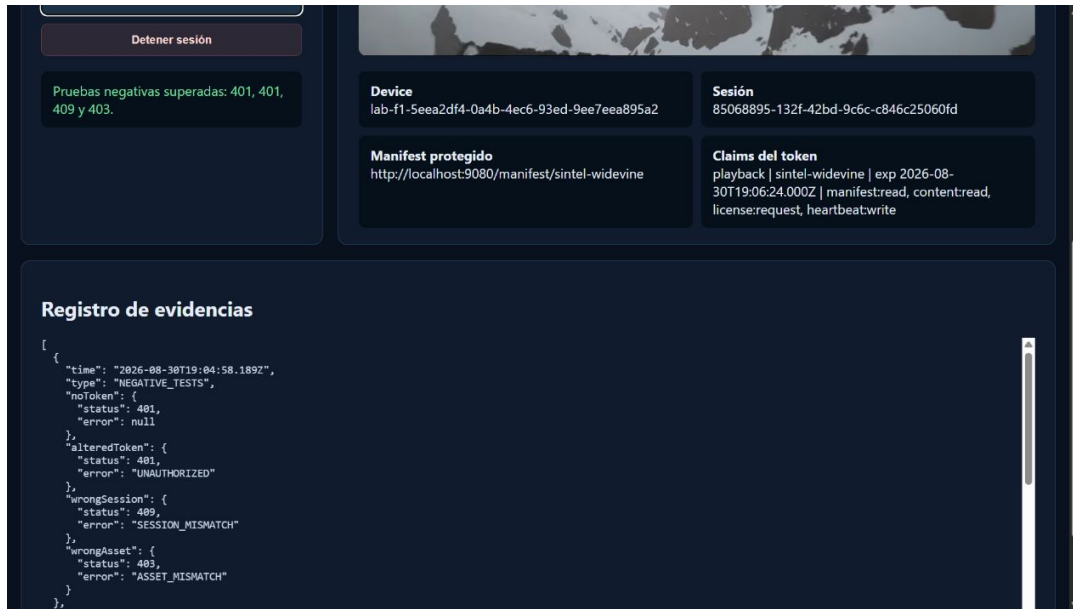


Figura 7.2: Pruebas negativas de Fase 1 con reproducción legítima activa y binding de token, activo y sesión.

La revisión de la topología ha confirmado que el origen, el servidor de licencias y el plano de control no tienen bindings en el anfitrión. `/license/no_auth` ha devuelto un 401 sin credencial y un 404 con un token válido, lo que demuestra que la ruta no se limita a permanecer oculta tras la autenticación, sino que ha desaparecido del backend. Después de detener la sesión, el mismo token de reproducción ha devuelto un 401 al solicitar el manifest.

Fase 2 ha conservado estos 14 comportamientos preventivos y ha añadido capacidades de administración, detección y recuperación. La cuenta sin *entitlement* tampoco ha podido consultar `/admin/overview`. Los dos escenarios de abuso han generado los bans esperados y el plano administrativo ha permitido retirarlos sin perder observabilidad. El PCAP de concurrencia ha registrado 35 solicitudes y respuestas HTTP, mientras que el de autenticación ha registrado 25. Los estados JSON contienen, respectivamente, 19 y 18 eventos de servidor.

La corrección de `/license` durante la campaña refuerza una conclusión metodológica: compartir arquitectura no garantiza que dos fases apliquen exactamente las mismas validaciones. La regresión automatizada descubrió una diferencia que

no era visible en la reproducción legítima. Tras la corrección, manifest, contenido y licencia contrastan de forma coherente la sesión indicada por el cliente.

7.5. Evaluación de Key Leak y CDN leeching

La batería complementaria ha separado tres conceptos relacionados, pero no equivalentes. El Key Leak presupone que la clave de contenido ya es conocida; el CDN leeching consiste en utilizar la infraestructura del proveedor para obtener manifiesto y segmentos fuera del cliente previsto; y el *restreaming* vuelve a distribuir el contenido después de haberlo recuperado o descifrado. Los controles HTTP pueden limitar las dos primeras etapas, pero no retirar los bytes que un usuario autorizado ya ha recibido.

En Fase 0 se ha repetido el acceso al activo CENC utilizado en el Capítulo 4. El manifiesto, la inicialización y un segmento han respondido con HTTP 200 sin token, dejando 138.757 bytes accesibles en la muestra. La captura ya incluida en dicho capítulo demuestra que la clave conocida permite reproducir el activo desde la herramienta externa; esta evidencia se ha reutilizado para no duplicar figuras.

En Fase 1 y Fase 2, la petición del manifiesto y el acceso directo a un segmento sin token han devuelto 401. Cuando la prueba se ha ejecutado desde un origen web no permitido, el servidor ha respondido con HTTP 200 pero ha conservado como `Access-Control-Allow-Origin` el cliente oficial. El navegador ha bloqueado la lectura por CORS. Este resultado se ha interpretado como contención del navegador, no como autorización: el backend del laboratorio, que simula un cliente IPTV no sujeto a CORS, ha podido completar las mismas solicitudes al aportar un token válido.

El caso positivo ha utilizado una única sesión, una instancia coherente y la clave CENC conocida. En ambas fases endurecidas se han recuperado tres objetos de contenido con HTTP 200, 253.023 bytes en total y ninguna solicitud al endpoint de licencia. El navegador ha reproducido el vídeo. En Fase 1 esta secuencia no ha generado una señal adicional. En Fase 2, después del manifest y del tercer objeto, se ha creado un único evento `key_leak.pattern_detected` y se han añadido 20 puntos con la razón `POSSIBLE_KEY_LEAK_LICENSE_BYPASS`. Una cuarta solicitud no ha vuelto a puntuar la misma sesión.

La Figura 7.3 recoge este caso límite. Su aceptación no constituye un defecto accidental del laboratorio: con un token válido y una sola reproducción, el servidor observa un acceso autorizado. La ausencia de licencia resulta sospechosa, pero también podría deberse a una licencia disponible previamente; por ello se ha tratado como señal y no como ban inmediato.

Key Leak y CDN Leeching — FASE2
Reproductor externo con clave conocida. Se diferencia el bloqueo del navegador de la autorización aplicada en el CDN.

1. Probar cliente IPTV sin token
2. Probar navegador desde origen externo
3. Reproducir con token válido y clave
4. Probar concurrencia, clon y otro canal
Detener sesión

Caso límite confirmado: una sesión válida reproduce desde el backend externo con la clave conocida.

El modo positivo usa un backend equivalente a un servicio IPTV: obtiene una sesión válida, consume los segmentos protegidos y los vuelve a servir al navegador. No realiza ninguna petición de licencia.

Resultado de controles

```
{
  "CASO_POSITIVO": {
    "result": "REPRODUCCION_EXTERNA_INICIADA",
    "keySource": "clave conocida",
    "licenseRequest": false,
    "session": {
      "labSessionId": "da908677-6a35-4193-9658-"
    }
  }
}
```



SESIÓN 9020df4c-6c5f-4fb9-9fa1-0065a58e51e1	SEGMENTOS SERVIDOS 11	BYTES RETRANSMITIDOS 903.974
LICENCIAS SOLICITADAS 0	RIESGO FASE 2 20	RAZONES POSSIBLE_KEY_LEAK_LICE NSE_BYPASS

Figura 7.3: Reproducción externa en Fase 2 con sesión única, token válido y clave conocida; se han servido segmentos sin licencia y el riesgo ha aumentado en 20 puntos.

Las pruebas de contexto han acotado este resultado. Una segunda sesión de la misma cuenta ha recibido 409; el mismo token presentado con otro `clientInstanceId`, 401; su aplicación al activo Widevine, 403; y la recuperación de contenido después de detener la sesión, 401. La Figura 7.4 muestra estas respuestas y el bloqueo de CORS observado desde el navegador. La distribución del PCAP de Fase 1 ha sido de siete respuestas 200, una 201, cuatro 401, una 403 y una 409. En Fase 2 se han obtenido once respuestas 200, una 201, cuatro 401, una 403 y una 409; las respuestas 200 adicionales corresponden a la consulta del estado y los eventos de riesgo.

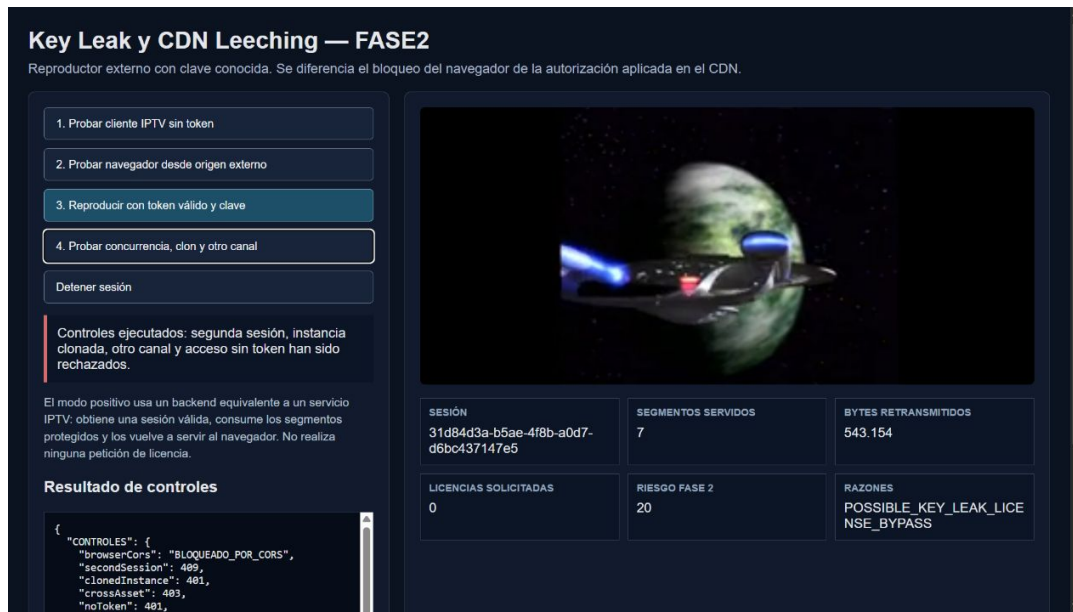


Figura 7.4: Controles de Fase 2 frente a origen web externo, concurrencia, copia entre instancias, cruce de activo y ausencia de token.

Tabla 7.3: Resultados específicos de Key Leak y CDN leeching

Prueba	Fase 0	Fase 1	Fase 2
MPD y contenido sin token	200	401	401
Origen web no permitido	Sin restricción efectiva	Bloqueo CORS	Bloqueo CORS
Cliente externo con token y clave	Reproduce	Reproduce	Reproduce y +20
Segunda sesión	Permitida	409	409
Token desde otra instancia	Sin binding	401	401 y evento
Token aplicado a otro activo	Sin binding	403	403
Contenido después de detener	Sin sesión	401	401
Comprobaciones complementarias	3/3	11/11	13/13

Los resultados han evidenciado que la mejora no consiste en hacer imposible cualquier uso externo. Las fases endurecidas han eliminado el acceso anónimo al CDN y han limitado la reutilización del token a un activo, una sesión y una instancia. Sin embargo, una cuenta legítima todavía puede actuar como origen de un *restreaming* serial. Fase 2 aporta una señal temprana para correlacionarla con otros comportamientos, pero requiere más actividad o nuevas señales para alcanzar el umbral de bloqueo.

7.6. Comparativa de resultados

La Tabla 7.4 concentra el resultado final. “Rechazo” significa respuesta preventiva; “detección” exige además conservar una señal, y “respuesta” implica modificar el estado de acceso. Esta distinción evita atribuir a Fase 1 una capacidad antifraude que no implementa.

Tabla 7.4: Comparación final de las tres fases

Propiedad	Fase 0	Fase 1	Fase 2
Widevine legítimo	Funciona	Funciona	Funciona
Ruta pública de licencia	Sí	No	No
Origen/licencia publicados	Sí	No	No
Token efímero de playback	No	Sí, 90 s	Sí, 90 s
Binding	No	Sí	Sí
cuenta–dispositivo– instancia–activo–sesión			
Manifiesto y contenido local protegidos	No	Sí	Sí
Revocación por parada	No	Sí	Sí
Concurrencia	Sin control	Rechazo 409	409 y score
Sesión válida con clave conocida	Reproduce	Reproduce	Reproduce y genera señal
Eventos persistentes	No	No	Sí, Redis
Ban automático	No	No	Cuenta y dis- positivo
Recuperación administrativa	No	No	Sí
Comprobaciones generales	12/12	21/21	32/32
Comprobaciones CDN leeching	3/3	11/11	13/13
Total por fase	15/15	32/32	45/45

La mejora principal de Fase 1 ha sido cualitativa: se ha evitado que el acceso anónimo dependa de abandonar o no la interfaz prevista. El mismo control se ha aplicado al manifest, al contenido local y a la licencia, y los servicios internos han dejado de estar publicados. La prueba externa ha mostrado, no obstante, que una sesión auténtica sigue siendo utilizable desde otro tipo de cliente mientras conserve el contexto exigido. En Fase 2 no se ha incrementado el cifrado del contenido, sino que se ha añadido memoria y capacidad de reacción sobre una ruta ya autorizada.

Los tiempos muestran que esa memoria tiene un coste, aunque se mantiene reducido en el laboratorio. No se ha observado una penalización uniforme: la mediana de creación de sesión de Fase 2 ha sido incluso menor que la de Fase 1,

mientras que las medianas de heartbeat y parada prácticamente se han duplicado. Esta dispersión, junto con los percentiles 95, aconseja no convertir una campaña secuencial de 20 muestras en una afirmación de escalabilidad.

7.7. Discusión técnica

7.7.1. Mejoras observadas

La primera mejora es la eliminación del bypass. En la línea base, una decisión correcta de *entitlement* coexistía con un proxy que no recibía identidad. En las fases endurecidas no existe un camino equivalente: el frontal y el backend exigen una credencial de reproducción y el servidor interno de licencias solo acepta contexto procedente del plano de control.

La segunda es la reducción de la ventana y el contexto de reutilización. El token de reproducción expira en 90 segundos, se liga a una sesión y a una instancia de cliente, y se renueva mediante heartbeat. Detener la sesión invalida su uso aunque el token todavía sea criptográficamente válido. Activo, cuenta, dispositivo, instancia y sesión se contrastan antes de servir recursos sensibles.

La tercera mejora solo se ha observado en Fase 2. Los rechazos repetidos dejan de ser respuestas independientes y pasan a formar una señal acumulada. Los eventos de Redis han permitido reconstruir la secuencia desde el login hasta `ban.created`, el rechazo posterior, `ban.cleared` y `playback.stopped`. También se ha conservado el consumo de contenido sin licencia como una señal de riesgo única por sesión. Esta trazabilidad constituye la base para la operación y el análisis posterior.

7.7.2. Limitaciones persistentes

El laboratorio utiliza HTTP, credenciales firmadas localmente y secretos de demostración incluidos en Docker Compose. Un despliegue real necesitaría TLS extremo a extremo, un proveedor de identidad y mecanismos externos de rotación y custodia de claves.

El activo Widevine, su MPD y el servicio CWIP pertenecen a infraestructura pública de pruebas. Los segmentos Widevine no atraviesan el origen privado del laboratorio. Por ello, esa parte de la campaña demuestra gobierno local del manifest y la licencia, pero no cuantifica una CDN comercial ni una política de licencia de producción. El activo CENC local sí ha permitido comprobar la autorización

de inicializaciones y segmentos a través de Varnish, aunque con un volumen y una duración reducidos.

Los umbrales de Fase 2 son ilustrativos. No se han calibrado con una población real, falsos positivos conocidos, diversidad geográfica o dispositivos heterogéneos. Redis funciona sin persistencia, replicación ni alta disponibilidad para facilitar el reinicio. Las medidas temporales proceden de un solo equipo, tráfico secuencial y 20 muestras; sirven para comparar este prototipo, no para dimensionar producción.

El `deviceId` lo genera JavaScript y no constituye una prueba fuerte de identidad de hardware. El watermark es visible, pero no forense. Tampoco se evalúan HDCP, robustez del CDM, seguridad del sistema operativo, captura de pantalla ni compromiso del dispositivo final.

7.7.3. Ataques que seguirían siendo posibles

Un atacante que obtuviera un token de reproducción válido y pudiera reproducir el mismo contexto de sesión, dispositivo e instancia dispondría de una ventana corta hasta la expiración o revocación. Al tratarse de una credencial bearer y de identificadores declarados por el cliente, el binding reduce la reutilización simple, pero no equivale a atestación criptográfica.

La prueba con clave conocida ha confirmado un caso más concreto: una única sesión autorizada puede recuperar un canal y reproducirlo externamente sin superar la concurrencia. Fase 2 suma 20 puntos por la ausencia de licencia observada, pero no bloquea de inmediato. Si el contenido se vuelve a servir desde una infraestructura ajena después de descargarlo, ni CORS ni la revocación posterior pueden retirar esa copia; la plataforma solo puede detener nuevas entregas y correlacionar la actividad.

Un patrón distribuido y lento podría mantenerse por debajo de los umbrales o repartir actividad entre múltiples cuentas y dispositivos. Del mismo modo, compartir una cuenta de forma serial, sin sesiones simultáneas, no activa el control de concurrencia. Un adversario también podría dirigir denegación de servicio contra Varnish, el plano de control o Redis, ámbitos no cubiertos por esta campaña funcional.

Finalmente, ningún control de acceso impide por sí solo que un usuario autorizado vuelva a capturar la salida ya descifrada. Este límite se conoce como *analog hole*: para que una persona pueda consumir una obra audiovisual, el sistema debe transformar en algún momento el contenido protegido en imágenes y sonido perceptibles, que pueden volver a registrarse mediante una cámara o

un micrófono. Por ello, incluso un DRM correctamente implementado pierde el control técnico directo sobre la copia obtenida después de la representación, una limitación inherente descrita también para los sistemas DRM de *streaming* [33]. En sentido estricto, la captura de pantalla o la extracción de búferes descifrados antes de la conversión física son ataques digitales relacionados, pero no forman parte del *analog hole* clásico. En ambos casos, el servidor observa inicialmente una reproducción autorizada, de modo que los tokens, la concurrencia y los bans pueden detener entregas posteriores, pero no retirar el material ya capturado. La marca visible empleada en el laboratorio solo aporta una atribución limitada, pues puede recortarse o taparse; técnicas como el *watermarking* forense o la protección de la salida pueden dificultar el abuso, aunque tampoco eliminan por completo este límite.

7.7.4. Mitigaciones posibles

La evolución natural sería reemplazar los secretos locales por un gestor de claves, emplear TLS externo y mTLS entre servicios, y utilizar un proveedor de identidad con tokens estándar, rotación de claves y revocación. El identificador de dispositivo podría reforzarse con atestación, claves no exportables o señales del SDK DRM, siempre evaluando privacidad y compatibilidad.

En la distribución convendría sustituir el token general del prototipo por URLs o cookies firmadas de CDN con alcance reducido, mantener la autorización sobre segmentos, emplear políticas Widevine de producción y rotar claves por activo o periodo. La señal de contenido sin licencia debería correlacionarse con volumen, número de canales, cambios de instancia y velocidad de descarga antes de bloquear. Un watermark forense individualizado complementaría al visible cuando el riesgo justifique su coste.

Para detección, los eventos deberían enviarse a una plataforma externa con retención, correlación entre regiones y control de acceso. Los umbrales se calibrarían con tráfico histórico, cohortes y métricas de falsos positivos. Rate limiting distribuido, protección WAF/DDoS y Redis persistente y replicado completarían la disponibilidad. Estas medidas no alteran la conclusión experimental: la defensa efectiva procede de combinar DRM con autorización extremo a extremo, contexto de sesión, observabilidad y respuesta, no de confiar en una única capa.

8

Conclusiones

8.1. Conclusiones generales

El trabajo realizado ha permitido responder a la pregunta de investigación planteada: desplegar Widevine DRM de forma aislada no proporciona seguridad suficiente frente al abuso de distribución en una plataforma OTT. Widevine aporta una protección necesaria sobre las claves, las licencias y el descifrado dentro del CDM, pero no decide por sí mismo quién puede iniciar una reproducción, cuántas sesiones puede mantener una cuenta, desde qué contexto se reutiliza un token o si el contenido se está consumiendo de una forma compatible con la política del servicio. Estas decisiones pertenecen a la plataforma y deben aplicarse de manera coherente en toda la cadena.

La Fase 0 ha evidenciado esta diferencia. El cliente oficial disponía de autenticación y *entitlements*, pero la existencia de rutas alternativas permitía alcanzar el origen o el proxy de licencia sin conservar ese contexto. La demostración ClearKey ha mostrado además que, cuando una clave ya es conocida, proteger únicamente la aplicación o el formulario de acceso no impide recuperar el manifiesto y los segmentos desde otro cliente. El sistema podía funcionar correctamente para un usuario legítimo y seguir siendo vulnerable fuera del recorrido previsto por la interfaz.

Fase 1 ha corregido este problema mediante controles preventivos. El origen y

el servidor de licencias han dejado de estar publicados, y el manifiesto, el contenido local, el heartbeat y la licencia han pasado a depender de una sesión efímera ligada a cuenta, dispositivo, instancia de cliente y activo. La concurrencia se ha limitado en el plano de control y una sesión detenida ha dejado de ser utilizable aunque el token aún no hubiera alcanzado su expiración criptográfica. La mejora fundamental no ha consistido en añadir otro cifrado, sino en extender la autorización desde la aplicación hasta los recursos que hacen posible la reproducción.

Fase 2 ha añadido una segunda capacidad: recordar y relacionar lo que sucede. Los eventos, los contadores temporales, la puntuación de riesgo y los bans han permitido transformar rechazos aislados en una respuesta observable. Los escenarios de concurrencia y fallos de autenticación han demostrado el bloqueo automático de cuenta y dispositivo, mientras que la prueba de Key Leak ha incorporado como señal el consumo de contenido sin una licencia observada en la sesión. La batería completa ha terminado con 65 de 65 comprobaciones generales y 27 de 27 comprobaciones complementarias superadas.

Estos resultados confirman que la protección no puede limitarse al nivel de aplicación. Debe abarcar el cliente, la identidad, los tokens, el ciclo de vida de las sesiones, la CDN, el origen, el servidor de licencias, la observabilidad y la capacidad de revocación. Si una sola ruta equivalente queda fuera de la política, un atacante puede evitar las capas anteriores sin necesidad de romper el DRM. Del mismo modo, CORS puede limitar un navegador de origen ajeno, pero no autentica un reproductor nativo ni un backend IPTV. La decisión efectiva debe mantenerse en el servidor y repetirse en cada recurso sensible.

La arquitectura propuesta es amplia dentro del alcance experimental, pero no constituye una solución perfecta. El caso positivo de CDN leeching resulta especialmente importante: una única sesión válida, con un token vigente y una clave conocida, todavía puede reproducir un canal desde un cliente externo. Fase 2 ha registrado el patrón y ha añadido riesgo, pero no lo ha bloqueado de inmediato porque una licencia previamente disponible podría producir una secuencia parecida. Esta limitación no invalida el sistema; muestra que la seguridad real exige equilibrar prevención, detección, continuidad del servicio y falsos positivos.

Esta necesidad es aún mayor en el contenido en directo. El valor de una retransmisión se concentra en una ventana temporal corta y un abuso puede materializarse antes de que termine un proceso manual de análisis. Protegerla requiere esfuerzos de ciberseguridad constantes: revisar rutas y configuraciones, rotar claves y secretos, observar nuevas formas de automatización, ajustar umbrales, investigar alertas y comprobar que los mecanismos de revocación siguen funcionando. La arquitectura no debe entenderse como una configuración que se instala una vez, sino como un proceso operativo que evoluciona con las amenazas.

En este sentido, las tres fases constituyen el inicio de un modelo de madurez para plataformas OTT. Fase 0 representa un servicio funcional apoyado principalmente en DRM; Fase 1 introduce gobierno preventivo del acceso; y Fase 2 incorpora observabilidad, riesgo y respuesta. El valor del modelo no reside únicamente en sus controles concretos, sino en ofrecer una ruta incremental y evaluable para avanzar desde la protección del contenido hacia la protección del sistema completo.

8.2. Cumplimiento de objetivos

El objetivo general se ha cumplido dentro del alcance de laboratorio. Se ha diseñado, implementado y auditado un entorno OTT reproducible con Widevine, contenido CENC local y una arquitectura de defensa en profundidad. La comparación entre las tres fases ha permitido observar la diferencia entre protección criptográfica, autorización preventiva y respuesta basada en comportamiento.

El cumplimiento de los objetivos específicos puede resumirse de la siguiente manera:

1. **Despliegue de la infraestructura OTT.** Se ha construido una cadena contenerizada con cliente, CDN basada en Varnish, plano de control, origen, proxy de licencia y laboratorios de auditoría. El contenido Widevine público ha permitido verificar el flujo DRM real y el activo CENC local ha hecho observable la autorización sobre manifiesto, inicializaciones y segmentos.
2. **Auditoría del ecosistema DRM.** Se han estudiado el intercambio de licencias, las rutas oficiales y alternativas y el comportamiento de un cliente externo. La auditoría no ha pretendido romper la criptografía de Widevine, sino comprobar si la plataforma conservaba identidad y autorización en todos los caminos equivalentes. Los PCAP, capturas y JSON del directorio de evidencias permiten reconstruir los escenarios.
3. **Autorización dinámica.** Se ha implementado un proxy protegido que valida identidad, *entitlement*, activo, sesión, dispositivo e instancia antes de entregar manifiesto, contenido o licencia. La IP se ha empleado como señal de correlación y no se ha realizado geolocalización. El objetivo se ha alcanzado en su función de autorización contextual.
4. **Control de abuso y respuesta.** Se han materializado límites de concurrencia, contadores temporales, detección de patrones, puntuación de riesgo, eventos y bans de cuenta o dispositivo. Se ha descartado el bloqueo

automático por IP como respuesta principal, dado que una dirección puede ser compartida o cambiar. La Fase 2 ha demostrado tanto la activación de los bloqueos como su retirada administrativa y la conservación de la trazabilidad.

La metodología incremental también se ha cumplido. Cada fase ha partido de la anterior, ha introducido controles identificables y ha sido sometida a una regresión. Esto ha permitido identificar errores de implementación en la Fase 1, y corregirlos antes de proceder con el desarrollo de la Fase 2.

8.3. Limitaciones del trabajo

La primera limitación deriva del carácter experimental del entorno. Los servicios utilizan HTTP local, credenciales y secretos de demostración y una única máquina Docker. Un despliegue de producción requeriría TLS extremo a extremo, custodia externa de secretos, rotación de claves, aislamiento más estricto y alta disponibilidad. Las mediciones temporales describen únicamente el equipo utilizado y no constituyen una prueba de carga o escalabilidad.

El activo Widevine y el servicio CWIP proceden de infraestructura pública de pruebas. Sus segmentos no atraviesan el origen privado del laboratorio, por lo que esa parte de la evaluación se ha concentrado en la protección del manifiesto y la licencia. El activo CENC local ha permitido estudiar el acceso a segmentos de manera controlada, pero su duración, volumen y patrón de consumo son menores que los de una emisión en directo real.

Los identificadores de dispositivo e instancia se generan en el cliente y no equivalen a una atestación criptográfica de hardware. Un atacante con control suficiente sobre el cliente podría reproducir esos valores. Tampoco se han evaluado la robustez del CDM, la seguridad del sistema operativo, la captura de pantalla o la extracción de la salida ya descifrada. Ningún control de acceso elimina por completo el denominado *analog hole*.

Las reglas de riesgo se han configurado para producir escenarios reproducibles, no a partir de tráfico real de una población de usuarios. No se ha calculado una tasa de falsos positivos ni se han estudiado comportamientos por región, dispositivo, tipo de conexión o perfil de consumo. Redis se ha desplegado sin persistencia ni replicación para poder reiniciar el laboratorio en un estado conocido.

Finalmente, la detección de Key Leak es heurística. La ausencia de una petición de licencia puede indicar que la clave es conocida o que el cliente dispone

de material válido obtenido anteriormente. Por este motivo, el patrón añade riesgo pero no genera un ban por sí solo. Un ataque lento, distribuido entre varias cuentas o limitado a una reproducción serial podría permanecer por debajo de los umbrales. La arquitectura reduce superficie y aumenta visibilidad, pero no puede garantizar la eliminación de toda redistribución no autorizada.

8.4. Líneas futuras

La primera línea de evolución consiste en aproximar el laboratorio a una infraestructura de producción. Esto implica incorporar TLS en el perímetro y mTLS entre servicios, un proveedor de identidad con tokens estándar, gestión centralizada de secretos, rotación de claves y políticas de licencia Widevine reales. La CDN debería utilizar URLs o cookies firmadas de alcance reducido, autorización sobre todos los segmentos y controles que impidan el acceso directo al origen.

La identidad del dispositivo puede reforzarse mediante claves no exportables, atestación y señales proporcionadas por el SDK DRM. Estas medidas deberían combinarse con watermarking forense individualizado, ya que la marca visible utilizada en el prototipo aporta atribución limitada. La rotación de claves por canal o periodo reduciría el valor temporal de una clave filtrada, especialmente en emisiones en directo.

En Fase 2, la evolución natural es desacoplar la observabilidad del plano de control y enviar los eventos a una plataforma externa. Un SIEM permitiría retención, consultas históricas, correlación entre regiones y alertas operativas. Los umbrales deberían calibrarse con tráfico real, cohortes y métricas de falsos positivos. La velocidad de descarga, el número de canales accedidos, los cambios de instancia y los patrones de licencia podrían formar una señal conjunta más robusta que cualquiera de ellos por separado.

También sería necesario evaluar disponibilidad y resistencia a ataques contra la propia defensa. Rate limiting distribuido, WAF, mitigación DDoS, colas de eventos y Redis persistente y replicado permitirían estudiar qué sucede cuando Varnish, el plano de control o el almacén de estado reciben carga hostil. La seguridad no debería introducir un punto único de fallo que interrumpa una emisión legítima.

8.4.1. Evolución hacia un modelo de madurez

Como continuación del trabajo, las fases pueden formalizarse como un modelo de madurez aplicable a plataformas OTT. La Tabla 8.1 propone una primera estructura que debería validarse posteriormente en entornos y organizaciones diferentes.

Tabla 8.1: Propuesta inicial de niveles de madurez para la protección OTT

Nivel	Enfoque	Capacidades principales
0	Protección funcional	DRM y reproducción legítima, con controles concentrados en la aplicación.
1	Prevención integral	Origen interno, licencia protegida, token efímero, binding contextual, autorización de CDN y concurrencia.
2	Detección y respuesta	Estado compartido, eventos, reglas de riesgo, bans, revocación y recuperación administrativa.
3	Operación adaptativa	Telemetría externa, inteligencia de amenazas, calibración continua, atestación, watermarking forense, alta disponibilidad y respuesta coordinada.

La progresión no debería evaluarse como una lista de productos instalados, sino mediante propiedades verificables. Una plataforma madura debe poder demostrar que no existen rutas equivalentes sin autorización, que puede atribuir una reproducción a un contexto, que detecta desviaciones, que revoca sin depender del cliente y que aprende de los incidentes. El laboratorio y su batería de evidencias ofrecen un punto de partida para definir pruebas y criterios de paso entre niveles.

Desde una perspectiva de ingeniería, trabajos preventivos como el desarrollo resultan más útiles como medida principal que confiar en el bloqueo posterior de contenidos o infraestructuras ilícitas, incluidos los bloqueos dinámicos de dominios y direcciones IP empleados en la estrategia antipiratería de LaLiga [34]. Estos bloqueos pueden complementar la respuesta frente a una retransmisión ya localizada, pero actúan cuando el contenido ha salido del sistema, no corrigen una licencia expuesta, un token reutilizable o una CDN mal autorizada y exigen especial precisión cuando una dirección comparte servicios heterogéneos. La prioridad debería ser impedir y detectar la extracción en el origen; el bloqueo externo

puede permanecer como apoyo, pero no sustituir a una arquitectura segura ni al esfuerzo continuo de ciberseguridad.



Organización, conservación y reproducción de las evidencias

Este apéndice describe la organización del directorio **evidencias**, que conserva los artefactos utilizados para respaldar la auditoría de Fase 0 y la evaluación comparativa de las fases endurecidas. Su finalidad no es repetir la interpretación desarrollada en los Capítulos 4 y 7, sino explicar cómo se relacionan las capturas de pantalla, las capturas de red, los estados exportados, los resultados automatizados y los scripts que permiten reproducir la campaña.

La estructura se ha diseñado atendiendo a tres requisitos. En primer lugar, cada afirmación relevante debe poder asociarse a un artefacto concreto. En segundo lugar, los datos originales deben mantenerse separados de los resúmenes derivados para que sea posible volver a analizarlos. Finalmente, la campaña debe poder repetirse sin depender de una secuencia manual no documentada. Por este motivo, cada fase dispone de su propio directorio y el nivel superior contiene los resultados comparativos, las herramientas de automatización y un inventario global de integridad.

A.1. Estructura general del directorio

La organización principal se muestra en el Listado A.1. Se han omitido los nombres individuales de las evidencias para mantener legible el árbol; estos se detallan en las secciones posteriores.

Código A.1: Estructura simplificada del directorio de evidencias

```
evidencias/  
|-- fase0/  
|   |-- capturas/  
|   |-- json/  
|   |-- pcap/  
|   |-- README.md  
|   '-- SHA256SUMS.txt  
|-- fase1/  
|   |-- capturas/  
|   |-- json/  
|   |-- logs/  
|   |-- pcap/  
|   |-- README.md  
|   '-- SHA256SUMS.txt  
|-- fase2/  
|   |-- capturas/  
|   |-- json/  
|   |-- logs/  
|   |-- pcap/  
|   |-- README.md  
|   '-- SHA256SUMS.txt  
|-- resultados/  
|-- scripts/  
|-- README.md  
'-- SHA256SUMS.txt
```

Los directorios `fase0`, `fase1` y `fase2` responden a la evolución del sistema descrita en la memoria. Esta separación evita mezclar la evidencia de una vulnerabilidad de la línea base con el rechazo producido por una fase posterior. Los ficheros `README.md` actúan como inventarios breves orientados a la consulta directa desde el repositorio, mientras que este apéndice ofrece una visión conjunta y explica las relaciones entre los distintos formatos.

A.1.1. Convención de nombres

Los artefactos específicos de un escenario siguen, siempre que resulta aplicable, el patrón `faseN_XX_descripcion.ext`. El primer campo identifica la fase; el segundo establece el orden de adquisición dentro de ella; y el tercero resume

el comportamiento observado. Por ejemplo, `fase2_02_fallos_auth_y_ban_device.pcap` corresponde al segundo escenario de red de Fase 2 y documenta el bloqueo de dispositivo tras una ráfaga de autenticaciones incorrectas.

Los ficheros cuyo índice es 00 agregan varios escenarios. No constituyen una nueva ejecución, sino una vista conjunta destinada a facilitar la apertura en Wireshark. Los nombres terminados en `_http.json` identifican resúmenes generados a partir de un PCAP.

A.1.2. Tipos de artefacto

La Tabla [A.1](#) resume la función de cada formato. La campaña combina fuentes diferentes porque una captura visual, por sí sola, no permite demostrar qué solicitud ha aceptado el servidor, y un código HTTP aislado tampoco acredita que el vídeo se haya reproducido realmente.

Tabla A.1: Tipos de artefacto conservados

Formato	Ubicación habitual	Finalidad
PNG	<code>capturas/</code>	Acreditar el resultado visible: reproducción, rechazo, puntuación o bloqueo mostrado por el laboratorio.
PCAP / PCAPNG	<code>pcap/</code>	Conservar el tráfico HTTP observado en el punto de entrada de la fase y permitir un análisis posterior con Wireshark o TShark.
JSON HTTP	<code>json/*_http.json</code>	Ofrecer una vista consultable del PCAP. Incluye recuentos, rutas, métodos y códigos.
JSON de estado	<code>fase2/json/</code>	Conservar sesiones, riesgo, bans y eventos de Redis asociados a un escenario con estado.
CSV / JSON	<code>resultados/</code>	Recoger la matriz de pruebas y las métricas en formatos adecuados tanto para lectura automática como para inspección manual.
LOG	<code>logs/</code>	Mantener la salida de los planos de control, servidores internos de licencia y laboratorios de CDN leeching.
SHA-256	<code>SHA256SUMS.txt</code>	Permitir la comprobación de integridad de los artefactos después de su adquisición.

Las capturas y los PCAP se consideran evidencias primarias de interfaz y red. Los JSON terminados en `_http` son artefactos derivados, ya que se generan leyendo los PCAP con TShark. Las matrices de resultados también son derivadas, pero conservan para cada caso la condición esperada, el valor observado y el resultado booleano. Esta distinción permite volver a producir los resúmenes sin modificar la captura original.

A.2. Evidencias de Fase 0

El directorio `fase0` contiene la línea base de la auditoría descrita en el Capítulo 4. Se han conservado tres escenarios, cada uno con captura visual, PCAP y resumen HTTP. La Tabla A.2 presenta su correspondencia.

Tabla A.2: Escenarios documentados en Fase 0

Escenario	Artefactos principales	Comportamiento acreditado
Flujo oficial Widevine	<code>fase0_01_*</code>	El usuario autorizado inicia sesión, obtiene la configuración y solicita dos licencias por la ruta oficial antes de reproducir.
Bypass Widevine	<code>fase0_02_*</code>	El cliente externo alcanza <code>/license/no_auth</code> sin login ni configuración de reproducción previa.
ClearKey con clave conocida	<code>fase0_03_*</code>	El MPD, las inicializaciones y los primeros segmentos se recuperan sin invocar una ruta de licencia, y la reproducción externa resulta visible.

El fichero `fase0_00_auditoria_completa.pcapng` agrega los tres PCAP para facilitar una revisión conjunta. Los archivos individuales se mantienen porque permiten aplicar filtros sin mezclar los caminos oficial, alternativo y ClearKey. En el escenario ClearKey cada recurso puede aparecer en ambos lados de Varnish: una entrada corresponde a la solicitud recibida y otra al reenvío hacia el origen. Esta duplicación deriva del punto de captura y no representa dos reproducciones independientes.

La carpeta `fase0/json` contiene los resúmenes seguros de las capturas. Estos incluyen número de trama, tiempo, IP de origen y destino, método, host, URI, código de respuesta y longitud declarada.

A.3. Evidencias de Fase 1

Fase 1 incorpora autorización sobre el manifiesto, el contenido local y la licencia. Su directorio conserva dos bloques de evidencia. El primero corresponde al laboratorio general y el segundo a la ampliación de Key Leak y CDN leeching.

La evidencia `fase1_01_laboratorio_protegido.pcap` registra el login, la creación de sesión, la obtención del manifiesto, dos solicitudes de licencia, cuatro casos negativos y la parada. La captura `fase1_01_pruebas_manifest.png` muestra en la interfaz los rechazos 401, 401, 409 y 403 correspondientes, respectivamente, a ausencia de token, token alterado, sesión distinta y activo diferente. El resumen HTTP permite comprobar la misma distribución sin abrir Wireshark.

El segundo bloque utiliza el activo `local-cenc-clearkey.fase1_02_cdn_leeching.pcap` demuestra que el acceso anónimo queda rechazado y que una sesión válida puede recuperar contenido desde un cliente externo controlado. El JSON de cada escenario conserva los códigos, el volumen servido, las latencias de los objetos de contenido, el resultado de CORS y la ausencia de solicitudes de licencia. Las pruebas de contexto añaden la segunda sesión, la instancia copiada, el cruce de activo y el acceso después de detener la reproducción.

La carpeta `logs` completa la trazabilidad con la salida de `control-plane`, `license-server` y `key-leak-lab`. Estos registros son útiles para confirmar el arranque y detectar errores de aplicación, pero no sustituyen al PCAP: describen la perspectiva de cada proceso y no necesariamente todo el tráfico observado en el frontal.

A.4. Evidencias de Fase 2

Fase 2 conserva los controles preventivos y añade estado en Redis, puntuación de riesgo y respuesta. Por ello, sus escenarios combinan cuatro perspectivas: interfaz, red, estado exportado y logs. Esta fase contiene tres bloques principales.

El escenario `fase2_01` documenta la concurrencia y el ban de cuenta. Se crea una sesión y se realizan cinco intentos adicionales, que devuelven 409. El JSON de estado conserva la progresión de riesgo 0, 25, 50, 75 y 100, los eventos `risk.incremented`, la creación del ban, su retirada administrativa y la parada final. El PCAP permite contrastar que se han producido una respuesta 201 y cinco conflictos para `/playback/session`.

El escenario `fase2_02` documenta ocho autenticaciones incorrectas desde el

mismo dispositivo. La evidencia recoge las respuestas 401, la creación de `AUTH_FAILURE_BURST`, el rechazo posterior del manifiesto y la recuperación tras retirar el ban. En este caso la respuesta se aplica directamente al dispositivo y no depende de que exista una cuenta autenticada desde la que acumular puntuación.

El escenario `fase2_03` corresponde a Key Leak y CDN leeching. La captura positiva muestra que una única sesión válida, un token vigente y la clave conocida permiten reproducir el activo local. El estado exportado conserva 253.023 bytes servidos sin solicitud de licencia, el evento `key_leak.pattern_detected` y el incremento de 20 puntos aplicado una sola vez a la sesión. La segunda captura reúne los controles de CORS, concurrencia, instancia, activo y ausencia de token.

A.5. Resultados comparativos

El directorio `resultados` concentra los datos que alimentan las tablas y la discusión del Capítulo 7. La Tabla A.3 resume su contenido.

Tabla A.3: Ficheros del directorio de resultados

Fichero	Contenido
<code>resultados_funcionales.json/.csv</code>	Matriz de 65 comprobaciones generales, con identificador, fase, categoría, condición esperada, observación y resultado.
<code>resultados_cdn_leeching.json/.csv</code>	Batería complementaria de 27 comprobaciones y resúmenes específicos de las tres fases.
<code>metricas_raw.csv</code>	240 observaciones temporales individuales, obtenidas con 20 repeticiones por combinación de fase y operación.
<code>metricas_resumen.json/.csv</code>	Media, mediana, percentil 95, mínimo, máximo y desviación típica calculados a partir de las observaciones.
<code>entorno_ejecucion.json</code>	Plataforma, versión de Docker, revisión Git, contenedores activos y consumo puntual de CPU y memoria.
<code>incidencia_binding_licencia_fase2.json</code>	Registro de la desviación encontrada en <code>/license</code> , su corrección y el resultado de la regresión posterior.

La conservación de la incidencia resulta especialmente importante. La primera ejecución general superó 64 de 65 casos y descubrió que Fase 2 no contrastaba la cabecera de sesión antes de reenviar una licencia. Después de corregirla, la regresión completa terminó en 65 de 65. El fichero de incidencia evita que el resultado final oculte el defecto encontrado durante la propia evaluación y documenta cómo la batería ha influido en la implementación.

La batería complementaria se mantiene separada de la general. De esta forma, las 240 mediciones temporales y los 65 casos previos no se han recalculado al introducir CDN leeching. El resultado agregado de 92 comprobaciones se obtiene sumando ambas campañas, pero cada una conserva su fecha, su estructura y sus variables específicas.

A.6. Scripts de adquisición y tratamiento

Los seis programas del directorio `scripts` automatizan tareas distintas. No se ha reunido todo en un único ejecutable para poder regenerar un resumen o las huellas sin repetir las pruebas de reproducción. Su función se recoge en la Tabla [A.4](#).

Tabla A.4: Herramientas de reproducción y tratamiento de evidencias

Script	Entrada o requisito	Salida principal
<code>run_evaluation.py</code>	Fases levantadas; modos <code>functional</code> o <code>metrics</code> .	Matriz funcional y CSV/JSON de métricas. Reinicia el estado cuando el escenario lo requiere.
<code>run_cdn_leeching.py</code>	Fase concreta o <code>all</code> ; opción <code>--skip-reset</code> .	27 comprobaciones complementarias y resúmenes de contenido, CORS, contexto y riesgo.
<code>export_phase2_stats.py</code>	Nombre del escenario y Redis de Fase 2 activo.	JSON con riesgo, bans, sesiones y eventos presentes en Redis.
<code>summarize_pcaps.py</code>	PCAP existentes y contenedor <code>nicolaka/netshoot</code> .	JSON HTTP seguro, recuentos de solicitudes/respuestas y SHA-256 de la captura fuente.
<code>collect_environment.py</code>	Docker activo y contenedores <code>tfm-*</code> .	Inventario del entorno, estadísticas puntuales y logs normalizados.
<code>generate_hashes.py</code>	Campaña ya finalizada.	Inventarios <code>SHA256SUMS.txt</code> de Fase 1, Fase 2 y del conjunto.

A.6.1. Orden recomendado de ejecución

Una repetición completa parte de las fases Docker levantadas y disponibles. El orden recomendado se muestra en el Listado A.2. Los comandos deben ejecutarse desde la raíz del repositorio.

Código A.2: Secuencia para regenerar los resultados derivados

```
python evidencias/scripts/run_evaluation.py functional
python evidencias/scripts/run_evaluation.py metrics --iterations
20
python evidencias/scripts/run_cdn_leeching.py all
python evidencias/scripts/summarize_pcaps.py
python evidencias/scripts/collect_environment.py
python evidencias/scripts/generate_hashes.py
```

La adquisición de los PCAP no se inicia automáticamente dentro de esta secuencia. Debe arrancarse antes del escenario correspondiente, compartiendo el espacio de red del contenedor CDN, y detenerse después de que finalicen las solicitudes. Cuando el estado ya se ha reiniciado y se desea evitar tráfico de arranque dentro del PCAP, `run_cdn_leeching.py` admite `--skip-reset`. Esta opción no omite controles funcionales; únicamente evita reiniciar de nuevo el servicio justo después de iniciar la captura.

El orden también responde a la dependencia entre artefactos. `summarize_pcaps.py` necesita que las capturas ya existan; `collect_environment.py` debe ejecutarse con los contenedores activos; y `generate_hashes.py` debe ser el último paso, pues cualquier modificación posterior cambiaría las huellas. Si se repite una prueba, deben regenerarse los resúmenes y los inventarios relacionados para no conservar hashes o recuentos de una versión anterior.

A.7. Integridad

Los inventarios `SHA256SUMS.txt` contienen una huella SHA-256 por fichero. Existen inventarios específicos en Fase 1 y Fase 2, además del inventario global. Fase 0 conserva también su propio fichero de huellas. Estos valores no demuestran por sí solos quién ha adquirido una evidencia, pero permiten comprobar que el contenido revisado posteriormente coincide bit a bit con el utilizado durante la redacción.

A.8. Método de consulta y trazabilidad

Para revisar un resultado se recomienda seguir una secuencia de tres niveles. Primero se consulta el registro general en `resultados`, donde aparece el identificador de prueba, la condición esperada y el resultado. Después se abre el JSON HTTP o el estado de Fase 2 para localizar el código, evento o cambio de riesgo. Finalmente, cuando sea necesario comprobar la secuencia exacta, se revisa el PCAP fuente y la captura de interfaz asociada.

Esta triangulación permite diferenciar tres afirmaciones que no son equivalentes: que la interfaz muestre un rechazo, que el servidor devuelva efectivamente un 401 o 409 y que el estado antifraude conserve una señal. En Fase 2, por ejemplo, la captura de concurrencia documenta el resultado visible; el PCAP acredita las cinco respuestas 409; y el JSON de Redis conserva los incrementos hasta 100 y la creación del ban. Del mismo modo, en CDN leeching la captura positiva demues-

tra reproducción, el PCAP conserva los objetos entregados y el estado exportado muestra la puntuación aplicada sin solicitud de licencia.

Los README de cada fase ofrecen filtros de Wireshark y recuentos rápidos. Se mantienen como documentación próxima al dato, mientras que el Capítulo 7 contiene la comparación y este apéndice explica el sistema de conservación. Esta separación evita que una modificación en el inventario obligue a reescribir la interpretación académica, pero mantiene una ruta clara desde cada conclusión hasta su evidencia.

A.9. Alcance y limitaciones de la evidencia

La campaña se ha ejecutado en un único equipo, sobre HTTP local y con contenedores Docker. Las marcas temporales, estadísticas de CPU y consumo de memoria describen ese entorno concreto. No constituyen una prueba de carga, una certificación de seguridad ni una estimación directa del comportamiento de una plataforma OTT comercial.

Parte del flujo Widevine depende de los activos y del servicio CWIP público de Shaka. El PCAP local permite estudiar el acceso al plano de control y al proxy, pero no descifra el tráfico HTTPS externo. El activo CENC local se ha incorporado precisamente para observar de forma controlada la entrega de manifiesto y segmentos cuando la clave de laboratorio ya es conocida.

Finalmente, la reproducibilidad no implica que dos ejecuciones tengan tiempos idénticos. La planificación de Docker, el estado del equipo y la disponibilidad del servicio externo pueden variar. Lo que debe mantenerse es la semántica de los casos: recursos autorizados o rechazados con los códigos previstos, creación de eventos, evolución de la puntuación y aplicación o retirada de los bloqueos. Las evidencias conservadas permiten comprobar esas propiedades y volver a calcular los resúmenes sin depender exclusivamente de las tablas incluidas en la memoria.

Bibliografía

- [1] Motion Picture Association, “2021 theme report,” 2022, consultado el 13-06-2026. [Online]. Available: <https://www.motionpictures.org/wp-content/uploads/2022/03/MPA-2021-THEME-Report-FINAL.pdf>
- [2] Ofcom, “Media nations 2025: Uk report,” 2025, consultado el 13-06-2026. [Online]. Available: <https://www.ofcom.org.uk/siteassets/resources/documents/research-and-data/multi-sector/media-nations/2025/media-nations-2025-uk-report.pdf>
- [3] Ericsson, “Mobile data traffic outlook,” <https://www.ericsson.com/en/reports-and-papers/mobility-report/dataforecasts/mobile-traffic-forecast>, 2025, ericsson Mobility Report, consultado el 13-06-2026.
- [4] W3C Media Working Group, “Encrypted media extensions,” <https://www.w3.org/TR/encrypted-media-2/>, 2026, w3C Working Draft, 11 June 2026, consultado el 13-06-2026.
- [5] Google for Developers, “Widevine drm overview,” <https://developers.google.com/widevine/drm/overview>, 2024, Última actualización visible: 2024-10-09; consultado el 13-06-2026.
- [6] S. Delaune, J. Lallemand, G. Patat, F. Roudot, and M. Sabt, “Formal security analysis of widevine through the w3c eme standard,” in *33rd USENIX Security Symposium*, 2024, consultado el 13-06-2026. [Online]. Available: <https://www.usenix.org/conference/usenix-security24/presentation/delaune>
- [7] Android Developers, “Digital rights management,” <https://developer.android.com/media/media3/exoplayer/drm>, 2026, consultado el 13-06-2026.
- [8] Mozilla Support, “Watch drm content on firefox,” <https://support.mozilla.org/en-US/kb/enable-drm>, 2022, consultado el 13-06-2026.
- [9] G. Patat, M. Sabt, and P.-A. Fouque, “Exploring widevine for fun and profit,” in *16th USENIX Workshop on Offensive Technologies (WOOT ’22)*, 2022.
- [10] —, “Your drm can watch you too: Exploring the privacy implications of browsers (mis)implementations of widevine eme,” *Proceedings on Privacy Enhancing Technologies*, 2023.
- [11] Google Cloud, “Use signed requests,” <https://cloud.google.com/media-cdn/docs/signed-requests>, 2026, consultado el 13-06-2026.
- [12] Amazon Web Services, “Serve private content with signed urls and signed cookies,” <https://docs.aws.amazon.com/AmazonCloudFront/latest/DeveloperGuide/PrivateContent.html>, 2026, consultado el 13-06-2026.
- [13] Akamai, “Token authentication,” <https://techdocs.akamai.com/adaptive-media-delivery/docs/add-token-auth>, 2026, consultado el 13-06-2026.

-
- [14] Adobe Experience League, “Concurrency monitoring introduction,” <https://experienceleague.adobe.com/en/docs/pass/cm/cm-home>, 2026, consultado el 13-06-2026.
 - [15] A. Perinkolam, B. Auxier, and A. Rawat, “Impatience and price motivate consumers to watch pirated content and borrow svod passwords,” <https://www.deloitte.com/us/en/insights/industry/technology/digital-media-trends-consumption-habits-survey/2024/password-sharing-pirated-content.html>, 2024, deloitte Insights, consultado el 13-06-2026.
 - [16] G. Simon and G. Doërr, “Next-generation access tokens to fight cdn leeching,” in *MHV ’24: Mile-High Video Conference*, 2024.
 - [17] International Organization for Standardization, *ISO/IEC 23009-1:2022: Information technology – Dynamic adaptive streaming over HTTP (DASH) – Part 1: Media presentation description and segment formats*, ISO/IEC Std., 2022. [Online]. Available: <https://www.iso.org/standard/83314.html>
 - [18] DASH Industry Forum, “About MPEG-DASH,” <https://dashif.org/about/>, 2026, accedido en 2026.
 - [19] R. Pantos and W. May, “HTTP Live Streaming,” Internet Engineering Task Force, RFC 8216, 2017. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc8216>
 - [20] Google, “Shaka Packager Documentation,” <https://shaka-project.github.io/shaka-packager/html/documentation.html>, 2026, accedido en 2026.
 - [21] —, “Shaka Packager DRM Tutorial,” <https://shaka-project.github.io/shaka-packager/html/tutorials/drm.html>, 2026, accedido en 2026.
 - [22] L. M. V. Rodrigues, D. S. Menasché, A. Serra, and A. A. d. A. Rocha, “Minimally Intrusive Access Management to Content Delivery Networks based on Performance Models and Access Patterns,” 2024. [Online]. Available: <https://arxiv.org/abs/2410.05642>
 - [23] Apple Developer, “FairPlay Streaming,” <https://developer.apple.com/streaming/fps/>, 2026, accedido en 2026.
 - [24] A. Rafi, C. Shepherd, and K. Markantonakis, “A First Look at Digital Rights Management Systems for Secure Mobile Content Delivery,” 2023. [Online]. Available: <https://arxiv.org/abs/2308.00437>
 - [25] Microsoft, “PlayReady Product Suite,” <https://www.microsoft.com/playready/>, 2026, accedido en 2026.
 - [26] World Wide Web Consortium, “cencInitialization Data Format,” <https://w3c.github.io/en-crypted-media/format-registry/initdata/cenc.html>, 2026, w3C Editor’s Draft / Registry.
 - [27] OWASP Foundation, “Credential Stuffing,” https://owasp.org/www-community/attacks/Credential_stuffing, 2026, accedido en 2026.
 - [28] —, “Session Hijacking Attack,” https://owasp.org/www-community/attacks/Session_hijacking_attack, 2026, accedido en 2026.
 - [29] T. Madushanka, D. S. Kumara, and A. A. Rathnaweera, “SecureRights: A Blockchain-Powered Trusted DRM Framework for Robust Protection and Asserting Digital Rights,” 2024. [Online]. Available: <https://arxiv.org/abs/2403.06094>
 - [30] Docker Inc., “What is docker?” <https://docs.docker.com/get-started/docker-overview/>, 2026, consultado el 13-07-2026.
 - [31] Python Software Foundation, “Python documentation,” <https://docs.python.org/3/>, 2026, consultado el 13-07-2026.

- [32] Wireshark Foundation, “Wireshark documentation,” <https://www.wireshark.org/docs/>, 2026, consultado el 13-07-2026.
- [33] D. Brahmbhatt and M. Stamp, “Digital rights management for streaming media,” in *Handbook of Research on Secure Multimedia Distribution*, S. Lian and Y. Zhang, Eds. IGI Global, 2009, pp. 22–38.
- [34] LALIGA, “Nota informativa en relación con el bloqueo de IPs durante las últimas jornadas de LALIGA EA SPORTS,” <https://www.laliga.com/es-GB/noticias/nota-informativa-en-relacion-con-el-bloqueo-de-ips-durante-las-ultimas-jornadas-de-laliga-ea-sports-vinculadas-a-las-practicas-ilegales-de-cloudflare>, 2025, consultado el 31-08-2026.